BZ_AutoDrive — Manual del administrador

> Framework de conducción de vehículos por NPC para DayZ. Versión didáctica para admins/modders. **Open source (MIT):** el código, este manual y el AI knowledge pack son públicos — descargalo, usalo en tu server, forkealo, extendelo y compartí tus rutas. *Manual 1.3 — guía práctica del administrador.*

Contenido

1. ¿Qué es BZ_AutoDrive?
2. La idea, en una frase
3. El modelo de uso: bajalo, adaptalo, es tu mod
4. Requisitos
5. Inicio rápido — tu primera ruta
6. El wizard en detalle 6B. Los 4 archivos que genera el wizard 6C. El envelope — lo que el framework aprende de tus tomas
7. Cómo grabar bien
8. El servicio de bus permanente
9. Cómo maneja el NPC (sigue tu línea y tu velocidad) 9B. DayZ 1.30 — qué cambió en los vehículos, y las motos *(experimental)*
10. Eventos y secuencias
11. Misiones — integración con Quests
12. Controles (teclas)
13. La interfaz (UI)
14. Audio
15. Si algo no anda
16. Ideas
17. Alcance y límites
18. Frontera abierta

- Apéndice A — Referencia completa de la config de ruta
- Apéndice B — Para desarrolladores (extender con código)

---

1. ¿Qué es BZ_AutoDrive?

BZ_AutoDrive hace que un **NPC maneje un vehículo de DayZ de forma autónoma**, siguiendo una ruta que **vos grabaste manejando**. No es teleport ni movimiento "falso": el NPC realmente **acelera, frena y gira**, usando la física real del juego — el auto se comporta como cuando lo manejás vos.

¿Para qué te sirve? Para cualquier cosa que necesite un vehículo manejado por IA:

- Un **bus de línea** que recorre la costa parando en cada pueblo.

- Un **taxi** que lleva jugadores de un punto a otro.
- Un **convoy de refuerzos** que llega a una zona de misión y despliega bots.
- Una **patrulla** motorizada que da vueltas por una zona.

> *Ejemplo:* querés un colectivo que una tres pueblos. Lo manejás vos **una sola vez** (grabás el recorrido), y a partir de ahí un NPC lo repite solo, las veces que quieras, sin que vos estés.

---

2. La idea, en una frase

> **Vos manejás la ruta una vez; el framework aprende tu línea y lee cómo es el vehículo; después un NPC la maneja.**

Lo potente está en el "lee cómo es el vehículo": BZ_AutoDrive **lee el config del vehículo** (su dirección, sus marchas, su motor) y maneja según eso. Por eso **el framework maneja cualquier vehículo sin setup por-auto**: no tenés que configurar nada por auto para que ande.

Y hay una segunda capa, que es la novedad de 1.2: además de **leer** lo que el config declara, el framework **mide** cómo se comporta ese vehículo de verdad — cuánto frena, cuánto gira con cada ángulo de volante, si se arrastra solo en ralentí. Eso lo aprende **de tus propias tomas**, sin que hagas nada, y lo guarda en el *envelope* de esa clase (§6C). Cuando lo declarado y lo medido no coinciden, **manda lo medido**.

> *Ojo con la grabación:* una grabación captura la corrida de **su** vehículo (su *fingerprint*, su freno medido, su marcha) — **es de ese auto**. ¿Querés la **misma ruta** en otro vehículo? **Grabala con ese vehículo.** No es una limitación: es la forma en que el framework trabaja mejor, porque de cada toma aprende cómo se comporta **esa** clase (§6C) — cuánto frena, cuánto gira. Una toma prestada trae las mediciones del auto equivocado.

(Por qué importa: DayZ no tiene "manejá hasta acá" para autos IA — solo te da el volante y los pedales. BZ_AutoDrive es el piloto que faltaba.)

---

3. El modelo de uso: bajalo, adaptalo, es tu mod

BZ_AutoDrive es un **punto de partida**, no un producto cerrado. El arco para cualquier admin es siempre el mismo:

1. **Descargás** el framework (open source).
2. **Lo adaptás a tu server:** grabás *tus* rutas y configurás *tus* escenarios. El **motor de manejo ya viene integrado** — no programás cómo maneja el NPC, eso ya está resuelto.
3. **Te queda tu mod**, con el framework adentro + tu contenido, corriendo lo tuyo.

Lo que agregás depende de para qué lo quieras:

- **Servicio de bus / taxi** → grabás la(s) ruta(s) y las dejás corriendo (§8). Sin tocar código.

- **Sistema de misiones** → sumás los Quests + las rutas de convoy/escolta (§11). El framework maneja los vehículos de tus misiones.
- **Algo a medida** → lo extendés con código (Apéndice B) o construís tu propio mod sobre él como dependencia (B.6).

> La gracia del open source: **todos bajan el mismo motor y lo hacen suyo.** Uno arma un bus de costa, otro un taxi urbano, otro una campaña de misiones — sin que ninguno tenga que reinventar el manejo.

> *Si lo repackageás como tu propio addon* (renombrarlo, tu Workshop): rebuildeás con tu propia key y copiás el `.bikey` a `keys/`. La licencia MIT lo permite — solo mantené los créditos.

---

4. Requisitos

- **DayZ server** con el mod **BZ_AutoDrive** cargado + su `.bikey` en `keys/`.
- **DayZ-Expansion, vía el Bundle** (`@DayZ-Expansion-Bundle`) — *imprescindible*. Trae las tres partes que el mod necesita: **AI (eAI)**, que provee el "cuerpo" del NPC chofer (el framework **conduce**, eAI pone el cuerpo); **Vehicles**; y **Quests**, que el mod **requiere para cargar** aunque no hagas misiones. ⚠ **No agregues `@DayZ-Expansion-AI` suelto**: si ya tenés el Bundle son las mismas clases en otra versión y el server no arranca.
- **El wizard** (`route_wizard.ps1`) corre en **tu PC** (fuera del juego), no en el server. Lo arrancás con doble clic en `tools\Wizard.bat` — no necesitás abrir PowerShell ni tipear nada.
- **Si agregás un pack de vehículos:** algunos packs necesitan **otro mod de base** (un *core*) que el Workshop **no descarga solo**, y sin él **el server se cae al arrancar**. Después de agregar un pack, buscá en el RPT `requires addon` : si aparece, instalá el mod que nombra y ponelo **antes** del vehículo en `-mod=` . Y acordate de que el **cliente** carga los mods de la lista de su launcher (DZSA), no del `start.bat` del server: si no lo tildás ahí también, no vas a ver los vehículos nuevos.
- **Python: ya viene incluido.** El conversor del wizard (`frame_to_route.py`), el que arma la ficha del vehículo (`envelope_emit.py`) y el import de v1 son Python, y el mod **trae su propio Python** en `tools\python\` : no tenés que instalar nada. *(Si esa carpeta faltara, el wizard usa el Python de tu PC si lo encuentra en el PATH, y si no hay ninguno te lo dice al abrir.)*
- **Qué corre por defecto: no tenés que activar nada.** El cerebro de manejo 1.2, la cinta y la ley de volante nueva vienen **encendidos**. Si por algún motivo querés el comportamiento anterior en tu servidor, se apaga con un archivo en `profiles\BZ_AutoDrive\` **con cualquier texto adentro**: `CEREBRO_11.txt` vuelve al cerebro 1.1, `BZ12_SIN_CINTA.txt` corre sin cinta, `BZ12_SIN_FF_PRIMARIO.txt` vuelve a la ley de volante anterior. **Un archivo vacío no hace nada.** Una ruta que no tiene `_cinta.csv` (por ejemplo, una importada de v1) corre sin cinta y lo avisa en el log. El filtro de cliente (§13.4) también viene encendido.

---

5. Inicio rápido — tu primera ruta

Vamos a armar una ruta de punta a punta. *Ejemplo que seguimos: un bus que va de la terminal de un pueblo a otro.*

Paso 0 — Revisá tus teclas *(hacelo ANTES de nada)*

El framework usa **solo 3 teclas**, ya asignadas por default. Entrá a **Opciones → Controles → categoría "BZ AutoDrive"** y verificalas:

Acción (en el menú)	Default
Open Control Panel	`INICIO` (Home)
Record (start/stop)	`NUMPAD 5`
Mark Event / Stop	`NUMPAD 4`

> ⚠ **Lo primero: fijate que no choquen con tus teclas.** Si ya tenés `INICIO` o `NUMPAD 4/5` asignadas a **otro mod** (o a una acción de DayZ), se van a **pisar** y algo no va a responder — y **DayZ no te avisa del conflicto**. Por eso, antes de grabar: abrí esa categoría, mirá que las 3 no colisionen con lo tuyo, y **reasigná** la que haga falta (clic en la tecla → apretás la nueva). Es un minuto y te ahorra el clásico "*grabo y no pasa nada*" o "*aprieto INICIO y no abre el panel*".

*(Estas teclas solo tienen efecto para el **admin** —§13.1— aunque todos los jugadores las VEN en su menú de Controles. Detalle completo de los controles en §12.)*

Paso 1 — Grabá la ruta (la manejás vos)

Subite al vehículo y manejá el recorrido **como querés que lo haga el NPC**.

- **NUMPAD 5** → empezá a grabar. Manejá tranquilo, a la velocidad que querés que vaya.
- **NUMPAD 4** → tocalo en cada **parada o punto importante** (la terminal, una esquina donde pasa algo). Marca ese punto para después poder colgarle eventos.
- **NUMPAD 5** de nuevo → terminá de grabar.

> Queda un archivo con tu recorrido: tu **línea** + tu **velocidad** + **dónde parás**. Eso es la materia prima.

> 🔑 **Lo más importante que pasa al grabar (y que no ves):** ese mismo NUMPAD 5, además de tu recorrido, **le saca una foto completa al vehículo** — su *fingerprint*. Queda en un `header_*.txt` al lado de tu grabación, y trae **todos los datos del auto, solos**:
>
> - el **classname**, el **wheelbase** (distancia entre ejes) y el **ángulo de dirección** → de ahí se calcula el **R_min** (el radio más chico que puede girar),
> - la cantidad de **marchas**, la **masa**, las **RPM** del motor,
> - y la lista **real de sus partes** (ruedas, puertas, batería, radiador, bujía…).
>
> Por eso **no configurás nada del vehículo a mano**: el wizard y el converter leen ese fingerprint y arman la ruta a su medida — calculan cuánto puede doblar, le ponen los attachments correctos (*no se adivinan, son los que el vehículo tenía*) y lo manejan según *su* física. Es el corazón del "config como manual de manejo".
>
> *Ejemplo real:* grabamos un UAZ-452 y de **un solo NUMPAD 5** salió todo: `UAZ_452`, wheelbase 2.53 m, R_min 3.9 m, 6 marchas, masa 2860 kg, + sus 4 ruedas (con la de auxilio), 6 puertas y batería/radiador/bujía. Cero configuración, cero adivinar.

Paso 2 — Convertí la grabación (el wizard)

El wizard es un **conversor**: agarra tu grabación cruda y la **convierte** en una ruta lista y **desplegada** en el Reproductor. Guarda tu manejo **por posición** —qué volante, acelerador, freno y marcha tenías en cada punto del camino: la *cinta*— y actualiza la **ficha medida** del vehículo (§6C). El NPC parte de eso y lo corrige según lo que el vehículo puede. *No calibra nada a mano: no hay ajustes por ruta.*

Arrancá el wizard con **doble clic en** `tools\Wizard.bat` (no hace falta abrir PowerShell ni tipear nada), elegís **[1] Convertir**, elegís tu grabación y le ponés un **nombre** — y te deja la ruta lista.

> **La primera vez** que abrís el wizard te pide tus **paths** — la carpeta de rutas de tu server (`RoutesDir`) y dónde están tus grabaciones. Los seteás **una sola vez**, el wizard se los acuerda, y después los cambiás cuando quieras con **[6] Configurar paths** (detalle en §6).

> *¿Por qué este paso?* La grabación cruda es un registro **por tiempo**. El NPC necesita tu manejo **por posición** (qué hacías en cada punto del camino, aunque él vaya un poco más lento o más rápido) y la ficha medida del vehículo. El wizard arma las dos cosas. *¿Dónde está mi grabación?* Vive en el **cliente**, la PC donde manejaste: `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\` . El wizard la busca ahí sola. La ruta convertida se **deploya** a tu carpeta de rutas del server (`RoutesDir`). Todo el detalle —el "trío" de archivos y las carpetas— en **§6**.

Paso 3 — Deploy (ya lo hace el wizard)

Al convertir, el wizard arma la ruta, actualiza la ficha del vehículo y deja todo **desplegado y listo en el Reproductor** (carga en caliente, **sin reiniciar el server**). No editás ningún archivo a mano. Si configuraste un segundo server en **[6] Configurar paths**, el wizard copia la ruta también allá.

Paso 4 — Probalo (que el NPC la maneje)

Abrí el **Reproductor** (la UI del admin, tecla `INICIO`) → elegí tu ruta en la lista → apretá **LOAD & SPAWN**: el NPC aparece y arranca a manejar. **Sin reiniciar el server.**

> *Ejemplo:* elegís "Bus pueblo" en la lista, apretás LOAD & SPAWN y ahí nomás el colectivo sale de la terminal, manejado por el NPC, parando donde marcaste.

Eso es todo el ciclo: `grabás → wizard → deploy → play` . El resto del manual es para sacarle más jugo a cada parte.

---

6. El wizard en detalle — tu herramienta central

El **wizard** (`tools\route_wizard.ps1`) convierte tu grabación cruda en una ruta lista, desplegada en el Reproductor. Es un menú interactivo (TUI) — corre en tu PC, fuera del juego. *(Es un **conversor**: no calibra ni te hace un cuestionario — el manejo sale de leer el config del vehículo.)*

> **Arrancalo con doble clic en** `tools\Wizard.bat` — abre el wizard directo, sin abrir PowerShell ni tipear comandos. (El `.bat` usa `-ExecutionPolicy Bypass`, que **solo afecta esa ejecución** — no cambia la política de tu sistema.)

> **Principio: todo pasa por el wizard.** El wizard corre las sub-herramientas (`frame_to_route.py`, `envelope_emit.py`, `frame_merge.py`) **por vos**, en orden. Vos **no** editás el JSON a mano ni corrés los scripts sueltos — el wizard es la única puerta de entrada.

⚠ Dónde se aloja cada toma (leé esto)

Cuando grabás (NUMPAD 5), el archivo **no queda donde lo buscarías intuitivamente**. La regla de oro:

- La **grabación humana** (`path_*.csv` / `frame_*.csv`, la que hacés con NUMPAD 5) cae en el **cliente** (la PC donde manejaste): `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`.
- Las **corridas de Boris** (`ai_run_*.csv` y `boris_native_*.csv`) se arman con los **checks del Reproductor** (§13, ya **no con teclas**) y caen en el **server que corrió**, no en el cliente. *Son mediciones para diagnosticar/comparar, no tomas para convertir (§15.1–16.2).*

El wizard busca tus grabaciones en el **cliente** automáticamente, así que normalmente la encuentra sola. Si corrés el server en otra PC, también podés apuntarle a esa carpeta:

Lugar	Carpeta
Cliente (donde manejás)	`%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`
Server (si grabaste ahí)	`<server>\profiles\BZ_AutoDrive_PathLogger\`

> Si grabaste y "no aparece", fijate en qué máquina estabas — cae en el cliente de esa PC.

Dos archivos más que conviene conocer:

- Junto a cada grabación hay un `header_*.txt` = el **fingerprint del vehículo** (dirección, marchas, wheelbase). El wizard lo **lee** para armar la ruta a la medida de ese auto (no lo calibra: lo lee).
- Las **rutas terminadas** (los cuatro archivos de §6) viven en tu carpeta de rutas del server (`RoutesDir` — ver `[6]`).

El menú del wizard

Arrancás el wizard (doble clic en `Wizard.bat`) y te aparece el menú:

> **[1] Convertir y binarizar · [2] Importar ruta de BrigadaZ Transport v1 · [3] Merge de tramos (continuar grabación) · [B] Binarizar rutas · [6] Configurar paths · [Q] Salir**

[1] Convertir y binarizar — de grabación a ruta lista

Agarrás una grabación cruda (`frame_*.csv` + su `header_*.txt` apareado) y el wizard la vuelve una **ruta desplegada y lista en el Reproductor**, en un paso. Lo único que te pide es un **NOMBRE** → la ruta sale como `BZBusRoute_<nombre>` y aparece con ese nombre en el Reproductor. En la lista de grabaciones, cada

una trae la etiqueta de la carpeta donde la encontró: `[Cl]` la del cliente, `[A]` la de grabaciones de tu server (`BZ_AutoDrive_PathLogger` , al lado de la carpeta de rutas) y `[B]` la del segundo server, si configuraste uno.

Qué genera y dónde va. De una grabación salen **cuatro archivos de la ruta**, todos en tu **carpeta de rutas** (`RoutesDir` — ver `[6]`):

Archivo	Qué es	¿Lo tocás?
`BZBusRoute_<nombre>_hdr.json`	el **header**: vehículo, medidas del vehículo, config de manejo y la lista de **eventos** de tus NUMPAD 4	**sí** — es el archivo que editás
`BZBusRoute_<nombre>_wp.csv`	los **waypoints**, uno por punto de la ruta (unos 900 en una ruta de 10 minutos)	no
`BZBusRoute_<nombre>_cinta.csv`	la **cinta**: tu manejo completo, muestreado fino a lo largo del recorrido —posición, rumbo, velocidad, volante, acelerador, freno y marcha— unas 12 veces más denso que los waypoints. Es de acá que sale la fidelidad: el NPC no interpola entre waypoints, reproduce esto	no
`BZBusRoute_<nombre>_cinta.bin`	el **binario de la cinta**: la misma cinta en el formato que el server lee más rápido. El wizard lo genera al convertir; sin él la ruta funciona igual, con el CSV (§6B)	no
`BZBusRoute_<nombre>.json`	el **master**: header + waypoints en un solo archivo	no, es respaldo

> **El par que manda es `_hdr.json` + `_wp.csv` .** Si los dos existen, el server carga ese par y **el `.json` completo queda sin usar**. Por eso los eventos van en el `_hdr.json` .

Y un quinto archivo, que no es de la ruta sino del vehículo:

Archivo	Qué es
`envelope_<clase de vehículo>.csv`	la **ficha medida de ese vehículo**: cuánto avanza solo sin acelerador, qué velocidad sostiene en cada curvatura, cómo frena. Se arma de tus tomas y **lo comparten todas las rutas de ese vehículo**

Dónde va. A la **misma carpeta que las rutas** (`RoutesDir` , tu `…\profiles\BZ_AutoDrive\`) — el wizard lo escribe ahí junto con los cuatro archivos. El nombre lleva **la clase del vehículo, no el de la ruta**: `envelope_BZ_ChernoBus_02.csv` , no `envelope_MI_RUTA.csv` . Por eso hay **uno por vehículo** y no uno por ruta: si grabás tres rutas con el mismo bus, las tres alimentan y usan la misma ficha.

La ficha se mide sola cuando terminás la toma como dice §7.1, y **el wizard no la pisa** en las tomas siguientes: cada toma nueva **suma** lo que aportó. Para rehacerla desde cero a propósito: `tools\python\python.exe tools\envelope_emit.py "<toma.csv>" --out "<RoutesDir>" --pisar` (o respondé **[R] regenerar** cuando el wizard te lo pregunte: lo hace solo si el vehículo cambió desde que se midió su ficha).

> **Si movés una ruta a otro servidor:** una ruta convertida con esta versión **lleva adentro** el envelope de su vehículo (en el `_hdr.json`), así que alcanza con los cuatro archivos. Una ruta convertida antes

> necesita además el envelope de su vehículo; sin él, el framework arranca sin lo medido y usa los valores de fábrica hasta que junte tomas nuevas. Detalle de qué guarda y cómo se llena: **§6C**.

> **Mientras el NPC maneja**, el server escribe además dos cosas en `profiles\BZ_AutoDrive\`: `BZ12_LOG_<ruta>_<hora>.csv`, con el detalle de esa corrida, y `AR_log.csv`, con una fila por cada vez que el AutoRecovery tuvo que intervenir (§9.6). Los podés borrar cuando quieras.

> Te avisa que quedó **desplegada** (carga en caliente, **sin reiniciar**). Si configuraste un 2º server, **copia los archivos de la ruta** allá.

[B] Binarizar rutas — el binario de las rutas que ya tenías

Las rutas convertidas con una versión anterior no tienen `_cinta.bin`, y una cinta que volviste a convertir puede tener uno viejo. **Al abrir el wizard te avisa** si hay algo que hacer (`5 rutas sin binario o desactualizadas: [B] binarizar`); si no, no dice nada.

- `[B]` lista las rutas de tu carpeta con su estado: `al dia`, `sin binario` o `desactualizado` (con el motivo: cambió el CSV, de otra versión del mod, cortado o dañado).
- Elegís **una por número**, o `[T]` para todas las que faltan. Una ruta que no se puede convertir sale en rojo y sigue con la siguiente; **el CSV nunca se toca**.
- Si tenés un server espejo configurado en `[6]`, el binario se copia allá también, pero **sólo si allá está la misma cinta** (mismo tamaño); si no, avisa y el server espejo sigue con su CSV.
- **No es obligatorio:** sin binario la ruta carga igual, más lento, y el server lo dice en su RPT (`binario de la cinta descartado: se carga el CSV`).

[2] Importar ruta de BrigadaZ Transport v1

¿Venías usando **BrigadaZ Transport v1** y ya tenés un recorrido grabado? **No lo grabes de nuevo** — esta opción lo trae al formato de BZ_AutoDrive.

- **Dónde busca el archivo.** La ruta de v1 **no está en el PBO**: vive en el profile de **tu** server, y esa carpeta **puede llamarse de cualquier forma** (un repack/fork le cambia el nombre), así que el wizard **no la adivina: se la indicás vos** con **[B]** (y se la acuerda). Apuntale a tu `profiles\` y busca en **todas las subcarpetas**. También mira la carpeta `_importar` que te crea dentro de tu carpeta de rutas (buzón para el JSON que te traés de **otro** server), y con **[P]** pegás el path a un `.json` puntual.
- **De qué vehículo es.** Te pregunta la **identidad del vehículo** (`Wheelbase`, `Fingerprint`), que la toma de v1 no sabe. Te lista lo que ya tenés: cualquier **grabación** de ese vehículo (el `header_*.txt` — sirve una de **10 segundos**) o una **ruta ya convertida** de ese vehículo (esas además traen el freno medido). Pone primero la que coincide con el vehículo que declara tu ruta.
- **El perfil de obstáculos.** Te pregunta si Boris debe sortear autos parados en el camino: **Robusto**, **Interceptable** o **Ninguno** (§A.5b). ⚠ **Sólo tiene efecto con el cerebro 1.1** (`CEREBRO_11.txt`): el cerebro 1.2, el que corre por defecto, no esquiva; si algo lo traba, actúa el AutoRecovery (§9.6).
- **Qué te corrige.** v1 marcaba la parada donde apretabas la tecla, no donde el vehículo frenaba: las paradas declaradas **en movimiento** se planchan a 0 con la frenada pintada hacia atrás; colapsa los puntos repetidos de cuando estabas detenido; marca el final como parada.

> Lo que **no** migra: el perfil de manejo es por vehículo. Si tu recorrido es con otro auto, **grabalo con ese auto**: la grabación trae el perfil de *ese* auto, no de la ruta.

[3] Merge de tramos — continuar una grabación

¿Grabaste una ruta larga **en partes** (te desconectaste, paraste a descansar, la seguiste otro día)? Esta opción une esas partes en **una sola toma**:

- Elegís los `frame_*.csv` de las partes (dos o más, **en cualquier orden**): el wizard los encadena **por dónde empieza y termina cada una**, no por la hora. El final de una parte tiene que estar **a menos de 8 m** del comienzo de la siguiente, y todas tienen que ser **del mismo vehículo** y **de la misma versión**: una parte grabada con la 1.2 no se une con una de la 1.2.1 (la 1.2.1 graba más datos por cuadro).
- Una parte repetida (mismo comienzo y mismo final) se descarta con aviso; las marcas de NUMPAD 4 de cada parte se conservan.
- Le ponés un nombre a la toma unida y el wizard te ofrece **convertirla en ruta** en el acto (igual que **[1]**).

> Para que empalme bien, **empezá cada parte donde terminó la anterior**: pará el vehículo, cortá la grabación, y al retomar arrancá desde ese mismo punto.

[6] Configurar paths — las carpetas del server

El wizard **recuerda** las carpetas en `wizard_config.json` (al lado del script) para no pedírtelas cada vez. En la **primera corrida** te las pregunta, en el idioma que elegiste; la carpeta de rutas es **obligatoria** (no hay una por defecto y sin ella no avanza); después las cambiás acá cuando quieras *(Enter = mantener cada una)*:

Carpeta	Qué es
RoutesDir	dónde van las **rutas** del server. Es tu `…\profiles\BZ_AutoDrive\`. **La única imprescindible.**
Grabaciones del cliente	de dónde lee tus `frame_/path_*.csv` (por default `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\`).
2º server (mirror B)	si tenés un server de test aparte, el wizard copia la ruta allá al convertir. `-` = ninguno.
Rutas v1	dónde busca las tomas de BrigadaZ Transport v1 (para `[2]`).

> Las carpetas son **portables**: un modder con el server en otro disco lo setea **una vez** y no toca `-RoutesDir` nunca más.

Qué hace la conversión (y qué no)

Es una conversión **directa**: `frame_to_route.py` toma tu grabación + su `header_*.txt` (el fingerprint) y arma los archivos de la ruta — **generaliza** la línea leyendo la física del vehículo del config, y le pega la config de manejo. **No hay linters interactivos ni "puntaje"**: es un conversor puro, no te va a preguntar "*¿capeo esta curva?*". La calidad sale de **grabar bien** (§7): si la toma quedó sucia (te subiste a un cordón, frenaste mal), **re-grabá** — es gratis y rápido.

> **¿Querés ver dónde se desvió Boris de tu toma?** El framework puede exportar su propia trayectoria

> (`boris_native`) en el mismo formato que una grabación, para compararla con la tuya. Ver §15.2.

6B. Los 4 archivos que genera el wizard — qué son y para qué sirve cada uno

Cuando convertís una toma, el wizard no produce un archivo: produce **cuatro** (y un quinto opcional, el binario de la cinta). Es la pregunta que más aparece, y la respuesta corta es que **cada uno se lee en un momento distinto de la vida de la ruta**.

```
BZBusRoute_<NOMBRE>.json          ~490 KB    la ruta COMPLETA en un archivo: le da el NOMBRE en la lista.
BZBusRoute_<NOMBRE>_hdr.json     ~3-12 KB    el CARNET: quien es el vehiculo y que sabemos de el.
BZBusRoute_<NOMBRE>_wp.csv         ~80 KB    los PUNTOS del camino (una fila por waypoint).
BZBusRoute_<NOMBRE>_cinta.csv     ~500 KB    tu manejo FIEL, 40 veces por segundo.
BZBusRoute_<NOMBRE>_cinta.bin     ~350 KB    la misma cinta, en binario: la carga mucho mas rapida
(opcional).
```

(El ejemplo es una cinta de 500 KB, con las proporciones medidas en 41 rutas: el binario pesa unas 7 décimas de la cinta; el master, lo mismo que la cinta; el `_wp.csv`, la sexta parte; el carnet, el 1 %. Por waypoint son unos 0,5 KB de cinta, 0,35 KB de binario y 0,08 KB de `_wp.csv`. Todo escala con el largo de la toma.)

Los cuatro —y el binario, si lo tenés— van juntos en la misma carpeta: `<perfil del server>\BZ_AutoDrive\`. Y para no buscarlo por todo el manual, **dónde va cada archivo del framework**:

archivo	dónde va
la grabación (`frame_*.csv` + `header_*.txt`)	en el **cliente**, la PC donde manejaste: `%LOCALAPPDATA%\DayZ\BZ_AutoDrive_PathLogger\` (si grabaste en el server, en `<perfil del server>\BZ_AutoDrive_PathLogger\`)
la ruta (`.json` , `_hdr.json` , `_wp.csv` , `_cinta.csv` y `_cinta.bin`)	`<perfil del server>\BZ_AutoDrive\` , todos juntos (es tu `RoutesDir` del wizard)
la ficha del vehículo (`envelope_<clase>.csv` , §6C)	`<perfil del server>\BZ_AutoDrive\` , al lado de las rutas
la configuración (`BZAutoDrive_settings.json` , §13.1)	`<perfil del server>\BZ_AutoDrive\` ; si no existe, el server la crea con los valores por defecto en el primer arranque
los logs de las corridas (`ai_run_*.csv` , `boris_native_*.csv`)	en el server que las corrió: `<perfil del server>\BZ_AutoDrive_PathLogger\`
la memoria del wizard (`wizard_config.json` : tus carpetas)	al lado del script del wizard, en `tools\` ; no viene en el paquete: cada admin arranca sin él y el wizard le pide sus carpetas la primera vez

El master — `BZBusRoute_<NOMBRE>.json`

Qué es. La ruta entera en un solo archivo: todos los waypoints con sus paradas y nombres, los eventos, el vehículo, los accesorios y las opciones de configuración de esa ruta (Apéndice A).

Para qué sirve. Es **el nombre de la ruta**: lo que aparece en la lista del Reproductor y lo que pedís cuando lanzás. También es tu **respaldo legible** — si querés ver la ruta completa en un solo lugar, es éste.

> **Ojo: no es el que se lee para manejar.** Cuando la ruta arranca, el servidor carga **el par compacto** `_hdr.json` + `_wp.csv` (y la cinta), no el master. Por eso los cuatro tienen que estar: el master te da el nombre, los otros hacen el trabajo.

El carnet — `_hdr.json`

Qué es. Unos pocos kilobytes (3 sin la ficha del vehículo, hasta unos 12 con ella) con lo que define a esa toma:

```
VehicleClass        con que vehiculo grabaste
Fingerprint         masa, marchas, angulo de direccion, wheelbase, radio minimo
Wheelbase  MaxGear  la geometria y la caja
Events              las paradas y lo que pasa en cada una
Attachments         las ruedas y demas piezas con las que grabaste
CreepKmh            este vehiculo, se arrastra solo en ralenti?
CreepSource         de donde salio ese numero  ("coast2", "chist2", "nocreep")
CreepSamples        con cuantas mediciones
CoastAtRest         la frenada del motor casi parado, en m/s2
Envelope            copia de la ficha medida del vehiculo (§6C)
WpRows  CintaRows   cuantas filas tienen los otros dos archivos
```

Para qué sirve. El servidor lo lee **primero y entero**, porque es chico. Con eso ya sabe qué vehículo spawnear, qué física tiene y **cuánto trabajo le va a costar leer el resto**.

> **Desde 1.2 el creep y la ficha del vehículo viajan acá**, y no sólo en archivos aparte. Eso arregla de raíz dos fallas que cuestan caro: un archivo suelto **se pierde** o **se desactualiza** respecto de la toma. Lo que viaja adentro no se despega.

Los puntos — `_wp.csv`

Qué es. Una fila por waypoint: posición, rumbo, marcha, ángulo de rueda, si es parada y su nombre.

Para qué sirve. Es lo que el NPC usa para saber **por dónde va el camino**. Se carga **de a pedazos**, repartido en varios cuadros del servidor.

La cinta — `_cinta.csv`

Qué es. Tu manejo tal cual, **40 muestras por segundo**: posición, rumbo, velocidad del chasis, ángulo de la rueda delantera, acelerador, freno, marcha y freno de mano. Está indexada por **distancia recorrida**, no por reloj.

Para qué sirve. Es la diferencia entre "seguir un camino" y **reproducir tu manejo**. De acá sale la velocidad que el NPC usa en cada punto: vuela donde vos volaste, frena donde vos frenaste.

> **Por qué por distancia y no por tiempo.** Porque el NPC no repite tu reloj: repite tu **recorrido**. Si esperaste 6 segundos en un semáforo, eso no se reproduce solo — la espera se pone con un evento (§10).

El binario de la cinta — `_cinta.bin` *(desde la 1.3, opcional)*

Qué es. Una copia de `_cinta.csv` que el server carga mucho más rápido: los mismos números, ya en el formato en que los lee. Lo crea el wizard al convertir («Convertir y binarizar»), y `[B]` lo genera para las rutas que ya tenías.

Para qué sirve. Leer la cinta era la fase más larga de la carga. Con el binario esa fase pesa una fracción, y lanzar o relanzar varias rutas cuesta menos cuadros.

Se puede borrar sin perder nada: sin él, el server carga el CSV. Y si el binario no corresponde a su CSV (cambiaste la cinta, o viene de otra versión del mod), el server lo descarta solo, carga el CSV y lo dice en el RPT: `[CARGA] <ruta>: binario de la cinta descartado (<motivo>): se carga el CSV`. **Después de actualizar el mod, abrí el wizard:** si avisa que hay rutas sin binario o desactualizadas, tocá `[B]` y después `[T]`.

★ Por qué cuatro y no uno

Por una razón concreta y medida: **cargar una ruta entera en un solo cuadro cuelga el servidor.**

Una ruta son más de medio mega entre puntos y cinta. Leerlo todo de una vez congela la simulación el tiempo que tarde, y si se lanzan varias rutas juntas, se multiplica.

La solución es una **cola de carga con presupuesto por cuadro**: el servidor lee un poco de cada ruta en cada cuadro y nunca gasta más de lo que puede. Para repartir el trabajo necesita **saber de antemano cuánto hay**, y eso es exactamente lo que declaran `WpRows` y `CintaRows` en el carnet.

```
el carnet     se lee ENTERO y al instante: quien es el vehiculo y cuanto pesa el resto
los puntos    se cargan por tajadas
la cinta      se carga por tajadas; es la fase mas larga
```

El resultado: el costo por cuadro es constante — con 1, 2, 3 o 5 rutas lanzadas a la vez, el peor cuadro es el mismo.

Preguntas frecuentes

¿Puedo borrar alguno? No. Sin `_wp.csv` la ruta la maneja el cerebro 1.1; **sin `_cinta.csv` el NPC sigue el camino pero ya no reproduce tu manejo** (corre sin cinta); sin `_hdr.json` el servidor carga el master entero, que es lento y **congela el servidor** mientras lo lee; y sin el master la ruta no aparece en la lista. Una ruta que se carga por el master entero lo avisa en el RPT, y también a los admins conectados (`… is in the old format (no _wp.csv): loading it froze the server for N s…`): volvé a convertir la toma con el Wizard. **El binario (`_cinta.bin`) sí se puede borrar:** sin él, el server carga el CSV.

¿Puedo editarlos a mano? El master y el carnet son JSON y se dejan. **La cinta no se toca**: es una medición, no una configuración. Si algo de tu ruta no te gusta, la respuesta es **volver a grabarla**, no corregir los

números.

¿Puedo usar esta ruta con otro vehículo? No conviene, y la razón está en el carnet: la toma trae el *fingerprint* y las mediciones **de ese vehículo**. Si querés la misma ruta con otro auto, **grabala con ese auto** — te va a salir mejor que cualquier adaptación, porque el framework aprende de lo que grabás (§6C).

¿Los tengo que copiar a mano? No: el wizard los deja donde van. Sólo asegurate de copiar **los cuatro, y el binario si lo tenés,** si movés una ruta de un servidor a otro (van todos en la misma carpeta); sin el binario, `[B]` lo rehace del otro lado.

6C. El envelope — lo que el framework aprende de tus tomas

Además de los 4 archivos de cada ruta, el framework mantiene **un archivo por clase de vehículo**:

```
<perfil del server>\BZ_AutoDrive\envelope_<CLASE>.csv
```

Qué es. Lo que el framework **midió** de ese vehículo: cuánto frena, cuánto acelera, cuánto gira con cada ángulo de volante, si se arrastra solo. No lo configurás vos: sale de tus tomas.

Por qué existe. El framework lee el config del vehículo (motor, caja, dirección) y con eso ya maneja. Pero el config **declara** cómo debería comportarse; el envelope **registra cómo se comporta de verdad**. Cuando los dos dicen cosas distintas, manda lo medido.

Qué guarda, campo por campo

```
scalars    radio de rueda y wheelbase
fp         el fingerprint: la firma del vehiculo, para no mezclar clases distintas
medido     la procedencia: de cuantas tomas y cuantos cuadros salio este envelope
k          curvatura: cuanto gira de verdad con cada angulo de volante
ratio      la transferencia de cada marcha (rpm -> velocidad)
stx2       transferencia comando -> angulo de rueda
shist      histograma de direccion: que angulos usaste
ahist      aceleracion: lo que este vehiculo puede
coast2     desaceleracion en punto muerto, por superficie   <- de aca sale el creep
chist2     arrastre medido en ralenti (si lo hay)
nocreep    cuantas veces se midio que NO se arrastra
lhist      acelerador al despegar, durante el manejo normal
lcal       el piso de despegue medido aparte                <- ver mas abajo (en esta version queda vacio)
lcalrun    la firma de cada medicion del piso
```

Cómo se llena

Con las tomas que grabás. Cada conversión aporta lo que esa toma haya mostrado. Una toma corta en línea recta aporta poco; una con curvas cerradas, frenadas fuertes y varias velocidades aporta mucho.

> **La riqueza depende de la variedad, no del largo.** Diez vueltas a la misma rotonda enseñan una sola curva. El envelope está completo cuando **cubre lo que ese vehículo va a tener que hacer**: si tu sedán sólo va a recorrer 10 metros en recta, no necesita más.

> **Un envelope es de UNA clase.** No se comparte entre vehículos, ni siquiera entre variantes del mismo modelo: dos libreas distintas son dos clases distintas y llevan envelopes separados. El `fp` está justamente para que no se mezclen.

Qué ficha usa una ruta

Al arrancar, el vehículo usa **la ficha de su clase** (`envelope_<clase>.csv`), la que se enriquece con cada toma que grabás de ese vehículo. **Para que un vehículo aproveche tomas nuevas no hace falta reconvertir nada.**

Cada ruta convertida con esta versión lleva además, adentro de su `_hdr.json`, **una copia de la ficha tal como estaba al convertirla**. Esa copia sólo se usa en dos casos:

- **en el servidor no existe la ficha del vehículo** (por ejemplo, una ruta que llevaste a otro servidor);
- **la ficha del servidor midió menos que la copia** (un servidor recién instalado, con una ficha de una sola toma corta).

El RPT dice cuál usó, en la línea `[Envelope]`. Reconvertir una ruta sólo actualiza la copia que viaja adentro.

El piso de despegue: lo único que tus tomas no pueden medir

Qué es. El acelerador mínimo con el que ese vehículo rompe el reposo. Por debajo de ese valor, pisar el acelerador no hace nada.

Por qué no sale de tus tomas. Por dos razones medidas:

1. **El teclado es binario.** En 502 tomas hay 1.873 arranques desde parado, y en **1.778 de ellos el acelerador estaba a fondo**. La tecla W no tiene medias tintas, así que tus tomas no contienen ningún valor intermedio del que deducir el piso.
2. **El arranque desde parado es un transitorio del motor.** En 2 de cada 3 despegues, la velocidad sube hasta ~2 km/h y se resetea **sin que el vehículo se haya movido 10 centímetros**. Ese tramo no refleja tu manejo: es cómo el motor de física saca un vehículo del reposo.

Qué usa entonces el framework. Un valor de flota **medido**: el piso de siete clases de vehículo (un auto, una moto, un 4x4 militar, un camión y tres buses), medido con el vehículo quieto en llano y el acelerador subiendo de a poco, entre cinco y nueve veces cada uno. Van de 0,05 a 0,25, y el framework usa la mediana, **0,13**, para cualquier vehículo. No hace falta que hagas nada.

Qué implica — y qué no. Seamos claros:

> **No cambia cómo maneja.** En una ruta sin paradas, entre el valor de flota y el de tu vehículo hay **un tick de rampa**: unas cuatro décimas de segundo al cruzar 5 km/h.
>
> **Dónde sí puede notarse:** en el **arranque en pendiente**, donde el framework tiene que sostener el acelerador justo por encima del piso para no retroceder. Si un vehículo tuyo retrocede al arrancar en

> una cuesta empinada, ese es el lugar donde mirar.

Medir el piso de cada vehículo es una herramienta que existe en desarrollo y **no viene en esta versión**. Cuando venga, se va a explicar acá.

7. Cómo grabar bien (la calidad sale de acá)

El NPC va a manejar **tan bien como manejaste vos** — la grabación es su manual de manejo. El wizard te avisa de los problemas, pero **no puede inventar una buena línea**: esa la ponés vos al grabar. Estos hábitos hacen la diferencia:

- **Manejá suave y parejo.** Nada de acelerones ni frenazos secos: el NPC copia tu perfil. Frená **progresivo y antes** de la curva, no encima.
- **Tomá las curvas con la línea que querés** que haga el NPC — ni cortes el vértice ni te abras de más.
- **Ritmo constante en las rectas:** la velocidad que grabás es la referencia que va a usar el NPC.
- **Una toma limpia vale más que diez sucias.** Si te subiste a un cordón o frenaste mal, **re-grabá** — es gratis y rápido.
- **Cambios de sentido (K-turn, galpón):** frená del todo y, **sin soltar el freno, cambiá de marcha** y seguí — el **intercambio se auto-detecta** del cambio de gear (no tocás ninguna tecla); la reversa también sale del gear (§9.4). Si soltás el freno antes de cambiar, el vehículo puede seguir avanzando unos centímetros (en bajada, siempre), y ese avance queda grabado como parte del recorrido.
- **Paradas:** una sola frenada continua hasta detenerte del todo, y ahí marcá con **NUMPAD 4**. Ver "Cómo parar" más abajo.
- **Bocina y luces:** en esta versión **no se graban** — lo que hagas con la bocina o las luces mientras grabás no queda en la toma. Las **luces** las prende el framework según la hora del juego (`LightsMode: "auto"` , el default) y la **bocina** se configura por ruta con `HornMode` (§9 "Bocina y luces" y Apéndice A.2b).

  > **Para que las luces se vean:** el vehículo necesita **batería + bombillas (faros) instaladas**. El framework le energiza la batería al spawn, pero las bombillas tienen que estar entre sus `Attachments` (normalmente ya vienen del fingerprint del vehículo). Sin faros físicos, la tecla L no prende nada.

> *Regla mental:* manejá pensando "así quiero que lo haga el bot". Lo que vos hagas, lo hace él.

7.1 — Cómo terminar la toma (de esto depende que el NPC llegue exacto)

Al final de la toma, **terminá en recta**: los últimos metros con el volante derecho, frenando hasta detenerte, y cortá con **NUMPAD 5** con el motor encendido. **El final puede quedar en cuesta**: el NPC llega igual, también en subida.

Si el final es en llano, antes de cortar **soltá todo** —acelerador, freno y freno de mano— y **esperá a que el motor baje a ralentí**: son dos o tres segundos, más en un deportivo. En esos segundos el framework mide cuánto avanza el vehículo solo, sin acelerador. Esa medición es la que le permite clavar el punto final. Sin ella usa un valor estimado y la llegada se corre. **Si el final es en cuesta, mantené el freno de pie pisado** mientras cortás (si soltás, el vehículo rueda hacia atrás y eso queda grabado) y hacé esa medición en un tramo llano de la toma (§7.2).

- **El freno de mano no sirve para esto:** con el freno de mano puesto no se puede medir si el vehículo avanza solo.
- **En pendiente no se mide** (la medición exige menos de unos 3° de inclinación): vale en cualquier tramo llano de la misma toma, o en una toma corta en llano del mismo vehículo.
- **Alcanza con hacerlo una vez por vehículo.** El wizard guarda la medición por vehículo (`envelope_<clase>.csv`) y no la pisa en las tomas siguientes del mismo vehículo. Para rehacerla: `tools\python\python.exe tools\envelope_emit.py "<toma.csv>" --out "<RoutesDir>" --pisar`.

7.2 — Cómo parar (paradas intermedias)

Una sola frenada continua hasta detenerte. No frenes, sueltes y te arrastres los últimos metros: el NPC puede llegar un poco más lento y se queda en la *primera* frenada, metros antes de tu parada.

Recién detenido del todo, marcá NUMPAD 4, y seguí.

- **No hace falta esperar en la toma.** El tiempo de espera lo ponés en el evento de esa marca (§10.1); al grabar, detenete lo justo y arrancá.
- **No hace falta esperar el ralentí en cada parada**: eso es una sola vez por vehículo (§7.1). Podés mantener el freno pisado.
- **Cambiar de marcha estando detenido está bien** —bajar a primera para arrancar—: el framework lo resuelve.
- **En un cambio de sentido, el freno pisado hasta poner la otra marcha.** Si te detenés, soltás y el vehículo se arrastra antes de la reversa, la toma registra ese arrastre y el NPC lo copia. El framework frena igual al final de cada tramo, pero la llegada más exacta sale de una toma limpia.

Si parás en pendiente, mantené el freno de pie pisado mientras marcás o cortás la toma. Si soltás, el vehículo rueda unos centímetros hacia atrás y ese retroceso queda grabado. Usá el freno de pie, no el de mano. Y la medición de §7.1 hacela en cualquier tramo llano de esa misma toma: lo más cómodo es al principio, arrancá a grabar con el motor encendido, quedate quieto dos o tres segundos con todo suelto, y recién ahí salí.

7.3 — Tres cosas que arruinan una toma

Terminar derrapando. Si al frenar el vehículo se cruza o hace un trompo, regrabá ese final. Lo que el vehículo hace de costado no lo puede reproducir ningún chofer, y el framework puede llegar a interpretarlo como una maniobra que no existe. El wizard te avisa con el segundo exacto: `(!) TROMPO en la fila 7393 de la toma (t 184.8 s ...): el chasis gira 216 grados y desliza 179 -- la toma NO es reproducible desde ahi`.

Pasar rozando en un lugar angosto. En calles angostas y curvas cerradas junto a carteles, postes o muros, pasá más despacio y bien centrado. El NPC repite tu línea con variaciones de centímetros entre una corrida y otra: si en tu toma pasaste rozando, a veces va a tocar.

Ignorar los avisos de salto. A alta velocidad el server a veces corrige tu posición hacia atrás mientras grabás; se siente como un tirón. El conversor lo limpia solo, pero si en la salida del wizard ves `(!) glitch en racha` o `(!) salto persistente`, revisá ese tramo o regrabalo.

Dos avisos más, desde la 1.2.1. El wizard también te dice:

- `(!) La toma tiene N retroceso(s) grabado(s)`: mientras grababas, el server te corrigió la posición hacia atrás (el tirón). La ruta los reproduce como están y el NPC no sigue ese salto, pero si en ese tramo se desvía, regrabalo.
- `(!) En N cambio(s) de marcha el vehiculo avanzo sin freno antes de cambiar`: te detuviste, soltaste el freno y el vehículo se arrastró antes de poner la otra marcha. Al grabar, mantené el freno pisado hasta cambiar (§9.4).

7.4 — Grabá vacío

Grabá con el vehículo **vacío**. Baúl lleno y pasajeros no cambian cómo lo maneja el NPC: velocidad, aceleración, trayectoria y llegada dan igual. Lo que grabaste vacío sirve cargado.

---

8. El uso más simple: un servicio de bus permanente

Es el caso para el que **nació** el framework: **un bus de línea que recorre tu server solo, todo el día, sin que vos hagas nada.** Sin misiones, sin admin operándolo — transporte ambiental para tus jugadores. (Es el uso "solo para el server", el más común al bajar el mod.)

Cómo dejarlo corriendo como servicio permanente:

1. **Grabá una ruta en loop** (que vuelva cerca del inicio) — ej. la costa parando en cada pueblo.
2. **Dejala como la ruta default** del server (`BZBusRoute.json`).
3. Al arrancar el server, el bus **aparece solo y empieza a manejar**. Cuando termina la vuelta (o si lo destruyen), **respawnea** a los `RespawnDelay` segundos (`300` por default; `0` o un número negativo valen **60 s**, no lo apagan) → **servicio continuo, 24/7**. *(Las rutas que largás desde el Reproductor no se re-lanzan: al terminar, el vehículo se retira solo.)*

> No necesita Quest ni que vos estés conectado: solo el mod cargado + tu ruta default. Lo prendés y se olvida. *Ejemplo:* el bus de la costa de Chernarus uniendo 14 paradas, dando vueltas todo el día — los jugadores lo toman para moverse, como un colectivo de verdad.

Las tres formas de correr una ruta (para tenerlas claras):

Forma	Cómo arranca	Para qué
Servicio permanente	sola, al boot del server (ruta default + loop)	bus de línea ambiental
On-demand (Reproductor)	la elegís en la UI, sin restart	probar, eventos puntuales
Por misión (Quest)	la dispara un quest	convoyes, refuerzos, emboscadas (§11)

8.1 — Parada a demanda: hacele señas al bus *(validado in-game con el cerebro 1.1; con el 1.2 falta probarlo)*

El bus de línea no para solo en los waypoints `isStop`: **cualquier jugador puede pedirle que pare haciéndole señas**, como en la vida real.

> **Viene desactivado por default** (opt-in — es una regla que el admin elige). Para habilitarlo, poné `HailGestureEnabled = true` en los *settings* globales del mod; sin eso, el bus solo para en los waypoints `isStop`. Mientras esté activo, la regla aplica a **todas** tus rutas.

Cómo se usa (jugador):

- Parate **en el camino, de frente al bus** (donde el chofer te ve), a ≤30 m.
- Hacé el emote **OK / pulgar arriba** (`ID_EMOTE_THUMB`).
- Boris **frena, te espera ~10 segundos** para que subas, y **sigue** la ruta.

Qué pasa por dentro:

- La detección es **server-side**: el framework recorre los jugadores cerca del vehículo, mira si están en el **cono frontal** del bus (producto punto rumbo-del-bus · dirección-al-jugador > 0.25 → "el chofer lo ve", no detrás ni al costado) y si están ejecutando el emote OK.
- Al detectarlo, **pausa** el bus (reusa el freno del modo pausa) y marca la reanudación a **+10 s**.
- A los 10 s reanuda. Con el cerebro 1.2 el seguimiento es por posición, así que retoma desde donde quedó el vehículo; con el 1.1 reanuda **re-localizando** el waypoint a la posición real del bus (ver abajo).

Parámetros: el **on/off** ya es config (`HailGestureEnabled` , default off); el resto está hoy en constantes (fáciles de exponer a JSON): radio 30 m · cono `dot > 0.25` (~±75°) · espera 10 s (20 ticks) · emote `ID_EMOTE_THUMB` (9).

Dos cosas que costó resolver (notas para quien extienda):

1. **Leer QUÉ emote hace el jugador, server-side.** `EmoteManager.GetGesture()` NO sirve: devuelve `m_GestureID` (otro campo, seteado por `SetGesture()`), no el emote en curso. El emote real vive en `m_CurrentGestureID` , que es `protected` y sin getter público. Solución: un `modded class EmoteManager { int BZ_CurrentGesture() { return m_CurrentGestureID; } }` — es script class (no engine → moddeable), y el `protected` es accesible desde la subclase.
2. **Retomar derecho tras la parada** *(cerebro 1.1)*. Al frenar, el bus sigue de largo ~10-15 m mientras se detiene, pero el índice de waypoint queda congelado donde frenó. Como el avance del índice está capado por la velocidad (no avanza desde 0 km/h), al reanudar el bus está *adelante* de su objetivo → el control apuntaba a un wp que quedó atrás → **timonazo al costado**. Fix: al reanudar, **re-localizar el índice al waypoint más cercano a la posición real** (una vez, salteando el cap).

> **El gesto como primitiva.** Es el primer uso de un patrón más general: *el emote del jugador como input de control del framework*. Hoy está cableado a "OK → parar", pero el mismo mecanismo (leer el emote server-side + condición de proximidad/visión) sirve para cualquier comando (seguir, esperar, dar la vuelta). Es la contraparte jugador↔NPC de los triggers del DSL de eventos (§10).

8.2 — Lanzar y encadenar rutas desde el `init.c` de tu misión (sin mod propio) *(probado en juego)*

Es la forma más básica de disparar secuencias: unas líneas en el `init.c` de la misión del server (`mpmissions\<tu misión>\init.c`) y el framework hace el resto. **No hace falta armar un mod:** el `init.c` ve las clases de los mods cargados.

Dónde va. Dentro de la clase `CustomMission` que ya tiene tu `init.c` agregás un `OnInit` (si ya tenés uno, sumá las líneas adentro, después del `super.OnInit()`) y los métodos de abajo. Las rutas se nombran por **archivo**: `BZBusRoute_<nombre>.json`, tal como las deja el Wizard en `profiles\BZ_AutoDrive\`.

```
class CustomMission: MissionServer
{
    // ... lo que ya tenias ...

    override void OnInit()
    {
        super.OnInit();   // primero: ahi arranca el framework
        // espera a que el mundo este listo antes de lanzar (60 s)
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_Lanzar, 60000, false);
    }

    // lanza una ruta con su propio runner; devuelve null si no arranco
    BZBusService BZ_Ruta(string archivo, string etiqueta)
    {
        BZBusService r = new BZBusService();
        BZBusService.GetRunners().Insert(r);
        r.SetHoldAtStops(false);          // false: en las paradas espera lo grabado y sigue solo
        r.SetEtiqueta(etiqueta);          // firma el log: <ruta>#<etiqueta>
        r.RespawnFromPath(archivo);
        if (!r.RunnerIsActive() && !r.RunnerEsperandoCabecera())
        {
            BZBusService.UnregisterRunner(r);
            return null;
        }
        return r;
    }

    void BZ_Lanzar()
    {
        BZ_Ruta("BZBusRoute_PATRULLA.json", "P1");
        // otra, 2 minutos despues
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_LanzarConvoy, 120000, false);
    }

    void BZ_LanzarConvoy()
    {
        BZ_Ruta("BZBusRoute_CONVOY_01.json", "C1");
    }
}
```

Encadenar: cuando termina una, largar la siguiente. Un runner lanzado así **se retira solo** cuando llega completo (a los `EndHoldSeconds`, 3 s por defecto), así que alcanza con mirar cada tanto si terminó. Estos métodos van en la misma clase:

```
ref BZBusService m_BZ_Tramo;       // el runner que esta corriendo
int m_BZ_Paso = 0;

void BZ_Cadena()   // llamala una vez (por ejemplo desde BZ_Lanzar)
{
    m_BZ_Tramo = BZ_Ruta("BZBusRoute_IDA.json", "IDA");
    GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.BZ_Vigilar, 5000, true);   // cada 5 s
}

void BZ_Vigilar()
{
    if (!m_BZ_Tramo || !m_BZ_Tramo.RunnerRouteEnded()) return;   // todavia maneja
    if (!m_BZ_Tramo.RunnerRouteCompleted())
        BZBusService.UnregisterRunner(m_BZ_Tramo);             // termino mal: retiralo vos
    m_BZ_Paso++;
    if (m_BZ_Paso == 1) m_BZ_Tramo = BZ_Ruta("BZBusRoute_VUELTA.json", "VUELTA");
    else
    {
        m_BZ_Tramo = null;
        GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).Remove(this.BZ_Vigilar);   // fin de la cadena
    }
}
```

Lo que tenés que saber:

- **`SetHoldAtStops(false)`.** Con `true` —el default de `PlayRouteByName`— el vehículo se queda en cada parada hasta que alguien llame a `ResumeFromHold()`: desde un `init.c` quedaría parado para siempre en la primera.
- **Un runner propio por ruta**, no `PlayRouteByName`: así cada uno se retira solo al terminar y lo podés dar de baja con `UnregisterRunner`. Sale en el panel con su etiqueta, y desde ahí también lo podés parar.
- **Si termina mal** (`ROUTE_INCOMPLETE`, `STUCK`), el vehículo queda quieto: retiralo vos, como en el ejemplo. *(Desde la 1.3, si no lo retirás vos ni leés el fin en 60 s, el framework lo retira solo: §B.7.)*
- **Si el punto de salida está ocupado**, el runner espera a que se libere (`RunnerEsperandoCabecera()`): en una cadena de ida y vuelta, la vuelta sale cuando se retira el vehículo de la ida.
- **Para un servicio que da vueltas para siempre**, dejá esa ruta como la ruta por defecto del server (§8), o relanzala cuando termine con el mismo esquema de la cadena.
- **Los eventos de cada ruta** (§10) funcionan igual: el `init.c` decide cuándo arranca cada una; lo que pasa adentro lo ponen sus `Events`.
- **Un error en el `init.c` no deja arrancar la misión.** Mirá el RPT (`Can't compile`). Valen las reglas de Enforce del Apéndice B.5: no hay operador ternario, `new` va sin argumentos, etc. La API completa (estado, pausa, eventos) está en el Apéndice B.7.

8.3 — Qué pasa al final de la cinta *(desde la 1.3)*

Cuando una corrida termina —llegó, no llegó o quedó trabada— y antes de que el framework retire o relance el vehículo:

- **El jugador que viajaba sentado baja solo**, con la animación de siempre y **sin aviso**. Si querés avisarle, poné un evento `ui_broadcast` en el último waypoint (§10).
- **Desde ese momento nadie sube.** El vehículo queda **cerrado a jugadores** (el cierre de Expansion: el cliente ni ofrece «subir») hasta que el framework lo borra. Lo mismo con el **STOP del panel** y en cualquier **retiro**: primero cierra, después baja al que esté sentado, y recién vacío borra. Si el retiro llega a su tope y deja el vehículo en el mundo, lo **reabre** como estaba.
- **La moto también:** la bajada y el cierre son los mismos.
- El retiro **nunca** borra un vehículo con un jugador sentado o subiendo: lo baja y espera.
- **Si programás un mod:** el FIN baja a los jugadores. Para **conservar pasajeros entre cintas** (un comercio que viaja con gente a bordo, un taxi que encadena viajes), **retené el vehículo con `HoldRunner()` antes del final** (B.7.3) y seguí desde ahí, como hace un servicio de comercio: el hold no baja a nadie ni cierra el vehículo; el FIN, sí.

---

9. Cómo maneja el NPC — sigue tu línea y tu velocidad

No elegís "modo": hay un solo control, y es el que reproduce tu manejo. El NPC:

- **sigue tu línea** grabada (*pure-pursuit*: apunta a un punto adelante sobre tu traza), y
- **usa tu velocidad grabada** —vuela en las rectas, frena antes de las curvas, igual que vos—, **moderada** por lo que el vehículo **puede** hacer en cada curva: si grabaste una curva más rápido de lo que ese auto aguanta, la **capea sola**.

El acelerador y el freno parten de **tus pedales grabados** (la cinta) y los corrige un **modelo inverso** derivado del **config del vehículo** (motor, torque, caja) y de lo que se **midió** de él: si el vehículo puede menos que lo que pediste, lo acomoda. Por eso **el mismo controlador maneja cualquier vehículo sin tuning por-auto** (config-como-manual): cada uno lo hace según SU física — el chico clava la línea, el pesado la hace más lento pero llega. *(La grabación en sí es de su vehículo: para correr la misma ruta en otro auto, **grabala con ese auto** — §6B, §6C.)*

> **No configurás nada:** el wizard produce este control solo, leyendo el fingerprint del auto (§5). *(Antes existían "modos 1/2/3" —repetir pedales / velocidad por curvatura / velocidad grabada—; quedaron **unificados en este único control**, el que valida esta versión. Los flags que los prendían — `FollowPath`, `UseInverseModel` …— son **del cerebro 1.1**: con el 1.2, el que corre por defecto, no se leen (Apéndice A.2).)*

> **¿Un vehículo pesado se ve forzado con tu ritmo?** Lo mejor es **grabar la ruta con ese vehículo** — así la corrida ya nace a su medida. *(Si después querés esa misma ruta para autos más chicos, **grabala con uno de ellos** en vez de reusar la grabación del pesado: cada toma alimenta el envelope de SU clase — §6C.)*

> **Lo que el NPC no reproduce: el tiempo detenido.** El NPC repite tu recorrido **por posición, no por reloj**. Lo que hiciste parado —esperar, mirar alrededor— no se repite: al llegar a una parada arranca apenas se detiene. La espera la ponés vos con un evento en esa marca (§7.2 y §10.1).

9.4 — Cambios de sentido (K-turn, entrar de cola): auto-detectados

Cuando tu ruta necesita **cambiar de dirección** —un giro de tres puntos (**K-turn**), dar la vuelta, entrar de cola a un galpón— **ya no marcás nada con una tecla**. El framework lo hace **solo**: detecta el **cambio de sentido** a partir del **cambio de gear (forward↔reverse)** que hiciste al grabar, y corta el tramo ahí (`legBreak`) por su cuenta.

Cómo se graba — la regla: siempre DETENIDO al invertir. No se puede pasar de forward a reversa (ni al revés) en movimiento —ni vos ni Boris—, y ese cambio de marcha **siempre cae a velocidad ~0**. Justo por eso el conversor lo puede leer sin ambigüedad. En un K-turn:

1. Venís grabando en **forward**. Llegás al punto donde vas a dar marcha atrás → **frená del todo** → metés la **marcha atrás**. *(No tocás ninguna tecla: el cambio forward → reversa es el intercambio, y el conversor lo detecta acá.)*
2. Hacés la reversa: **despacio (3–8 km/h), con correcciones chicas** (el arco no tiene que ser perfecto, lo vas corrigiendo). El framework **auto-detecta** que ese tramo es reversa —del gear— y lo trata como reversa.
3. Al terminar → **frená del todo** → metés **forward** y seguís. *(De nuevo sin tecla: el cambio reversa → forward es el segundo intercambio.)*

Repetís en cada cambio de sentido, siempre igual: **frenás del todo, cambiás de marcha sin soltar el freno, seguís**. Cada uno de esos cambios de gear corta el tramo (`legBreak`), y Boris lo trata como un **nuevo arranque**: llega alineado, se planta, y sale limpio en el sentido nuevo. Es el *cusp* (reversa↔forward), el punto más difícil del control — hoy **resuelto** (§17).

> **Por qué sin soltar el freno.** Si te detenés, soltás el freno y el vehículo se arrastra a paso de hombre antes del cambio (en bajada pasa siempre), la toma lo registra. El NPC repite ese arrastre y frena al final del tramo, pero **lo que el vehículo avanzó a menos de 1 km/h dentro de tu última parada no se repite** (típico: 30-40 cm): el tramo cierra donde el vehículo quedó quieto al entrar en esa parada. Con el freno pisado hasta poner la otra marcha, la parada y el cambio caen en el mismo lugar.

> **Un 0 km/h NO es siempre un intercambio.** Frenar del todo **sin cambiar de sentido** (esperar, ceder el paso) es una **pausa**, no un corte de tramo. El conversor distingue las dos cosas: solo el 0 km/h **con cambio de gear forward↔reverse** genera el `legBreak` . Por eso no necesitás marcar nada — el gear ya lo dice todo.

Lo que el framework hace SOLO (no marcás nada):

- **El intercambio** — sale del **cambio de gear** (forward↔reverse) grabado: ahí el conversor pone el `legBreak` y corta el tramo.
- **La reversa** — sale del **gear grabado**: si manejaste en marcha atrás, ese tramo es reversa. Boris la reproduce con su controlador de reverse (dirección por eje trasero) y la **generaliza a cada vehículo por su física** (uno de giro ancho reversa más lento y por eso sigue mejor el arco).
- **La aproximación** a la maniobra — **automática**: Boris frena predictivo ANTES de la maniobra, sin que marques nada.
- **La parada final** (endpoint) — autoadaptativa por vehículo + piso (§17).

> **Regla de oro: frená en la recta, no en la curva.** El punto donde frenás para invertir la marcha es donde el auto-detect **corta el tramo** — así que hacelo donde la **trayectoria YA ES RECTA** (el rumbo ya se estabilizó), no en plena curva. El detalle de por qué (los ángulos de volante de la reversa son del vehículo, no del camino) y la validación multi-vehículo están en §9.5.

> *Consejo (config-as-manual):* grabá la reversa **del lado seguro** — entrada lo más **recta** posible y mínimas correcciones. Cuanto más limpia tu demo, más limpio el NPC. En una **rampa cuesta arriba** (galpón) mantené el momentum: **no frenes a cero en la subida**.

> *Tomas que arrancan en reversa.* El conversor define la dirección del **tramo inicial** por el **primer movimiento real** (>1 km/h), no por el gear en que estás parado. Si tu ruta empieza dando marcha atrás, hacé que tu **primer movimiento SEA la reversa** —sin creepear ni un pelo hacia adelante— y largá decidido, así el arranque queda leído como reversa.

Maniobras finas. ¿Querés el arco EXACTO de un estacionamiento? Se consigue **grabándolo con calma**: frenás del todo, metés la marcha, hacés la maniobra despacio (3-8 km/h) con correcciones chicas. El framework reproduce **lo que hiciste**, así que una maniobra grabada con cuidado sale mejor que una apurada. Si no te gustó cómo salió, volvé a grabar ese tramo: es más rápido que corregirlo después, y el resultado es fiel a la física de ese vehículo.

> `maniobra` **(legacy):** existió un modo `maniobra` (replay directo con salida por cruce de waypoint) que se marcaba con tecla. **Deprecado** desde 2026-06-17 y **sacado del menú**: cruise + aproximación automática + geometría lo cubren. No aplica a tomas nuevas.

9.5 — Que una traza con maniobra sirva en varios vehículos (dónde cortar el cambio de sentido)

La **reversa** reproduce los **ángulos de volante** que hiciste (la cinta), con una corrección chica para no salirse de la línea; si la cinta no alcanza, un control de reversa propio sigue el camino. Como la corrección es chica, vale la premisa **"llegar listo"**: el vehículo tiene que alcanzar el tramo de reversa en la **pose y la velocidad** que demostraste — un error grande de entrada no se corrige del todo.

El punto fino: los ángulos de volante son del vehículo, no del camino. Un mismo ángulo de volante da un **radio más ancho o más cerrado según el wheelbase** (un auto largo se abre, uno corto cierra). Entonces un tramo reproducido con tus ángulos **sobre una curva NO generaliza** a otro vehículo: el que grabó cierra la curva, pero un auto más largo, con esos mismos ángulos, se abre y se la lleva puesta.

La regla de oro — cortá en la recta, no en la curva. Hacé el **cambio de sentido** (frená del todo antes de invertir la marcha, ahí el auto-detect corta el tramo) donde la trayectoria **YA ES RECTA** (el rumbo ya se estabilizó), no en plena curva. Así:

- La **curva** la maneja el **control cerrado** (el pure-pursuit sigue el **camino**, agnóstico al vehículo — cada auto la hace con SU volante).
- Solo la **recta + la maniobra** dependen de tus ángulos de volante, y en recta eso es seguro (no hay curva que se abra).

Cross-vehículo: grabala de nuevo. Una **grabación** es de **su** vehículo: captura su *fingerprint*, su freno medido, su marcha, y alimenta el envelope de **esa clase** (§6C). Por eso **no se recomienda header-swapear grabaciones** — meterle el carnet de otro auto le pone mediciones que no son suyas.

Para correr la misma ruta **con maniobra** en otro vehículo, **grabala con ese vehículo**. Con una maniobra de por medio la razón es más fuerte todavía: la reversa **reproduce tus ángulos de volante** y esos ángulos son **del vehículo, no del camino** — un auto más largo con esos mismos ángulos se abre y se lleva la curva puesta. Eso no se arregla reasignando: se arregla grabando.

El framework, por su lado, **no reentrena nada por vehículo**: lee el config del auto (motor, torque, caja — "config como manual de manejo"), le suma lo que midió en el envelope de esa clase, y arma su curve advisory y su modelo inverso a la medida de ESE auto.

> **Validado (la regla del corte).** Probando la **misma traza** en dos vehículos de wheelbase distinto — **OffroadHatchback** (2.357 m) y **CivilianSedan** (2.935 m, más largo)—:
>
> - con el **corte en la recta** → completó la maniobra sin chocar; la reversa clava el intercambio bien alineada (precisión de parada resuelta — §17).
> - con el **corte en la curva** → el Sedan, más largo, **se abrió y chocó** (esos ángulos de volante eran del Hatchback).
>
> Moraleja práctica: **hacé el cambio de sentido en recta, no en curva.** Y si querés esa misma ruta para otro vehículo, **grabala con ese vehículo**: la maniobra es lo primero que deja de generalizar.

La aproximación a la maniobra es automática. El control frena predictivo antes del cambio de sentido, sin que marques nada. Si el corte cae en una **zona rápida** y notás que **frena de más** y se planta justo en la transición, la respuesta es **cortar la maniobra donde ya venís lento** (no en plena recta rápida) — así la entrada es suave.

Sin teleport de alineación. El vehículo entra a la reversa manejando, sin ningún salto de posición. *(El cerebro 1.1 tenía un "snap" de alineación, `ModeEntrySnapEnabled`, apagado por defecto; el 1.2 no lo usa.)*

Bocina y luces — los modos `HornMode` / `LightsMode`

Aparte de **cómo** maneja, una ruta define **qué hace con la bocina y las luces**. **En esta versión la toma no graba ni la bocina ni las luces**, así que las dos se configuran por ruta (Apéndice A.2b):

- `HornMode` — `off` (**default**: nunca) · `stops` (bocina al terminar cada parada, antes de salir) · `finish` (bocina al llegar al final, cuando la ruta termina completa). `replay` viene de cuando la toma grababa la bocina: hoy sólo suena el verbo `horn`, y el server lo avisa al cargar la ruta.
- `LightsMode` — `auto` **(DEFAULT)** · `off` · `auto_inverted` · `on` (detalle abajo). `replay` no sirve en esta versión: no hay luces grabadas que reproducir (sólo actúan los verbos, y el server lo avisa al cargar). **Un verbo `lights_on` / `lights_off` manda sobre `LightsMode` hasta el próximo spawn.**

> **¿Bocina o luces en un punto exacto?** Marcá ese punto con **NUMPAD 4** al grabar y colgale un evento con el verbo `horn`, `lights_on` o `lights_off` (§10).

Luces automáticas por default (`auto`). Todas las tomas arrancan en `auto`: de noche el vehículo **prende los faros solo** desde que arranca el motor (apenas Boris se sienta y enciende, antes de empezar a rodar), y

de día los deja apagados. Si la hora cambia durante la ruta, las luces la siguen. **No hace falta grabar nada.** El umbral de "noche" es la hora del mundo del juego: **luces ON de 19:00 a 06:00**, OFF el resto del día.

Los modos de `LightsMode`:

- `auto` — *(default)* prende de noche (19:00–06:00), desde que arranca el motor y también si la noche cae a mitad de ruta; off de día.
- `off` — luces **siempre apagadas**. Es el override para **misiones nocturnas sigilosas**: el vehículo no se delata con los faros aunque sea de noche.
- `auto_inverted` — al revés del `auto`: **apaga de noche** (sigilo por horario) y prende de día.
- `on` — luces **siempre encendidas**, de día y de noche.

> **Cómo DESACTIVAR las luces.** Si querés que una ruta corra **a oscuras** (misión sigilosa nocturna, no delatar el vehículo), poné en el header/JSON de la ruta:
>
> ```
> "LightsMode": "off"
> ```
>
> Con `off` los faros nunca se prenden, aunque sea de noche. (Para volver al comportamiento automático, sacá el campo o ponelo en `"auto"`.)

> *Ejemplo:* un bus de pasajeros normal no necesita tocar nada — con el default `auto` ya prende de noche solo. Un convoy de infiltración usa `"LightsMode": "off"` (o `auto_inverted`) para no delatarse en la oscuridad. Recordá: para que las luces se vean, el vehículo necesita **batería + faros instalados** (§7).

9.6 — Cuando se traba: el AutoRecovery

Una ruta larga, corriendo desatendida durante horas, tarde o temprano se encuentra con algo: un auto que alguien dejó en la calzada, una parada en pendiente donde el vehículo se fue unos centímetros hacia atrás, una subida donde el motor se ahoga. El **AutoRecovery** se ocupa de eso. Viene **activado**, funciona solo, y **no salta el vehículo a otra parte de la ruta**: lo destraba donde está —como mucho lo re-ubica unos metros— y la ruta sigue desde ahí.

Cómo se da cuenta. No mira sólo la velocidad: un vehículo que empuja algo, o que patina contra un cordón, marca medio km/h sin avanzar un metro. Por eso mira el **avance sobre la ruta**:

- si en **10 s** no avanzó y el acelerador está a fondo, está **trabado**;
- si en **20 s** no avanzó nada, sea cual sea el motivo, también;
- una **parada grabada** (un lugar donde vos te detuviste al grabar) **no cuenta** como traba: el vehículo está quieto a propósito.

Trabaja en tres niveles, de menos a más invasivo.

1. Se trabó el cerebro, el vehículo puede moverse. Es el caso típico de una parada: el vehículo quedó unos centímetros fuera del punto donde la grabación lo espera y no vuelve a enganchar. El AutoRecovery lo re-arranca en el lugar, hasta tres veces. No se ve nada raro en pantalla: el vehículo arranca y sigue.

2. Se ahogó el motor. En una cuesta, si el vehículo llega con menos velocidad que en la grabación, la marcha que trae puede no alcanzar. El controlador fuerza **primera** y arranca como se arranca en pendiente. Esto lo hace el controlador, no el AutoRecovery, y se configura aparte (`BogRecovery`). Pero acelerador a fondo con el motor en ralentí también es lo que se ve cuando el vehículo **empuja algo**. Por eso, si con la primera avanza un poco y se vuelve a clavar, o si en 10 s no avanza ni lo que avanzaría a 2 km/h, **no era un ahogo**: pasa a la escalera en ese mismo momento.

3. Traba física: chocó, quedó cruzado, montado sobre algo, hundido o volcado. Acá entra la **escalera**, que prueba estos pasos en orden, en el mismo lugar:

1. **Salir manejando.** Se aleja 3 m en línea recta, a no más de 5 km/h, en el sentido **contrario** al que venía (marcha atrás si iba hacia adelante; hacia adelante si el tramo es de los que se graban en reversa), y después vuelve a encarar la ruta solo. Si de ese lado hay un obstáculo —otro vehículo, una garita, un poste, un guardrail, una pared—, no se mueve: lo detecta y pasa al paso siguiente.
2. **Volver sobre sus huellas.** Se re-ubica en el último punto que pisó bien, un largo entre ejes más atrás, con la orientación que tenía el humano al grabar ahí.
3. **Zonas seguras hacia adelante.** La primera está a la mitad del largo del vehículo más metro y medio, que es la primera posición donde el paragolpes trasero pasa el punto de la traba. Después avanza de a un metro.

Los tres pasos funcionan también en los tramos de reversa (salir de un andén marcha atrás, entrar de cola). Si un destino está ocupado por otro vehículo, o el vehículo queda montado sobre algo, ese intento no se gasta: corre la posición un metro y prueba de nuevo. Cada re-ubicación fuerza el arranque desde ese punto, y la escalera se reinicia cuando el vehículo retoma la marcha.

Sólo si la escalera se agota la ruta se da por trabada (`STUCK`), y el log dice por qué. Si la escalera está en medio de un paso, no se corta.

Qué no hace. No frena ni esquiva ante un vehículo detenido en el camino: si lo empuja sin avanzar, a los 10 s lo trata como traba. *(El `ObstacleSlow` del Apéndice A.5b es del cerebro 1.1.)*

Nada de esto tapa el problema. Cada intervención queda registrada:

- En el RPT, con el lugar y la evidencia que la disparó: `[AR] <ruta> wp 380 s 594 · FISICO (...) · escalon 1: maniobra` . Cuando la escalera se abre sale `[C12] BLOQUEADO en wp N · <motivo>` , y cuando se cierra, `[C12] BLOQUEO terminado tras X s` : tiene que haber tantos de uno como del otro. Si tu mod necesita enterarse (por ejemplo, para que otro vehículo no le ceda el paso a uno que no se puede mover), consultá `RunnerBloqueado()` (Apéndice B.7).
- Al final de la ruta, en la línea del `FIN` . **Desde la 1.2.1, una intervención física del AutoRecovery (maniobra o re-ubicación) deja la ruta en `ROUTE_INCOMPLETE` , aunque llegue:** `*** ROUTE_INCOMPLETE en wp 335 · AR fisico con accion: wp 135 maniobra: … · REPOSO tras 0.5 s: a 0.106 m …` . Los re-arranques suaves (`NO-FISICO`) no la invalidan, pero quedan listados: `AR x1 (wp 93 no-fisico)` . Ver §17 (el veredicto).
- En `profiles/BZ_AutoDrive/AR_log.csv` , una fila por intervención de todas las corridas: fecha, ruta, vehículo, waypoint, tipo, evidencia, acción y resultado.

Si una ruta necesita el AutoRecovery **siempre en el mismo lugar**, eso es información: andá a ese punto y mirá qué hay. Suele ser un tramo de la grabación para rehacer.

Intervención manual. Si estás mirando una corrida y querés destrabarla vos, escribí el nombre de la ruta en `profiles/BZ_AutoDrive/BZ12_AR_MANUAL.txt`. El vehículo se re-ubica 10 m más adelante sobre la ruta, contados desde el último punto que pisó bien. Si ese lugar está ocupado, no hace nada.

Todos los valores se configuran por ruta en el Apéndice A.5.

9B. DayZ 1.30 — qué cambió en los vehículos, y las motos

9B.0 — Qué cambió en los vehículos con la 1.30 (también para autos)

La 1.30 tocó los vehículos por dentro. BZ_AutoDrive **1.2.1** ya lo resuelve, y el mismo paquete corre en la 1.29 y en la 1.30. Lo que te conviene saber como admin:

qué cambió	qué pasaba	qué hace AutoDrive 1.2.1
Las luces usan un sistema nuevo; la función vieja de los faros ya no existe	un mod que prende faros a la manera vieja **no compila** en la 1.30. AutoDrive 1.2 tampoco: **el server no levanta**	usa el sistema nuevo en la 1.30 y el viejo en la 1.29, y hace que los demás jugadores vean las luces del NPC
Borrar un vehículo con alguien sentado	en la 1.30 **tumba el server** (todos afuera)	al terminar una ruta, primero baja a los NPC y después borra el vehículo vacío. Si hay un **jugador** sentado, espera a que se baje
Dónde busca archivos el juego	con la 1.2, la **lista de rutas sale vacía** en la 1.30	lee las rutas de la carpeta del perfil de otra manera. Necesita que el server arranque con un `-profiles` **relativo** (p. ej. `-profiles=profiles`): con una ruta absoluta la lista sigue vacía y el RPT lo avisa (§15)
Los neumáticos tienen parámetros nuevos (fricción lateral y longitudinal, rugosidad, tipo)	—	**medido:** las rutas grabadas en la 1.29 se manejan igual en la 1.30, con la misma precisión
Quién simula el vehículo: con un jugador arriba, lo simula **su** cliente	—	nada que hacer: el NPC maneja igual con pasajeros jugadores en un auto
Las motos son una clase nueva, aparte de los autos	—	las maneja (ver abajo)
El velocímetro de un vehículo recién aparecido puede no dar un número durante unos segundos (medido en el M1025, de caja automática)	la ruta quedaba trabada en el arranque	usa la velocidad real del vehículo mientras dure y lo avisa en el RPT (`[Velocimetro] … no da un numero`)

> ⚠ **Si usás otro mod que borra vehículos** (limpieza, respawn, eventos): en la 1.30 **no tiene que borrar nunca un vehículo con alguien sentado**. Revisalo antes de actualizar el server.

Auto contra moto, de un vistazo:

	auto (`CarScript`)	moto (`MotorbikeScript` , 1.30)
reversa	sí (cambios de sentido, §9.4)	**no**: hacia atrás se camina
frenos	freno + freno de mano	freno = **delantero**; freno de mano = **trasero**
se cae	no	sí: el NPC la levanta
se inclina en las curvas	no	sí: la inclinación de tu toma es la referencia
caminarla	—	sí, con el motor prendido o apagado: el NPC la repite empujando
asiento de atrás / pasajeros	sí	no, en la 1.30
grabar y convertir	NUMPAD 5 y el wizard	igual
destrabe	la escalera del AutoRecovery (§9.6)	caminar 3 m atrás, y si no alcanza, re-ubicar

Las motos *(experimental)*

Desde la 1.30, DayZ trae motos, y **una moto no es un auto**. El juego las arma sobre una clase propia (`MotorbikeScript`), que es una rama aparte de la de los autos (`CarScript`). AutoDrive las reconoce **por esa jerarquía**, así que no hay que configurar nada por nombre:

- **Las dos motos vanilla** de la 1.30 (`Motorbike_01_*` , la Moped, y `Motorbike_02_*` , la Enduro) andan tal cual.
- **Una moto de mod hecha sobre el script nuevo** también anda, sin tocar nada.
- **Una moto de mod vieja, hecha como auto** (`CarScript`), sigue yendo por el camino de los autos, como hasta ahora.

Todo lo de motos existe **solo en la 1.30**. El mismo paquete corre en la 1.29 y ahí esa parte sencillamente no está.

9B.1 — Qué hace distinto el NPC con una moto

Grabás, convertís y lanzás igual que con un auto. Lo que cambia es cómo la maneja:

- **Los frenos.** El freno de la moto (la tecla de frenar) es el **delantero**, el principal; el freno de mano es el **trasero**. El NPC usa los dos así.
- **No tiene reversa.** Una ruta de moto no tiene cambios de sentido (§9.4). Para ir hacia atrás, la moto se **camina** (ver §9B.2).
- **Marchas y arranque.** Al arrancar, el juego a veces "pierde" el primer cambio: el NPC insiste hasta que entra. Arranca sin la patada, y sube la pata al tomar el mando (la baja al soltarla).
- **La inclinación.** En las curvas la moto se inclina. El NPC toma como referencia **cuánto te inclinaste vos en ese mismo punto**: mientras no se incline más que en tu toma (con 2° de margen), no frena ni recorta el volante.
- **La línea.** El conductor de motos corrige con amortiguamiento, para no zigzaguear, y con la sensibilidad de **esa** moto, que el wizard mide de tus tomas y guarda en su envelope (§6C). **Arriba de 65 km/h** corrige de otra manera: una moto tarda medio segundo en inclinarse cuando se gira el manubrio, así

que el NPC la inclina según la curva del camino que viene, en vez de corregir con el manubrio como un auto.

- **La llegada.** Frena hasta quedar quieta sobre tu punto final, midiendo en esa misma frenada cuánto frena de verdad la moto. En las pruebas llega a 1-4 cm, también entrando rápido.
- **Si se cae,** la levanta y vuelve a sentar al chofer.
- **Si choca de frente o se traba,** camina **3 m hacia atrás** y reintenta. Si vuelve a pasar en el mismo lugar, la re-ubica unos metros más adelante sobre la ruta, como el AutoRecovery del auto (§9.6). El log lo dice: `[MOTO] DESTRABE (choque de frente) en el arco 800 m: camina 3 m hacia ATRAS…`.

9B.2 — Cómo grabar en moto

Igual que un auto: **NUMPAD 5** para empezar y para terminar (§5, §7). Subite, prendé el motor, grabá, manejá y terminá **detenido**. **Terminá en recta**: los últimos metros con el manubrio derecho, frenando hasta detenerte. Si terminás doblando a paso de hombre, la moto puede quedar a medio metro del punto (§7.1).

Caminar la moto vale. Si la sacás de un lugar caminándola hacia atrás, o caminás un tramo corto hacia adelante, grabalo así, con el motor prendido. El wizard detecta esos tramos (`[moto] caminando: … se reproduce por empuje`) y el NPC los repite **a tu velocidad**, empujando la moto con el chofer sentado. Se ve caminando, con la animación del juego, **hacia atrás y hacia adelante**. Si la doblaste mientras la empujabas (por ejemplo, sacándola marcha atrás y encarándola para otro lado), el NPC reproduce ese giro y corrige si se corre de la línea. Después respeta la espera que hayas hecho y arranca en primera, a fondo.

Evitá puertas y portones. El estado de las puertas del mapa (abierta o cerrada) **cambia entre reinicios del server**. Si tu ruta pasa por un portón o termina adentro de un garaje, el día que esté cerrado la moto choca. Se destraba sola, pero la llegada ya no es limpia. AutoDrive **no abre puertas**: qué hacer con ellas es decisión tuya como admin. Grabá por caminos sin puertas.

Los escombros no son choques. Pasar por encima de escombros u objetos del piso queda registrado como contacto, pero sin pérdida de velocidad no cuenta como choque.

Avisos del wizard propios de la moto:

- `toma con el reloj roto … NO se convierte` : la toma se grabó con una versión vieja del grabador. Regrabala.
- `caminando: … se reproduce por empuje` : un tramo caminado. Es normal.
- `(!) toma con choque en el arco X m` : en la toma chocaste de verdad (perdiste velocidad contra algo). El NPC va a repetir tu línea **hacia** ese choque: regrabá ese tramo.

9B.3 — Límites (por qué dice experimental)

- **Probado** en la 1.30 experimental y en la **estable** (15/10): rutas completas con la Moped (con caminatas hacia atrás que giran) y con la Enduro **hasta 120 km/h** (desvío máximo 0,4-0,56 m en toda la ruta, llegada a 3-4 cm). Solo las dos motos vanilla: las de mods se suman a medida que sus autores las adapten a la 1.30.
- **El asiento de atrás** no se puede usar: en la 1.30 experimental el juego lo tenía a medio hacer (no lo volvimos a probar en la estable), y un acompañante queda encimado con el chofer. AutoDrive no sienta acompañantes en motos.

- **Los NPC de eAI en moto:** probados el 15/10 con Expansion 1.9.79. El chofer eAI maneja la Enduro y la Moped (la caminata también) y el retiro lo baja y borra igual que en un auto.

---

10. Eventos y secuencias — que pasen cosas en la ruta

Una ruta no es solo manejar de A a B: podés hacer que **pasen cosas** en puntos definidos — el vehículo se detiene, baja gente, suena un audio, aparecen bots. Eso se arma con el marcador **NUMPAD 4** + los eventos.

10.1 — Tu primera secuencia, paso a paso (de cero)

Objetivo: que el bus PARE en un punto, suene un mensaje, espere 3 segundos y siga. Lo hacemos de principio a fin.

Paso 1 — Grabá y marcá el punto. Manejás la ruta normal (NUMPAD 5 para empezar a grabar). Cuando llegás al lugar donde querés que pase algo (la parada), tocás **NUMPAD 4** *una vez*. Seguís manejando hasta el final y tocás **NUMPAD 5** para cortar.

> Cada NUMPAD 4 deja una marca en ese punto exacto. Podés marcar varios puntos en la misma vuelta.

Paso 2 — Pasala por el wizard. Corrés el wizard → **Convertir** → elegís tu grabación. Te genera la ruta y en el resumen te dice **cuántos eventos** quedaron (`eventos N`): uno por cada NUMPAD 4, en el orden en que los tocaste.

> El **número de waypoint (wp)** de cada marca lo ves en el paso siguiente, en el `_hdr.json` . Es la dirección de tu marca — la vas a usar para enganchar la acción.

Paso 3 — Dónde quedó la marca en el archivo. Abrí `BZBusRoute_<nombre>_hdr.json` en tu carpeta de rutas. El wizard ya dejó ahí, en la lista `Events` , un evento **por cada NUMPAD 4** (en orden), con su `wp` y la lista de acciones vacía. Si tocaste NUMPAD 4 una sola vez, hay uno solo; anotá su `wp` (en el ejemplo, 88):

```
{ "wp": 88, "trigger": { "type": "wp_reached", "wp": 88 }, "actions": [] }
```

Completás las acciones en ese mismo bloque.

> ⚠ **Editá el `_hdr.json` , no el `.json` completo.** Cuando existen `BZBusRoute_<nombre>_hdr.json` y `BZBusRoute_<nombre>_wp.csv` —que es lo normal, el wizard los genera— el server carga **ese par**, y el `.json` completo queda solo como respaldo. Los eventos que escribas en el `.json` no tienen efecto. *Si tocaste varias marcas:* la primera de la lista `Events` es tu primer NUMPAD 4, la segunda el segundo, y así.

Paso 4 — Enganchá la secuencia desde ahí. En el mismo JSON, agregás (o editás) el bloque `Events` , apuntando a ese wp:

```
"Events": [
  {
    "wp": 88,
    "trigger": { "type": "wp_reached", "wp": 88 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 0.5 },
      { "verb": "unfreeze_vehicle", "delay": 3.0 }
    ]
  }
]
```

> El **`"wp": 88` es el enganche**: conecta tu marca (el NUMPAD 4) con lo que pasa ahí. Cambiá `88` por el `wp` de tu marca. Cada acción puede llevar un `delay` en segundos, **contado desde que disparó el evento** (no desde la acción anterior).

> ⚠ **Cada verbo se deshace con su par, y un `delay` solo no detiene nada.** `delay` demora *esa* acción N segundos; no frena el vehículo. La espera siempre son dos acciones: una que detiene y su contraria con el `delay` puesto.
>
> ```
> freeze_vehicle   <->  unfreeze_vehicle     (clava el vehículo: freno + freno de mano)
> stop_route       <->  resume_route         (pausa el avance de la ruta)
> ```
>
> Mezclarlos deja el vehículo clavado para siempre: **`resume_route` no descongela** lo que congeló `freeze_vehicle`. Mientras está congelado, el cerebro de manejo no corre, así que la grabación no interfiere con la espera.

Paso 5 — Deploy y probá. Guardás el archivo y **relanzás la ruta desde el Reproductor** (LOAD & SPAWN): eso relee el `_hdr.json`. *(Un vehículo que ya estaba corriendo, o que se re-lanza solo, sigue con la versión anterior del archivo.)* El bus llega al wp 88 → **frena** → **suena el mensaje** → **espera 3s** → **sigue.** Esa es tu primera secuencia. 🎉

> *Para más acciones (bajar bots, prender motor, etc.) y más triggers (al acercarse un jugador, al recibir daño), seguí leyendo — abajo está la lista completa.*

10.2 — Cómo funciona

Cuando grabaste, tocaste NUMPAD 4 en los puntos importantes. Cada uno quedó como un **nodo de evento**. Después le colgás **acciones**: "*cuando el vehículo llegue acá, hacé esto*".

Cada evento tiene dos partes:

- **Cuándo dispara** (el *trigger*): al llegar a un waypoint, cuando un jugador se acerca al vehículo o a un punto de la ruta, cuando un jugador se sube, cuando el vehículo recibe daño, a los X segundos de

aparecer el vehículo.
- **Qué hace** (las *acciones*, los "verbos"): cada una con una demora opcional (`delay` , en segundos **desde que disparó el evento**) para coreografiar.

> ⚠️ **El evento NO obliga a parar.** Marcar un NUMPAD 4 **detenido** crea una **parada** real (`isStop`) — eso es lo que querés para un taxi. Pero un evento es solo *"trigger → acciones"*: podés colgarlo en **cualquier** waypoint (editando el `Events` del `_hdr.json` — §6B, Apéndice A) y que dispare **en pleno cruise, sin frenar** — para prender luces, tocar bocina, cargar el baúl o dejar una línea en el log al pasar. Y los triggers que **no** son `wp_reached` ni siquiera dependen de dónde esté el vehículo.

Ejemplos:

- *Taxi (parada real):* en cada NUMPAD 4 colgás `stop_route` → `play_sound` (`delay` 0,5, "subió un pasajero") → `resume_route` (`delay` 3). Para, levanta pasajero y sigue.
- *Arranca cuando alguien se sube:* dos eventos. Uno en el wp 0 con `freeze_vehicle` y `stop_engine` (con un `delay` que caiga después de la espera de arranque, por ejemplo 10 s): el bus espera **con el motor apagado**. Otro con el trigger `player_enter_vehicle` y `unfreeze_vehicle` : cuando un jugador se sienta, el bus **sale solo** (el motor se enciende solo al soltarlo).
- *Cambiar el inventario en ruta (`add_cargo`):* al llegar al depósito, mete cajas/suministros en el baúl — sin frenar si lo colgás mid-cruise.
- *Volverlo frágil en la emboscada (`set_vehicle_mortality` con `"mortal"`):* viene irrompible; en el punto caliente lo hacés **destructible** para que el jugador pueda reventarlo.
- *El vehículo como recompensa (`set_vehicle_parts` con `"unlocked"`):* viene **inviolable**: mientras el framework lo maneja, nadie le saca ruedas ni piezas. Cuando el jugador se lo gana, lo desbloqueás. Si el chofer muere y el vehículo queda de botín, el framework lo desbloquea solo.
- *Accidente (gancho de misión):* colgás `freeze_vehicle` y `stop_engine` (`delay` 1). El vehículo queda detenido con el motor apagado hasta que otro evento lo suelte con `unfreeze_vehicle` . *(`stop_engine` solo, sin congelar, dura medio segundo: el framework vuelve a encender el motor.)*

(El branching condicional —"si pasa X, hacé Y"— no vive acá: los eventos no tienen condiciones que un admin pueda usar. De eso se encarga el sistema de Quests, que se acopla aparte. Ver §11.)

10.3 — Referencia completa: disparadores, verbos y condiciones

Todo lo que acepta la lista `Events` , sacado del código de esta versión. Funciona igual con los dos cerebros; donde el efecto cambia, cada ficha lo dice en la línea **Cerebro 1.2**.

Lo básico: formato, `delay` y cuándo se revisan

Un **evento** es un "cuándo" más un "qué": un **trigger** (el disparador) y una lista de **acciones** (los verbos). Los eventos van en la lista `Events` del header de la ruta, **`BZBusRoute_<nombre>_hdr.json`** . Si existen el `_hdr.json` y el `_wp.csv` (lo normal: los genera el wizard), el server carga ese par y **el `.json` completo no se lee**: un evento escrito ahí no tiene efecto.

El wizard ya deja un evento por cada NUMPAD 4 que tocaste, con el trigger puesto y la lista de acciones vacía:

```
{ "wp": 88, "trigger": { "type": "wp_reached", "wp": 88 }, "actions": [] }
```

Formato de un evento

Campo	Tipo	Si falta	Qué es
`wp`	número entero	`0`	el waypoint del evento. Si no hay `trigger`, es el punto donde dispara
`trigger`	objeto	se usa `wp_reached` sobre `wp`	cuándo dispara (ver *Triggers*, abajo)
`actions`	lista de acciones	el evento no hace nada	qué hace (ver *Los verbos, uno por uno*, abajo)

Campos del `trigger`:

Campo	Tipo	Unidad	Si falta	Lo usan
`type`	texto	—	vacío = `wp_reached`	todos
`wp`	entero	índice de waypoint	`0`	`wp_reached`, `player_near_waypoint`
`radius`	decimal	metros	`0` (= nunca dispara)	`player_in_radius`, `player_near_waypoint`
`threshold`	decimal	fracción de salud, 0 a 1	`0` (= nunca dispara)	`vehicle_health_below`
`seconds`	decimal	segundos	`0` (= dispara enseguida)	`timer`

Campos de una acción (una acción usa sólo los que le corresponden; los demás se ignoran):

Campo	Tipo	Unidad	Si falta	Lo usan
`verb`	texto	—	la acción no hace nada y deja un aviso en el log	todos
`delay`	decimal	segundos	`0` = inmediato	todos
`msg`	texto	—	vacío	`log_event`, `ui_broadcast`
`format`	texto	—	`"toast"`	`ui_broadcast` (por dónde: `"toast"`, `"chat"` o `"both"`)
`value`	texto	—	vacío	`set_vehicle_mortality`, `set_driver_mortality`, `set_vehicle_parts`, `set_var`, `play_sound`, `spawn_guard`, `ui_broadcast`
`var`	texto	—	vacío	`set_var`
`items`	lista de `{ "cls": texto, "qty": entero }`	—	lista vacía	`add_cargo`
`count`	entero	bots	`1`	`spawn_guard`
`faction`	texto	—	`"Raiders"`	`spawn_guard`
`loadout`	texto	—	`"BanditLoadout"`	`spawn_guard`
`radius`	decimal	metros	`8` (`guard_stop`) · `150` (`ui_broadcast` con `"near"`)	`guard_stop`, `ui_broadcast`
`holdS`, `clearS`, `maxS`, `boardS`, `unreachableS`	decimal	segundos	`20`, `5`, `120`, `30`, `15`	`guard_stop`
`slot`	entero	—	`-1`	reservado: ningún verbo lo lee hoy
`fvalue`	decimal	—	`0`	reservado: ningún verbo lo lee hoy

> Los números van **sin comillas** y los textos **con comillas**. Los nombres de verbos y de triggers se comparan como texto exacto: `Freeze_Vehicle` o `freeze-vehicle` no existen. Qué pasa si un número viene entre comillas o el JSON tiene un error de sintaxis: *(sin verificar)*; mirá que la línea `[EVT-LOAD]` de cada evento aparezca al cargar la ruta.

Ejemplo: un evento con dos acciones demoradas

```
"Events": [
  {
    "wp": 120,
    "trigger": { "type": "wp_reached", "wp": 120 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 1.0 },
      { "verb": "unfreeze_vehicle", "delay": 8.0 }
    ]
  }
]
```

Qué pasa: al llegar al wp 120 el vehículo se clava (segundo 0); al segundo 1 suena el audio; **al segundo 8** se suelta y sigue. No es al segundo 9: cada `delay` se cuenta desde que disparó el evento.

Cómo funciona `delay`

- **Se cuenta desde el disparo del evento**, no desde la acción anterior. Las acciones **no se encadenan**.
- Las acciones sin `delay` (o con `0` o negativo) se ejecutan en el mismo instante del disparo, en el orden de la lista.
- Las demoradas se ejecutan cuando se cumple su tiempo: **el orden lo da el número, no la posición en la lista**. Una acción sin `delay` escrita al final corre antes que una demorada escrita arriba. Dos acciones con el mismo `delay`: en qué orden salen, *(sin verificar)*.
- Una acción demorada actúa sobre el vehículo que exista cuando le toca. Si ya no hay vehículo, no hace nada.
- Si el vehículo se despawnea o se respawnea antes, las acciones demoradas pendientes se cancelan. *(Si se cancelan todas las pendientes o sólo una: no verificado.)*
- Nada más las cancela: un `unfreeze_vehicle` demorado se ejecuta aunque en el medio haya pasado otra cosa (por ejemplo, un `guard_stop` a medias).

Cuándo se revisan los triggers

- **Cada 0,5 segundos**, por vehículo, recorriendo la lista `Events` en orden. En una misma revisión pueden disparar varios eventos.
- Se revisan también con el vehículo congelado, frenado por `stop_route`, pausado desde el panel, durante la espera de arranque (el *spawn-hold*) y aun con la ruta terminada, mientras el vehículo siga en el mundo.
- **No** se revisan mientras el chofer o la tripulación están subiendo al vehículo (el embarque animado, incluido el re-embarque de `guard_stop`). Se retoman cuando terminan de subir.
- Si el chofer muere y nadie puede manejar, el framework suelta el vehículo como botín y sus eventos dejan de revisarse.
- **Funcionan igual con los dos cerebros.** La revisión de eventos corre antes del punto donde el runner elige entre el cerebro 1.2 (el de siempre) y el 1.1. Lo que cambia en algunos verbos es el *efecto*: está en cada ficha, en la línea **Cerebro 1.2**.

Una vez por corrida; se re-arman en cada spawn

- **Cada evento dispara una sola vez por corrida.** Si el trigger se vuelve a cumplir, no pasa nada.
- **Cada vez que el vehículo nace** (respawn al final de la línea, relanzarla desde el Reproductor, reintento de spawn, respawn desde el panel o por la API) **todos los eventos se re-arman**, y además vuelve a cero todo lo que los verbos cambiaron: el vehículo sin congelar y sin `stop_route`, la mortalidad del vehículo y del chofer y el bloqueo de piezas a lo que dice el header (`VehicleInvincible`, `DriverInvincible`, `VehiclePartsLocked`), las variables de `set_var` vacías y el reloj de `timer` en cero.
- **El respawn automático reusa la ruta que ya está en memoria.** Si editás el `_hdr.json`, el cambio recién entra cuando la ruta se vuelve a cargar: relanzala desde el Reproductor (eso sí relee el archivo) o reiniciá el server.

Qué ves en el log del server (RPT)

- **Al cargar la ruta**, una línea por evento: `[EVT-LOAD] Event <n> wp=<wp> actions=<cantidad> trigger=type='<tipo>' verb0='<primer verbo>'`. Si el tipo de trigger no existe: `evento <n> wp=<wp>: trigger '<tipo>' desconocido -> nunca dispara (validos: ...)`.
- **Al disparar**, una línea por acción: `[EVENT <n> @ wp <m>] <verbo>: ...`. `<n>` es la posición del evento en la lista, **empezando en 0**. Estas líneas **no llevan el nombre de la ruta**: con varios vehículos andando, no dicen de cuál son.
- **Verbo mal escrito:** `[EVENT <n>] verbo no implementado todavia: '<verbo>'`. Sale **cuando el evento dispara**, no al cargar.

Los verbos (resumen)

Todos aceptan además `delay`.

Verbo	Qué hace	Campos
`add_cargo`	mete ítems en el baúl	`items` (`cls` , `qty`)
`log_event`	escribe una línea en el log del server	`msg`
`ui_broadcast`	aviso, sólo para los admins por default; también todos, los que están cerca o los pasajeros, en pantalla o en el chat	`msg` , `value` , `format` , `radius`
`play_sound`	sonido 3D pegado al vehículo, una vez	`value` (nombre del SoundSet)
`freeze_vehicle`	clava el vehículo: freno a fondo + freno de mano, sin avanzar	—
`unfreeze_vehicle`	suelta lo que clavó `freeze_vehicle`	—
`stop_route`	detiene el avance con freno a fondo (sin freno de mano, motor en marcha)	—
`resume_route`	levanta el `stop_route`	—
`start_engine`	enciende el motor	—
`stop_engine`	apaga el motor (sólo dura con el vehículo congelado)	—
`set_vehicle_mortality`	vehículo destructible o irrompible	`value` (`"mortal"` o cualquier otra cosa)
`set_driver_mortality`	chofer mortal o invencible	`value` (`"mortal"` o cualquier otra cosa)
`set_vehicle_parts`	vehículo inviolable (nadie le saca piezas) o con piezas que se pueden sacar	`value` (`"unlocked"` o cualquier otra cosa)
`repair_vehicle`	lleva la salud general del vehículo al máximo	—
`refuel`	llena el tanque	—
`drain_fuel`	vacía el tanque	—
`lights_on`	prende los faros	—
`lights_off`	apaga los faros	—
`horn`	un bocinazo corto (0,6 s)	—
`set_var`	guarda una variable de la corrida (**hoy nadie la lee**)	`var` , `value`
`spawn_guard`	crea bots armados parados al costado del vehículo	`value` (clase), `count` , `faction` , `loadout`
`dismount_guard`	baja a la tripulación (`Crew`) y la deja patrullando un anillo de 8 m	—
`guard_stop`	parada con escolta completa: frena, baja la tripulación, espera, la sube y sigue	`radius` , `holdS` , `clearS` , `maxS` , `boardS` , `unreachableS`
`despawn_vehicle`	borra vehículo, chofer y bots; no respawnea	—

Verbo	Qué hace	Campos
`lock_seat` · `unlock_seat` · `eject_passenger`	**reservados: no hacen nada** (sólo un aviso en el log)	(`slot` , sin uso)

Los verbos, uno por uno

`add_cargo`

Mete ítems en el baúl del vehículo. Lo que **se apila** (el propio ítem lo dice: su cantidad máxima es mayor que 1) se crea como **un solo ejemplar** con `qty` unidades, topeado al máximo de ese ítem. Lo que **no se apila** (armas, herramientas, ruedas) se crea **`qty` veces**. Lo que no entra se **descarta**: nunca tira nada al piso.

- **Campos:** `items` , una lista de `{ "cls": "<classname>", "qty": <entero> }` . `qty` si falta o es menor que 1 vale 1.
- **Si falla:** un classname que no existe (o un baúl lleno) deja `[CARGO] No se pudo crear '<cls>' (baul lleno o classname invalido?)` y sigue con el siguiente ítem. Si pediste más de lo que apila: `se pidieron N y la pila topa en M` . Si en los no apilables se llena el baúl: `entraron K de N (baul lleno). El resto se descarta` . Sin `items` o con la lista vacía, no crea nada.
- **Ejemplo:**

```
{ "verb": "add_cargo", "items": [ { "cls": "Rag", "qty": 6 }, { "cls": "Hammer", "qty": 2 } ] }
```

- **Trampas:** sin anidado: un arma con cargador puesto aparece vacía; meté el cargador como ítem aparte. Para ítems cuya "cantidad" no es un conteo (una botella mide líquido), `qty` se lee como cantidad del ítem, no como número de botellas *(no verificado en juego)*. Cómo se comportan munición y cargadores con `qty` *(sin verificar)*.
- **Cerebro 1.2:** sí, no depende del cerebro.

`log_event`

Escribe una línea en el log del server (RPT) con tu mensaje. No se ve en el juego. Sirve para marcar en el log que la ruta pasó por un punto.

- **Campos:** `msg` (texto). Sin `msg` , la línea sale vacía.
- **Qué aparece:** `[EVENT <n> @ wp <m>] log_event: <msg>` .
- **Ejemplo:**

```
{ "verb": "log_event", "msg": "El convoy cruzo el puente" }
```

- **Cerebro 1.2:** sí.

`ui_broadcast`

Manda tu mensaje **a quien elijas**, en pantalla o en el chat. Sin `value` va sólo a los admins: para avisarles algo a los jugadores, elegí el destinatario. Qué ve cada uno, en §13.3.

- **Campos:** `msg` (texto); `value`, a quién: `"all"` (todos los conectados), `"near"` (a menos de `radius` metros del vehículo; sin `radius`, 150 m), `"passengers"` (sentados en el vehículo), `"admins"` o vacío (los admins); `format`, por dónde: `"toast"` (pantalla, default), `"chat"` o `"both"`.
- **Ejemplo** (a todos, en pantalla, al salir de la terminal):

```
{ "verb": "ui_broadcast", "value": "all", "format": "toast", "msg": "The bus left the terminal" }
```

- **Trampas:** un `value` mal escrito va **sólo a los admins** (a propósito: un error de tipeo nunca le muestra a los jugadores algo que era para vos); un `format` mal escrito sale en pantalla. Cada jugador lo recibe una vez aunque cumpla dos condiciones (un admin que va de pasajero). El RPT dice a cuántos llegó: `[EVENT n @ wp X] ui_broadcast (all → 3 jugadores, toast): <msg>`.
- **Cerebro 1.2:** sí.

`play_sound`

Reproduce un sonido **una vez**, en 3D, pegado al vehículo (se mueve con él). El sonido lo declarás vos en tu addon (`CfgSoundShaders` + `CfgSoundSets`).

- **Campos:** `value` = el nombre del **SoundSet**. Sin `value`, no hace nada (ni siquiera deja línea en el log).
- **Ejemplo:**

```
{ "verb": "play_sound", "value": "MiVoz_SoundSet" }
```

- **Trampas:** el sonido se lanza desde el server; el propio código avisa que **puede no llegar a los clientes** y pide verificarlo en el juego *(sin verificar)*. Si el SoundSet no existe, qué pasa *(sin verificar)*.
- **Cerebro 1.2:** sí, no depende del cerebro.

`freeze_vehicle`

Clava el vehículo donde está: a cada tick le pone freno a fondo y freno de mano, no avanza la ruta y **el cerebro de manejo no corre** (ni el 1.2 ni el 1.1). Es la parada "dura". Mientras está congelado, el framework **no le vuelve a encender el motor**: es el único estado donde `stop_engine` dura.

- **Campos:** ninguno.
- **Par:** `unfreeze_vehicle`. `resume_route` **no lo suelta.**
- **Ejemplo:**

```
{ "verb": "freeze_vehicle" }
```

- **Trampas:**
  - Durante la espera de arranque (spawn-hold) el congelado todavía no manda: esa fase va primero (motor encendido, freno puesto) y el vehículo pasa a manejar igual; recién ahí queda clavado. Un evento al principio de la ruta con `freeze_vehicle` funciona, pero un `stop_engine` al mismo tiempo se deshace (ver `stop_engine`).
  - `guard_stop` con la parada dura (sin el cerebro 1.2) usa el **mismo** congelado: al terminar lo suelta, aunque vos hayas congelado aparte. Con el 1.2 frena suave y no lo toca.

    - Los eventos se siguen revisando con el vehículo congelado: un `timer` o un `player_in_radius` pueden soltarlo.
- **Cerebro 1.2:** sí. Con el vehículo congelado el actuador del 1.2 se hace a un lado y se aplican freno y freno de mano.

`unfreeze_vehicle`

Suelta el congelado de `freeze_vehicle`: saca el freno de mano y el vehículo retoma la ruta.

- **Campos:** ninguno.
- **Par:** `freeze_vehicle`. **No levanta un** `stop_route`: si también había un `stop_route`, sigue detenido hasta un `resume_route`.
- **Ejemplo:**

```
{ "verb": "unfreeze_vehicle", "delay": 10 }
```

- **Trampas:** también suelta el congelado de un `guard_stop` **duro** en curso: el vehículo se va con los guardias afuera.
- **Cerebro 1.2:** sí.

`stop_route`

Detiene el avance: a cada tick pone **freno a fondo**, **sin freno de mano**, con el **motor encendido** (si está apagado, lo enciende). El cerebro de manejo no corre mientras dure. Es la parada "blanda".

- **Campos:** ninguno.
- **Par:** `resume_route`. `unfreeze_vehicle` **no lo levanta.**
- **Ejemplo:**

```
{ "verb": "stop_route" }
```

- **Trampas:** como vuelve a encender el motor a cada tick, **no sirve para esperar con el motor apagado** (para eso: `freeze_vehicle` + `stop_engine`).
- **Cerebro 1.2:** sí, igual que `freeze_vehicle` (el actuador del 1.2 se hace a un lado).

`resume_route`

Levanta el `stop_route` y el vehículo sigue.

- **Campos:** ninguno.
- **Par:** `stop_route`. **No descongela** lo que congeló `freeze_vehicle`: si mezclás los pares, el vehículo queda clavado.
- **Ejemplo:**

```
{ "verb": "resume_route", "delay": 3 }
```

- **Trampas:** no reabre una ruta que ya terminó.

- **Cerebro 1.2:** sí.

`start_engine`

Enciende el motor si está apagado.

- **Campos:** ninguno.
- **Par:** `stop_engine`.
- **Ejemplo:**

```
{ "verb": "start_engine" }
```

- **Trampas:** casi nunca hace falta: el framework enciende el motor solo a cada tick en todos los estados salvo el congelado. Tras un `unfreeze_vehicle` el motor se enciende igual.
- **Cerebro 1.2:** sí.

`stop_engine`

Apaga el motor si está encendido.

- **Campos:** ninguno.
- **Par:** `start_engine`.
- **Ejemplo** (esperar con el motor apagado; el `delay` deja pasar el `freeze_vehicle`):

```
{ "verb": "stop_engine", "delay": 2 }
```

- **Trampas: solo, dura medio segundo.** A cada tick el framework vuelve a encender el motor, manejando, con `stop_route`, en pausa y en la espera de arranque. **Sólo queda apagado con el vehículo congelado** (`freeze_vehicle` antes). Al principio de la ruta, dale un `delay` que caiga después de la espera de arranque (`SpawnHoldSeconds` más la carga de la ruta): durante esa espera el motor se vuelve a encender aunque esté congelado. *(Esto último sale del orden del código; no está probado en juego.)*
- **Cerebro 1.2:** igual que en el 1.1: el encendido automático corre antes de elegir cerebro.

`set_vehicle_mortality`

Cambia el vehículo entre **destructible** e **irrompible**. El framework lo vuelve a aplicar a cada tick, así que el cambio se sostiene. Irrompible, además, se auto-repara si llega a arruinarse.

- **Campos:** `value`. Sólo `"mortal"` (exacto, en minúscula) lo hace destructible. **Cualquier otra cosa, o no poner `value`, lo hace irrompible.** Para que se lea claro, usá `"invincible"`.
- **Par:** el mismo verbo con el otro valor. El estado inicial de cada corrida lo da `VehicleInvincible` del header (por defecto `true`).
- **Ejemplo:**

```
{ "verb": "set_vehicle_mortality", "value": "mortal" }
```

- **Trampas:** `"Mortal"` con mayúscula lo deja **irrompible**. `vehicle_health_below` sólo dispara con el vehículo destructible.
- **Cerebro 1.2:** sí.

`set_vehicle_parts`

Cambia el vehículo entre **inviolable** (los jugadores no le pueden sacar ruedas, puertas, batería, bujía, radiador ni ninguna otra pieza) y **con piezas que se pueden sacar**. Con el vehículo parado (en una parada, en una meseta, al final de la ruta), el juego deja sacarle piezas como a cualquier auto: dos ruedas menos y queda detenido para siempre; y "quitar rueda" compite con "subir" en la puerta del conductor. Inviolable, el cartel ni aparece. **No toca la carga**: lo que va en el baúl sigue accesible, y subir, bajar y abrir puertas funcionan igual.

- **Campos:** `value`. Sólo `"unlocked"` (exacto, en minúscula) deja sacar piezas. **Cualquier otra cosa, o no poner `value`, lo deja inviolable.** Para que se lea claro, usá `"locked"`.
- **Par:** el mismo verbo con el otro valor. El estado inicial de cada corrida lo da `VehiclePartsLocked` del header (por defecto `true`).
- **Ejemplo** (el vehículo es la recompensa):

```
{ "verb": "set_vehicle_parts", "value": "unlocked" }
```

- **Trampas:** `"Unlocked"` con mayúscula lo deja **inviolable**. Si el chofer muere y el vehículo queda de botín, el framework lo desbloquea solo. Un mod de vehículos que cambie las reglas de sus piezas sin respetar las de la clase base se puede saltear el bloqueo.
- **Cerebro 1.2:** sí.

`set_driver_mortality`

Lo mismo para el **chofer** (el que está manejando en ese momento). No toca a la tripulación (`Crew`) ni a los bots de `spawn_guard`.

- **Campos:** `value`, igual que el anterior: `"mortal"` lo hace mortal; cualquier otra cosa, invencible.
- **Par:** el mismo verbo con el otro valor. Estado inicial: `DriverInvincible` del header (por defecto `true`).
- **Ejemplo:**

```
{ "verb": "set_driver_mortality", "value": "mortal" }
```

- **Trampas:** para que otro de la tripulación tome el volante cuando matan al chofer (bloque `Drivers` del header), el chofer tiene que ser mortal.
- **Cerebro 1.2:** sí.

`repair_vehicle`

Lleva la **salud general** del vehículo al máximo.

- **Campos:** ninguno.
- **Ejemplo:**

```
{ "verb": "repair_vehicle" }
```

- **Trampas:** el código sólo toca la salud general. Las partes (ruedas, batería, radiador…) tienen su propia salud; si las repara o no, y si repara las zonas de daño del motor o la carrocería, *(sin verificar)*.
- **Cerebro 1.2:** sí.

`refuel`

Llena el tanque de combustible (pide una cantidad enorme y el juego la recorta a la capacidad del tanque).

- **Campos:** ninguno.
- **Par:** `drain_fuel`.
- **Ejemplo:**

```
{ "verb": "refuel" }
```

- **Cerebro 1.2:** sí.

`drain_fuel`

Vacía el tanque. El motor se termina parando por falta de combustible.

- **Campos:** ninguno.
- **Par:** `refuel`.
- **Ejemplo:**

```
{ "verb": "drain_fuel" }
```

- **Trampas:** el framework sigue intentando encender el motor a cada tick. Con el cerebro 1.2, un vehículo que no avanza se toma como trabado y arranca el AutoRecovery; si eso termina cerrando la ruta *(sin verificar)*. Si querés un vehículo "sin nafta" quieto y prolijo, congelalo.
- **Cerebro 1.2:** el verbo se ejecuta; la reacción del cerebro *(sin verificar)*.

`lights_on`

Prende los faros.

- **Campos:** ninguno.
- **Par:** `lights_off`.
- **Ejemplo:**

```
{ "verb": "lights_on" }
```

- **Trampas:** convive con `LightsMode` del header, y el verbo **manda**:
  - Con los dos cerebros, un `lights_on` / `lights_off` **manda sobre** `LightsMode` **hasta el próximo spawn**: después, en cada spawn, vuelve a decidir `LightsMode`.

- El framework manda un pulso por red para que los demás jugadores vean el haz estable, también con el cerebro 1.2.
- Para que se vean, el vehículo necesita batería y faros instalados.

`lights_off`

Apaga los faros. Mismas reglas que `lights_on`.

- **Campos:** ninguno.
- **Par:** `lights_on`.
- **Ejemplo:**

```
{ "verb": "lights_off" }
```

- **Cerebro 1.2:** ver `lights_on`.

`horn`

Un bocinazo corto: la bocina suena 0,6 segundos y se corta sola.

- **Campos:** ninguno (si ponés `value`, se ignora).
- **Ejemplo:**

```
{ "verb": "horn" }
```

- **Cerebro 1.2:** sí.

`set_var`

Guarda una variable de texto de la corrida (`var` = `value`).

- **Campos:** `var` (nombre), `value` (valor). Los dos texto.
- **Ejemplo:**

```
{ "verb": "set_var", "var": "fase", "value": "emboscada" }
```

- **Trampas: hoy nadie lee estas variables.** La única parte del código que las lee (la condición `var_equals`, ver *Condiciones*, abajo) no se llama desde ningún lado. En la práctica el verbo sólo deja la línea `set_var: fase = emboscada` en el log. Las variables se borran en cada spawn.
- **Cerebro 1.2:** se ejecuta igual; no tiene efecto en ninguno de los dos.

`spawn_guard`

Crea bots armados **parados al costado derecho** del vehículo: el primero a 2,5 m y cada uno de los siguientes 1,5 m más afuera. Quedan en un grupo propio de la facción que pongas, con el loadout que pongas. **No suben** al vehículo ni lo siguen.

- **Campos:**

  - `value`: la clase del bot. Si falta, `eAI_SurvivorM_Boris`.
  - `count`: cuántos. Si falta o es 0, 1.
  - `faction`: la facción de eAI. Si falta, `Raiders` (hostil al jugador, amiga del chofer).
  - `loadout`: el loadout de Expansion con que se arman. Si falta, `BanditLoadout`.
- **Si falla:** una clase que no se puede crear deja `[spawn_guard] no se pudo crear '<clase>'` y sigue con el próximo.
- **Ejemplo:**

```
{ "verb": "spawn_guard", "count": 3, "faction": "Raiders", "loadout": "BanditLoadout" }
```

- **Trampas:** no son la tripulación (`Crew`): `dismount_guard` y `guard_stop` no los mandan. El código no los hace invencibles. Se **borran** cuando el vehículo se despawnea o se respawnea.
- **Cerebro 1.2:** sí, no depende del cerebro.

`dismount_guard`

Baja a la **tripulación** (los bots de `Crew`) que esté sentada, salvo el que maneja. Bajan animados (abren la puerta), pasan a ser **mortales**, quedan en un grupo propio y **patrullan un anillo de 8 m** alrededor de donde está el vehículo, con una correa a 10 m. Si ven una amenaza pelean; al calmarse vuelven al anillo.

- **Campos:** ninguno (el radio es fijo: 8 m).
- **Ejemplo** (primero clavarlo, después bajar):

```
{ "verb": "dismount_guard", "delay": 3 }
```

- **Trampas:**
  - **El vehículo tiene que estar detenido**: DayZ no deja bajar con el vehículo en movimiento. Usalo después de un `freeze_vehicle`, con `delay`.
  - **No vuelven solos.** No hay verbo para volver a subirlos: si soltás el vehículo, se va sin ellos. El ciclo completo (bajar, esperar y volver a subir) es `guard_stop`.
  - Se **borran** con el vehículo cuando se despawnea o se respawnea, aunque ya estén abajo.
  - La facción del grupo es la del **primer** bot de `Crew` (o `Raiders`).
- **Cerebro 1.2:** sí, no depende del cerebro (pero necesita el vehículo congelado).

`guard_stop`

La **parada con escolta entera** en un solo verbo:

1. **Frena.** Con el cerebro 1.2, una **parada suave**: sigue la línea y frena hasta quedar quieto en un punto unos metros adelante (si la toma tiene una parada grabada ahí, en ella), y el despliegue arranca con el vehículo quieto. Sin el 1.2, en moto, o si la suave no se puede en ese momento (una recuperación en curso, el final pegado), **congela** el vehículo y espera a que baje de 3 km/h (mira cada 0,5 s, hasta 10 s; pasado eso despliega igual).
2. **Despliega**: igual que `dismount_guard`, pero con el anillo de `radius` metros (correa 2 m más afuera).
3. **Espera** y sale por la primera de estas tres:
   - **zona limpia**: pasaron `holdS` segundos **y** van `clearS` segundos seguidos sin combate;

    - **amenaza fuera de alcance**: hay combate pero ningún blanco a menos de `radius` + 5 m, durante `unreachableS` segundos;
    - **tope**: pasaron `maxS` segundos, pase lo que pase.
4. **Repliegue**: los vivos caminan de vuelta a sus asientos, hasta el vehículo donde esté.
5. **Embarque**: espera a que estén todos los vivos sentados, o `boardS` segundos como máximo. El que no llegó, **se queda**.
6. **Suelta la parada** y la ruta sigue desde ahí.

"Combate" es: algún bot de la tripulación con amenaza alta (el mismo umbral con que eAI entra a pelear) o el grupo en combate.

- **Campos** (todos en decimal; 0 o negativo = el default): `radius` 8 m · `holdS` 20 s · `clearS` 5 s · `maxS` 120 s · `boardS` 30 s · `unreachableS` 15 s.
- **Ejemplo:**

  ```
  { "verb": "guard_stop", "radius": 10, "holdS": 30, "clearS": 5, "maxS": 180, "boardS": 40,
  "unreachableS": 15 }
  ```

  `{ "verb": "guard_stop" }` solo ya anda con los defaults.
- **Qué ves en el log:** `[MISION] wp N: guard_stop · parada SUAVE (siguiendo la linea hasta el hold) · radio ...` (o `parada DURA (congelado)`), `despliegue X/Y`, `combate -> retengo`, `zona limpia tras ... -> repliegue` (o `amenaza fuera de la correa` o `tope de`), `embarcados X/Y ... -> sigue`.
- **Trampas:**
    - Necesita tripulación declarada en `Crew` del header. Sin `Crew` no baja nadie: el vehículo queda parado unos `holdS` segundos y sigue *(sale del código; no probado en juego)*.
    - Con la parada **dura** usa el **mismo** congelado que `freeze_vehicle`: al terminar lo suelta aunque vos lo hayas congelado aparte, y un `unfreeze_vehicle` en el medio suelta el vehículo con los guardias afuera. Con la parada suave (cerebro 1.2) el congelado no se toca.
    - **Dónde ponerlo** *(medido)*: un `guard_stop` donde la toma no frena, justo antes de una rampa final, deja al vehículo sin el envión de la toma para subirla (una cuesta del 8-10 % en los últimos metros lo clavó 6 de 7 veces). Ponelo en llano y lejos del final, o mejor: **marcá la parada con NUMPAD 4 al grabar** y el vehículo frena como vos.
    - Mientras la tripulación re-embarca, los eventos no se revisan.
    - La facción del grupo desplegado es la del primer bot de `Crew`; el campo `faction` de los demás no se usa. Con `Deploy.IgnoreInfected` (por defecto `true`), los guardias y los infectados se ignoran entre sí.
- **Cerebro 1.2:** sí (funciona sobre el mismo congelado).

`despawn_vehicle`

Fin de la historia: a los 50 ms borra el vehículo, el chofer, la tripulación (aunque ya haya bajado) y los bots de `spawn_guard`, cancela lo pendiente (acciones demoradas, respawn) y manda `BZ AutoDrive vehicle stopped.` a los admins. **No respawnea.**

- **Campos:** ninguno.
- **Ejemplo:**

```
{ "verb": "despawn_vehicle", "delay": 60 }
```

- **Trampas:** no hay vuelta atrás: la ruta no se re-lanza sola. Para mods que usan la API: el runner no sale del registro.
- **Cerebro 1.2:** sí (además corta el cerebro 1.2).

`lock_seat` · `unlock_seat` · `eject_passenger` (reservados)

No hacen nada. Existe el campo `slot` para ellos, pero el despachador no tiene código para estos verbos: cuando el evento dispara, dejan en el log `verbo no implementado todavia` y siguen. Cualquier otro nombre que no esté en esta referencia se comporta igual.

Los disparadores (triggers)

Trigger	Dispara cuando…	Campos
`wp_reached` (o `waypoint`, o sin trigger)	el vehículo llega a un waypoint (o lo pasa)	`wp`
`player_in_radius`	un jugador real está a menos de `radius` metros **del vehículo**	`radius`
`player_near_waypoint`	un jugador real está a menos de `radius` metros **de un waypoint fijo**	`wp`, `radius`
`player_enter_vehicle`	un jugador real está adentro del vehículo	—
`vehicle_health_below`	la salud general del vehículo baja de `threshold`	`threshold`
`timer`	pasaron `seconds` segundos desde que nació el vehículo	`seconds`

Para todos: se revisan cada 0,5 s, **disparan una sola vez por corrida** y **se re-arman en cada spawn** (ver *Lo básico*). Un tipo que no existe (o mal escrito) avisa al cargar la ruta y **nunca dispara**. "Jugador real" es un jugador conectado: los bots de eAI (el chofer, la tripulación, los guardias) no cuentan.

`wp_reached`

Dispara cuando el índice de waypoint del vehículo es **mayor o igual** al pedido. Como es "mayor o igual", si el vehículo salta puntos (una re-ubicación del AutoRecovery) el evento dispara igual apenas los pasa. Es el trigger que escribe el wizard. `waypoint` es otro nombre para lo mismo, y un evento **sin** `trigger` (o con `type` vacío) también es un `wp_reached`.

- **Campos:** `wp` (entero, índice de waypoint). Si el `wp` del trigger es 0 o falta, se usa el `wp` del evento.
- **Una vez / re-armado:** una vez por corrida; se re-arma en cada spawn.
- **Desde cuándo:** desde el primer tick en que se revisan eventos. Un evento en el wp 0 (o sin `wp` ni trigger) **dispara apenas nace el vehículo**, durante la espera de arranque.
- **Ejemplo:**

```
{ "wp": 340, "trigger": { "type": "wp_reached", "wp": 340 }, "actions": [ { "verb": "horn" } ] }
```

- **Trampas:**
  - Si querés un evento en el wp 0 usando el `wp` del trigger, tenés que poner también `"wp": 0` en el evento: un `wp` 0 en el trigger no cuenta y manda el del evento.
  - Con el **cerebro 1.2** el índice es el cursor del cerebro (el punto más cercano de su propia copia de la ruta). Esa copia descarta los puntos donde la grabación retrocede (glitches). En una ruta limpia los números coinciden; si al cargarla el RPT dice `[Route] saneo de retrocesos: N` con N mayor que 0, los puntos posteriores a un descarte quedan corridos y **el evento puede disparar unos puntos más tarde** *(cuánto: no verificado)*.

`player_in_radius`

Dispara cuando **algún jugador real** está a `radius` metros o menos **del vehículo** (distancia en 3D, altura incluida). El círculo se mueve con el vehículo.

- **Campos:** `radius` (decimal, metros). 0 o sin poner = nunca dispara.
- **Una vez / re-armado:** una vez por corrida.
- **Ejemplo:**

```
{ "wp": 0, "trigger": { "type": "player_in_radius", "radius": 25 }, "actions": [ { "verb": "stop_route" }, { "verb": "resume_route", "delay": 10 } ] }
```

- **Trampas:** dispara también durante la espera de arranque, si hay alguien cerca del punto de salida.

`player_near_waypoint`

Una **zona fija** anclada a un waypoint de la ruta: dispara cuando algún jugador real está a `radius` metros o menos de la posición de ese waypoint, **esté donde esté el vehículo**. Sirve para emboscadas o zonas fuera del camino.

- **Campos:** `wp` (entero: el waypoint que hace de centro; el 0 vale), `radius` (decimal, metros). Un `wp` fuera de la ruta o `radius` 0 = nunca dispara.
- **Una vez / re-armado:** una vez por corrida.
- **Ejemplo:**

```
{ "wp": 500, "trigger": { "type": "player_near_waypoint", "wp": 500, "radius": 60 }, "actions": [ { "verb": "spawn_guard", "count": 2 } ] }
```

- **Trampas:** acá el número es la fila del archivo de la ruta tal cual, sin el corrimiento del cerebro 1.2 que puede afectar a `wp_reached`.

`player_enter_vehicle`

Dispara cuando **algún jugador real está adentro** del vehículo, en cualquier asiento.

- **Campos:** ninguno.
- **Una vez / re-armado:** una vez por corrida: **sólo el primero que sube**. Si un jugador ya está adentro cuando se revisan los eventos, dispara enseguida.

- **Ejemplo** (esperar con el motor apagado hasta que alguien suba; el evento de espera va aparte, ver *Trampas*, abajo):

```
{ "wp": 0, "trigger": { "type": "player_enter_vehicle" }, "actions": [ { "verb": "unfreeze_vehicle" }, { "verb": "ui_broadcast", "msg": "Subio un pasajero" } ] }
```

`vehicle_health_below`

Dispara cuando la **salud general** del vehículo (de 0 a 1) queda **por debajo** de `threshold`.

- **Campos:** `threshold` (decimal, de 0 a 1; `0.5` = la mitad). 0 o sin poner = nunca dispara.
- **Una vez / re-armado:** una vez por corrida.
- **Ejemplo:**

```
{ "wp": 0, "trigger": { "type": "vehicle_health_below", "threshold": 0.6 }, "actions": [ { "verb": "freeze_vehicle" }, { "verb": "guard_stop" } ] }
```

- **Trampas: con el vehículo irrompible (el default) casi nunca dispara**: el framework le bloquea el daño a cada tick. Poné `"VehicleInvincible": false` en el header, o un `set_vehicle_mortality` con `"mortal"` antes.

`timer`

Dispara cuando pasaron `seconds` segundos **desde que nació el vehículo**. No es desde que arranca a manejar: el reloj incluye la espera de arranque (`SpawnHoldSeconds`), la carga de la ruta y la subida de la tripulación.

- **Campos:** `seconds` (decimal, segundos). 0 = en la primera revisión.
- **Una vez / re-armado:** una vez por corrida; el reloj vuelve a cero en cada spawn. Si el punto de salida estaba ocupado y el runner esperó, el reloj cuenta desde que el vehículo nació de verdad.
- **Ejemplo:**

```
{ "wp": 0, "trigger": { "type": "timer", "seconds": 600 }, "actions": [ { "verb": "despawn_vehicle" } ] }
```

- **Trampas:** como los eventos no se revisan mientras la tripulación sube, un timer corto puede salir más tarde que su segundo exacto. Resolución: medio segundo.

Condiciones ("si pasa X, hacé Y")

En el código existe una clase de **condición** con estos tipos: `AND`, `OR`, `NOT` (combinan otras condiciones), `player_in_radius`, `player_in_vehicle`, `vehicle_health_below`, `wp_reached`, `var_equals` (compara una variable de `set_var`), `engine_running`, `route_stopped` y `vehicle_frozen`. También existe la función que las evalúa.

Pero no se usan en ningún lado accesible:

- **Ninguna parte del código llama a esa función.**
- **Ningún campo del JSON de la ruta acepta una condición.** Un evento tiene sólo `wp`, `trigger` y `actions`; una acción no tiene campos de condición.
- El verbo `check_once` (con `condition`, `on_true`, `on_false`) **se sacó el 11/06/2026**: tener una acción adentro de otra acción, o una condición adentro de otra, rompía la carga del JSON y la lista `Events` entera no cargaba (ni los eventos simples). Desde entonces la acción es plana.

Conclusión: un admin no puede usar condiciones desde el JSON de la ruta. Si escribís una clave `condition` en un evento, se ignora *(no verificado cómo trata el cargador una clave desconocida)*. Por lo mismo, `set_var` **hoy no tiene efecto**: nadie lee sus variables. El "si pasa X, hacé Y" es territorio de Expansion Quests.

Trampas de los eventos

1. **Editá el** `_hdr.json`, no el `.json` completo. Y después **relanzá la ruta desde el Reproductor**: el respawn automático reusa la ruta en memoria y no ve tus cambios.
2. `delay` **se cuenta desde el disparo del evento**, no desde la acción anterior. `freeze` + `unfreeze` con `delay: 5` = 5 segundos quieto, aunque en el medio pongas otras acciones demoradas.
3. **Cada verbo se deshace con su par.** `freeze_vehicle` ↔ `unfreeze_vehicle`; `stop_route` ↔ `resume_route`. `resume_route` **no descongela** y `unfreeze_vehicle` **no levanta un** `stop_route`: mezclarlos deja el vehículo detenido para siempre. `guard_stop` usa el mismo congelado que `freeze_vehicle` y lo suelta al terminar.
4. **Un** `delay` **solo no detiene nada.** La espera siempre son dos acciones: una que detiene y su par con `delay`.
5. `stop_engine` **solo dura medio segundo.** El motor se vuelve a encender a cada tick salvo con el vehículo **congelado**. Esperar con el motor apagado = `freeze_vehicle` + `stop_engine` (con `delay` si es al principio de la ruta); arrancar = `unfreeze_vehicle` (el motor se enciende solo). Con `stop_route` el motor no se puede apagar.
6. `ui_broadcast` **sin** `value` **lo ven sólo los admins** (`AdminSteamIDs`). Para los jugadores, elegí `"all"`, `"near"` o `"passengers"` (§13.3).
7. `set_var` **no tiene efecto**: nadie lee las variables y no hay condiciones usables (ver *Condiciones*, arriba).
8. `vehicle_health_below` **necesita el vehículo destructible.** Con `VehicleInvincible` en `true` (el default) no le entra daño.
9. `set_vehicle_mortality` **y** `set_driver_mortality` **:** sólo `"mortal"`, exacto y en minúscula, los vuelve mortales. Sin `value`, o con `"Mortal"`, quedan **invencibles**. Lo mismo con `set_vehicle_parts`: sólo `"unlocked"` exacto lo desbloquea; con `"Unlocked"` sigue inviolable.
10. **Cada evento dispara una vez por corrida.** `player_enter_vehicle` sólo ve al **primer** pasajero; `player_in_radius`, al primer jugador que se acerca. En cada spawn todo se re-arma.
11. **Un evento sin** `wp` **y sin** `trigger` **dispara al nacer el vehículo** (es un `wp_reached` en el wp 0).
12. **Con el cerebro 1.2,** `wp_reached` **puede disparar unos puntos más tarde** en rutas con glitches de grabación (línea `[Route] saneo de retrocesos: N` con N mayor que 0 en el RPT).
13. **Luces:** un `lights_on` / `lights_off` manda sobre `LightsMode` hasta el próximo spawn, con los dos cerebros.
14. `dismount_guard` **no los vuelve a subir.** Si soltás el vehículo se va sin ellos, y cuando el vehículo se despawnea se borran. Para bajar, esperar y volver a subir, usá `guard_stop`. El vehículo tiene que estar

detenido para que bajen.

15. **Los bots de `spawn_guard` no son tripulación**: ni `dismount_guard` ni `guard_stop` los mandan.
16. **Los errores de nombre se ven en momentos distintos:** un trigger mal escrito avisa **al cargar** la ruta; un verbo mal escrito avisa recién **cuando el evento dispara**. Los nombres distinguen mayúsculas.
17. **Mientras suben el chofer o la tripulación, los eventos no se revisan.** Con el vehículo congelado, frenado o pausado desde el panel **sí** se revisan: un `timer` puede disparar mientras lo tenés en pausa.
18. **Las líneas `[EVENT ...]` del log no dicen de qué ruta son.** Con varios vehículos, guiate por el número de evento y el wp.
19. **Los verbos reservados** (`lock_seat` , `unlock_seat` , `eject_passenger`) y cualquier nombre que no esté en esta referencia **no hacen nada**.

---

11. Misiones — BZ_AutoDrive + Quests *(la integración estrella)*

Esta es la pieza más potente del framework: que un vehículo manejado por NPC sea **parte de una misión** — refuerzos que llegan, un convoy que escapa, una emboscada en la ruta. Para eso, BZ_AutoDrive se acopla con **DayZ-Expansion-Quests**.

11.1 ¿Por qué dos sistemas? (la división de labor)

La clave es que **cada sistema hace lo que sabe hacer**:

- **Quest** = los **bots** + la **lógica de misión**. Spawnea bots vivos y armados, y es el dueño de todo lo "misional": matarlos da reward, progresión, condiciones de victoria/derrota.
- **BZ_AutoDrive** = el **vehículo**. Lo spawnea, lo **maneja** por tu ruta grabada, y coordina que los bots suban y bajen.
- **eAI** (la IA base de Expansion) = lo básico del bot: caminar, subir y bajar de un auto.

> *¿Por qué no hace todo el framework?* Un bot que spawnea el framework por su cuenta **no tiene lógica de misión** — matarlo no da nada (ni reward ni progresión). Y sin un Quest, los bots ni "viven" bien (aparecen como maniquíes). **Quest es el único dueño de bots armados con sentido.** Por eso se componen: Quest pone la gente y el sentido; vos ponés el vehículo y el manejo — la pieza que a Quest le falta.

11.2 Cómo funciona una misión, de punta a punta

1. El jugador **acepta el quest** (define dónde y cuántos bots).
2. Los bots **aparecen** cuando el jugador se acerca a la zona (son "lazy": se materializan por proximidad).
3. El framework **detecta** esos bots y **spawnea el vehículo** en el arranque de tu ruta.
4. Según el guión: los bots **suben** (caminando o ya sentados) y el NPC **maneja** la ruta.
5. Al llegar (o al dispararse un trigger), los bots **se bajan** — a desplegarse, a campear, a lo que pida la misión.
6. Cuando la misión termina, el vehículo se **limpia** (no queda dando vueltas).

11.3 Ejemplo 1 — "El convoy que huye" *(validado in-game)*

La idea: hay un campamento con 5 bots. Si el jugador empieza a matarlos, los sobrevivientes **se escapan en un vehículo**.

Cómo se arma:

- En el **quest**: un campamento con 5 bots armados en una terminal.
- En el **framework**: grabás la ruta de la terminal a un patio de escape, y la marcás como "convoy que huye al matar 1".

Qué pasa en vivo:

1. El jugador llega → los 5 bots aparecen y el vehículo (un Cobra) aparece solo en la terminal.
2. El jugador mata 1 → **se dispara la huida**: los 4 sobrevivientes **dejan de disparar** (se "pacifican"), **caminan** hasta el vehículo y **suben** con la puerta animada.
3. El NPC **maneja** hasta el patio.
4. Al llegar, **frena** y los 4 **se bajan** (puerta animada).

> El gatillo es el **kill-count del quest** (matar 1). El framework no inventa la lógica — *escucha* al quest y maneja el vehículo.

11.4 Ejemplo 2 — "Interceptá el convoy" *(emboscada)*

La idea: un convoy enemigo cruza la zona; el jugador lo embosca.

Cómo se arma:

- En el **quest**: 3 bots en el arranque del circuito.
- En el **framework**: una ruta de circuito, marcada como "emboscada al recibir daño", con el vehículo **destructible**.

Qué pasa en vivo:

1. Los 3 bots arrancan **ya a bordo, armados**, y el NPC los maneja por el circuito.
2. El jugador les pega un tiro → **cualquier daño** (al vehículo o a un bot) dispara la emboscada.
3. El vehículo **frena**, espera a detenerse del todo, y los bots (+ el conductor) **se bajan y campean** hostiles.

> *Insight de diseño:* esta emboscada es, en el fondo, un **marcador NUMPAD 4 generalizado** — un trigger ("al recibir daño") que dispara una secuencia ("frenar + bajar + combatir"). Hoy viene pre-armado como un modo; la dirección es que lo definas vos en los eventos (§10), sin tocar código.

11.5 El lifecycle del vehículo (no dejar autos fantasma)

En una misión, el vehículo **no debe quedar dando vueltas** al terminar. Cuando el quest marca los objetivos como completos, el framework **limpia** el vehículo (lo despawnea y corta el manejo). Otras políticas posibles: dejarlo como **botín**, dejarlo en sitio para un objetivo encadenado, etc.

> *Ejemplo:* en el convoy que huye, cuando el jugador finalmente liquida a los 4 en el patio, el quest marca "completado" → el Cobra se despawnea solo. No queda un auto fantasma en el mapa.

11.6 Lo que necesitás para armarla

- **Dependencia:** Quests, que ya viene en el Bundle de Expansion y el mod requiere de todas formas (§4).
- **Tu parte (framework):** grabás la ruta del vehículo + la configurás con el modo de convoy y, si querés, los bots que viajan desde el arranque.
- **La parte del quest:** definís los bots y la lógica (reward, condiciones) en el editor de Expansion-Quests.
- **El enganche:** el framework *escucha* cuándo arranca el quest y dónde están sus bots, y desde ahí coordina el vehículo.

> ⚠ **El vehículo de la misión tiene que ser el que grabaste.** El `VehicleClass` del JSON no es una etiqueta: es **la clase con la que hiciste la toma**. Si armás la misión con un Cobra, la ruta tiene que estar grabada con un Cobra — no con "algo parecido". Cada toma alimenta el *envelope* de **esa** clase (§6C), y ahí es donde el framework aprende cuánto frena y cuánto gira ese vehículo.
>
> Si querés la misma escena con otro vehículo, **grabá la ruta con ese vehículo** y apuntá la misión a la ruta nueva. Es más rápido que pelearse con una toma prestada, y el resultado es fiel a la física del auto que el jugador va a ver.

11.7 — Cómo se escribe (ejemplos de config)

La parte del framework se configura en el **JSON de la ruta** (`BZBusRoute*.json`). Acá van los ejemplos reales de las dos escenas. *(Los bots y el reward los definís aparte, en el editor de Expansion-Quests.)*

Escena 1 — convoy que huye. El vehículo espera en la terminal; cuando el quest detecta que mataste 1 bot, los demás suben y huyen:

```
{
  "VehicleClass": "Star_APC_Cobra_white",
  "SpawnHoldSeconds": 600,
  "ConvoyMode": "flee_on_kill",
  "Crew": [],
  "Events": [],
  "Waypoints": [ /* tu ruta: de la terminal al patio */ ]
}
```

- `ConvoyMode: "flee_on_kill"` → activa la mecánica (pacificar → subir → manejar → bajar).
- `Crew: []` → vacío a propósito: **los bots los pone el quest**, no la ruta.
- `SpawnHoldSeconds: 600` → el vehículo espera quieto hasta que se dispare la huida.

Escena 2 — emboscada. Los bots arrancan a bordo armados; cualquier daño dispara el despliegue:

```
{
  "VehicleClass": "x5mcompetition_orange",
  "ConvoyMode": "ambush_on_damage",
  "VehicleInvincible": false,
  "Crew": [],
  "Waypoints": [ /* el circuito */ ]
}
```

- `ConvoyMode: "ambush_on_damage"` → bots instant a bordo + freeze + dismount al recibir daño.
- `VehicleInvincible: false` → **clave**: si el vehículo es irrompible, nunca recibe daño y la emboscada **no dispara**.

Bots que viajan desde el arranque (`Crew[]`). Si querés que el vehículo nazca con bots adentro (sin depender del quest), los listás:

```
"Crew": [
  { "cls": "eAI_SurvivorM_Boris", "seat": 1, "faction": "Raiders",
    "loadout": "BanditLoadout", "offsetRight": 2.0, "offsetForward": 0 }
]
```

- `seat` : 1+ (el 0 es el chofer). `faction` / `loadout` : cómo viene armado. `offsetRight/Forward` : dónde spawnea **afuera** antes de subir (que no quede dentro del cuerpo del auto).

Una secuencia de eventos en un punto (`Events[]`). Para colgar acciones a un nodo NUMPAD 4 — ej. una parada con sonido:

```
"Events": [
  {
    "wp": 88,
    "trigger": { "type": "wp_reached", "wp": 88 },
    "actions": [
      { "verb": "freeze_vehicle" },
      { "verb": "play_sound", "value": "MiVoz_SoundSet", "delay": 0.5 },
      { "verb": "unfreeze_vehicle", "delay": 3.0 }
    ]
  }
]
```

- `trigger` = cuándo (acá: al llegar al wp 88). `actions` = qué, en orden, con `delay` (segundos) para coreografiar.
- Triggers disponibles: `wp_reached` , `player_in_radius` (con `radius`), `player_near_waypoint` (con `wp` y `radius`), `player_enter_vehicle` , `vehicle_health_below` (con `threshold` 0..1), `timer` (con `seconds`). Referencia completa en §10.3.

11.8 — El hook en código (cómo el framework "escucha" el quest)

La integración es server-side, en dos piezas. **(1)** Un `modded class MissionServer` que avisa al framework cuando arranca un quest:

```
modded class MissionServer {
    override void Expansion_OnQuestStart(ExpansionQuest quest) {
        super.Expansion_OnQuestStart(quest);
        BZBusService.GetInstance().OnQuestStart(quest);
    }
}
```

(2) El framework guarda el ID del quest y **pollea**, porque los bots son *lazy* (aparecen recién cuando el jugador se acerca al campamento):

```
void OnQuestStart(ExpansionQuest quest) {
    if (!quest) return;
    ExpansionQuestConfig qc = quest.GetQuestConfig();
    if (!qc) return;
    m_QuestCheckID = qc.GetID();                        // guardamos el ID
    GetGame().GetCallQueue(CALL_CATEGORY_GAMEPLAY).CallLater(this.CheckQuestBots, 2000, true); // poll c/2s
}

void CheckQuestBots() {
    array<eAIQuestPatrol> patrols = new array<eAIQuestPatrol>();
    bool exists = ExpansionQuestModule.GetModuleInstance().QuestPatrolExists(m_QuestCheckID, patrols);
    int totalBots = 0;
    if (exists)
        for (int i = 0; i < patrols.Count(); i++)
            if (patrols[i] && patrols[i].m_Group)
                totalBots += patrols[i].m_Group.Count();
    // totalBots > 0  -> el convoy se materializó (guardamos el conteo inicial)
    // el conteo BAJA -> mataron uno -> disparamos el trigger (huida / emboscada)
}
```

> **La puerta de entrada:** `QuestPatrolExists(questID, patrols)` te da las patrullas vivas del quest, y `patrol.m_Group` son los bots. Desde ahí el framework los alcanza — son las referencias que necesitás para subirlos al vehículo.

11.9 — Cómo embarca los bots (la mecánica + el gotcha)

Al dispararse el trigger, el framework agarra los bots vivos del quest y los sube. Lo no-obvio —y que costó horas— es que hay que **pacificarlos primero**: un bot *en combate* no camina a un waypoint (el FSM de eAI exige "sin amenaza"):

```
void BoardQuestBots() {
    Transport transport = Transport.Cast(m_Bus);
    array<eAIQuestPatrol> patrols = new array<eAIQuestPatrol>();
    if (!ExpansionQuestModule.GetModuleInstance().QuestPatrolExists(m_QuestCheckID, patrols)) return;
    int seat = 1;
    for (int i = 0; i < patrols.Count(); i++) {
        eAIQuestPatrol p = patrols[i];
        if (!p || !p.m_Group) continue;
        for (int m = 0; m < p.m_Group.Count(); m++) {
            if (seat > 5) break;                          // Cobra: 5 plazas de pasajero
            eAIBase b = eAIBase.Cast(p.m_Group.GetMember(m));
            if (!b) continue;

            // 1) PACIFICAR (sino no caminan):
            b.eAI_SetPassive(true);
            b.eAI_SetThreatDistanceLimit(0.0);            // que no vuelva a fijar al jugador
            for (int tt = 0; tt < 16; tt++) {             // drenar los targets ya adquiridos
                eAITarget tg = b.GetTarget(0);
                if (!tg) break;
                b.eAI_RemoveTarget(tg);
            }

            // 2) Caminar a la puerta de SU asiento y subir (animado):
            vector door, ddir;
            transport.CrewEntryWS(seat, door, ddir);
            // ... SetMovementSpeedLimits + AddWaypoint(door); el Tick lo sube por la puerta ...
            seat++;
        }
    }
}
```

> **El gotcha que costó:** sin pacificar, el bot en combate no navega → terminaba subiendo por *teleport* (feo, "maniquí"). Con el combo `eAI_SetPassive(true)` + `eAI_SetThreatDistanceLimit(0)` + **drenar los targets**, el bot se "olvida" del jugador, **camina y abre la puerta** (animado, lindo). *Trade-off:* quedan pasivos durante la huida (coherente con que escapan).

> *Aún más detalle (todas las funciones del lifecycle, la lista completa de verbos, los gotchas de la API eAI) en el **AI knowledge pack**.*

11.10 — Ejemplo 3 — "Tomá el bus" (Travel) *(validado in-game)*

El tipo de objetivo más simple de integrar, y el **primero validado más allá de los convoyes**: un objetivo **Travel** (`ObjectiveType` 3) se cumple cuando el jugador **llega a una posición**. Si esa posición es el **final de una ruta** del framework, entonces **viajar en el vehículo cumple el objetivo**. El bus *es* el viaje.

La pieza nueva: el campo `QuestTravelID`. Una ruta declara qué quest la dispara. Cuando el jugador **reclama** ese quest, el framework auto-spawnea la ruta, espera `SpawnHoldSeconds` (para que llegues y subas) y maneja al destino. Cero intervención del admin — "reclamás y el bus aparece y anda solo".

Lo que armás (todo config; del lado del quest no se toca código):

1. El objetivo Travel (`Objectives\Travel\Objective_T_8.json`) — el destino = el final de la ruta:

```
{
  "ObjectiveType": 3,
  "ObjectiveText": "Viaja en el bus hasta la terminal.",
  "Position": [3322.8, 200.0, 13029.7],   // = ultimo waypoint de la ruta
  "MaxDistance": 30.0,
  "TriggerOnEnter": 1
}
```

2. El quest (`Quests\Quest_42.json`) que lo envuelve, con su giver (una pizarra) y su reward.

3. En la ruta (`BZBusRoute_..._hdr.json`), dos campos:

```
{
  "QuestTravelID": 42,        // el quest que auto-spawnea ESTA ruta
  "SpawnHoldSeconds": 20.0    // el vehiculo espera 20s antes de arrancar (para que subas)
}
```

El flujo in-game:

1. Reclamás el quest en la pizarra.
2. El vehículo aparece en el inicio de la ruta y **espera 20s**.
3. Subís de acompañante.
4. Boris maneja al destino → entrás al radio → **objetivo cumplido**, cobrás el reward.

El hook (en `OnQuestStart`): al arrancar un quest, el framework escanea las rutas; si alguna declara `QuestTravelID` == ese quest, la carga (`LoadConfigFromPath`) y la spawnea (`RespawnBus`); el `SpawnHoldSeconds` de la ruta hace la espera. Es el **mismo molde** que se extiende después a Escort/VIP (el vehículo lleva al VIP) y Delivery (el vehículo lleva la carga).

11.11 — Ejemplo 4 — Escolta al VIP (AIVIP) *(validado in-game)*

El objetivo que **cierra** la integración: un **AIVIP** (`ObjectiveType` 9, "escoltá al VIP a un lugar"). Expansion spawnea un **VIP** (un NPC eAI, ej. `eAI_SurvivorF_Gabi`); el framework lo **sube a un vehículo** y Boris lo maneja al destino, mientras vos lo **escoltás** (lo protegés en el camino). El VIP llega → objetivo cumplido.

Dos NPCs distintos (no confundir):

- **Boris** = el chofer (siempre el del framework).
- **El VIP** = el NPC del quest, el pasajero. Vos no manejás: protegés que llegue vivo.

La pieza nueva: el campo `QuestEscortID`. Igual que `QuestTravelID` (auto-spawnea la ruta al reclamar el quest), pero además **sube al VIP**:

```
{
  "QuestEscortID": 10,         // el quest AIVIP que dispara esta ruta
  "SpawnHoldSeconds": 35.0,    // generoso: el VIP debe spawnear, caminar y subir ANTES de arrancar
  "VehicleClass": "CivilianSedan"
}
```

El insight clave — cómo se agarra al VIP. Acá hubo que leer el código de Expansion: el VIP **NO** está en `QuestPatrolExists()` (esa lista es solo de AICamp/AIPatrol). Está **flageado con `Expansion_IsQuestVIP()`** y sale del enum global **`eAIBase.eAI_GetAll()`**. El framework escanea esa lista, filtra por el flag + proximidad al vehículo, y lo embarca en su propio grupo (sin tocar el grupo del jugador). *(Técnica útil: las `.c` de Expansion se leen como texto adentro del PBO — no hace falta extraer.)*

El flujo in-game:

1. Reclamás `Quest_10` en el NPC que da el quest.
2. Aparece el **CivilianSedan + Boris** en el inicio de la ruta y espera 35s.
3. **El VIP** spawnea cerca → el framework lo encuentra (`eAI_GetAll` + `Expansion_IsQuestVIP`) → lo sube caminando.
4. Boris lo lleva al destino (fin de ruta) → baja → **objetivo cumplido** (con `Autocomplete: 1`, se finaliza solo).

Gotchas que aprendimos (cuesta caro saltarlos):

- **La grab NO es `QuestPatrolExists`** — para AIVIP es `eAI_GetAll()` + `Expansion_IsQuestVIP()`.
- **El vehículo importa:** tiene que tener asientos de crew válidos + **puertas vanilla**. Un carpack (probamos el X5M) se **hundió** (bbox raro) y su puerta **crasheó el FSM de eAI** (`OpenVehicleDoor`) → spam de errores → server sobrecargado. El **CivilianSedan** (vanilla, 4 puertas) anduvo perfecto.
- **Asiento del VIP:** mandalo a un **asiento trasero**, no al acompañante delantero — sino el jugador y el VIP se pelean el mismo asiento y se fusionan (te traba adentro del auto).
- **Hold generoso:** el VIP necesita tiempo para spawnear, caminar y subir antes de que el auto arranque (`SpawnHoldSeconds` ~30-40s).
- **Si la quest queda colgada** (cambiar la config de un objetivo con la quest activa la rompe): poné `Autocomplete: 1` + reinicié — la quest cumplida se auto-finaliza.

Lo que esto cierra: con AIVIP validado, los objective types donde el vehículo tiene un rol — **AICamp/AIPatrol, Travel, AIEscort/VIP** — están todos andando. Los demás (Collection, Crafting, etc.) **coexisten** sin conflicto. Eso es la integración Quest completa.

12. Controles (las teclas)

El framework usa **solo 3 teclas** — el ciclo de grabación + abrir la UI. Todo lo demás (largar / parar / pausar un vehículo, spawnear vacíos, armar los loggers) se hace **desde el Reproductor** (§13), no con teclas. Las 3 viven en el **menú del juego**: `Opciones → Controles → sección "BZ AutoDrive"`, **rebindeables** como cualquier control de DayZ, y vienen **asignadas por default**:

Acción (nombre en el menú)	Default	Para qué
Open Control Panel	`INICIO` (Home)	**abrir / cerrar** el Reproductor (la puerta a toda la UI). También lo cierra `ESC` (sin abrir la pausa del juego). Si le asignás otra tecla en Controles, esa lo abre y lo cierra
Record (start/stop)	`NUMPAD 5`	empezar / terminar de grabar una ruta
Mark Event / Stop	`NUMPAD 4`	marcar una **parada / nodo de evento** (§10) mientras grabás

> *El cambio de sentido ya no es tecla:* el **intercambio / K-turn** (§9.4) se **auto-detecta** del cambio de gear forward↔reverse — frenás del todo, cambiás de marcha y seguís; el conversor corta el tramo ahí y Boris lo trata como un nuevo arranque.

> *Flujo típico:* abrís el panel con `INICIO`; para grabar, `NUMPAD 5` (con `NUMPAD 4` en las paradas/eventos; en los cambios de sentido solo frenás, cambiás de marcha y seguís — se auto-detectan), `NUMPAD 5` de nuevo para cortar; después elegís la ruta en el Reproductor y mirás cómo le va. Tus asignaciones se guardan en tu perfil (`Documents\DayZ\<perfil>.dayz_preset_User.xml`).

> ⚠ **Los controles los VE todo el mundo, pero solo funcionan para el admin.** DayZ le muestra la categoría "BZ AutoDrive" en Controles a **todos** los jugadores (registra los inputs de forma global — no se puede ocultar por admin). Pero las teclas **no hacen nada** si no sos admin: el gate es **server-side**, se valida en cada acción contra tu Steam ID (§13.1). Por eso el menú se dejó **mínimo** (3 teclas): un jugador común ve una categoría corta e inerte, no un tablero de debug.

> *Nota:* el `ai_run` (la caja negra de Boris) ya **no tiene tecla** — se arma desde el check del Reproductor (§13, §15.1). Las herramientas viejas de tuning/debug (Parking/Reverse/Approach en vivo, SysID, gear markers, spawn slots) se sacaron del menú; las maniobras salen del **intercambio auto-detectado** (cambio de gear) al grabar.

---

13. La interfaz (UI)

Un solo panel para el admin — el **Reproductor** — que lo abrís con una tecla (**INICIO/HOME** por default, configurable). Reúne todo en una pantalla: la lista de rutas, lo que se está manejando en vivo, y el spawn de vehículos vacíos. Se divide en tres zonas:

① **ROUTES (izquierda) — tu lista de rutas.** Todas las rutas guardadas. Hacés clic en una para **elegirla** y la largás con **LOAD & SPAWN**: el NPC la maneja al instante, **sin reiniciar el server**. Al seleccionarla, abajo te muestra su **ficha** (vehículo, cantidad de waypoints, distancia, velocidad máx) y un **scrubber** para recorrer la ruta wp por wp. Debajo de la lista, el botón **LOAD & SPAWN** (largar la ruta elegida) y los **checks de logger** (ver abajo).

> *Ejemplo:* tenés "Bus pueblo", "Taxi centro", "AB - Jeep" en la lista; elegís una, apretás LOAD & SPAWN y arranca.

② **ACTIVE RUNNERS (derecha, arriba) — tu tablero en vivo.** Cada vehículo que se está manejando **ahora** — tus rutas, las de misiones, las del arranque del server — con su **nombre + waypoint** y su **estado** (`DRIVING` / `PAUSED` / `STOPPED`). Por cada uno, cuatro botones: **RST** (reiniciar desde el wp 0 de su ruta, sin recargar), **TP** (teletransportarte al lado para interceptarlo), **II** (pausar / reanudar), **[]** (pararlo y sacarlo). Si hay muchos, la lista **scrollea** con la rueda. Arriba a la derecha, **STOP ALL** para bajar todos de una.

> *Ejemplo:* ves "Bus pueblo — wp 88/166 — DRIVING". Lo pausás con **II** o te TP al lado para subirte.

③ **ACTIVE SPAWN VEHICLE (derecha, abajo) — los vehículos vacíos que sembraste.** Cada vehículo que spawneaste vacío (con START/HERE/END, ver abajo) aparece acá como una fila —**ruta · wp · posición**— con dos botones: **TP** (ir a su lado) y **DELETE** (borrarlo). Podés dejar **varios** vacíos sembrados por el mapa y manejarlos desde acá. Scrollea si hay más de los que entran.

La barra de abajo — spawn de vehículos vacíos + scrubber. Muestra la ficha de la ruta seleccionada + el scrubber, y a la derecha **SPAWN EMPTY VEHICLE AT: START · HERE · END**: spawnea el vehículo de la ruta **vacío y manejable** en el inicio, en el wp del scrubber (**HERE**) o en el final. Sirve para **continuar una grabación** (spawnás el auto donde quedaste y seguís manejando) o para sembrar autos. **Si el lugar está ocupado** por otro vehículo, no spawnea y lo avisa en pantalla (`Spawn point occupied by <clase> (N m away)`). **Ya no te teletransporta** al vehículo: aparece en la lista **ACTIVE SPAWN VEHICLE** (③) y desde ahí lo TP o lo eliminás.

Los checks de logger (opt-in, antes de dar play). Debajo de la lista de rutas hay dos casillas — `[ ] boris_native` y `[ ] ai_run` — que **tildás ANTES de largar la ruta** (LOAD & SPAWN). Si están tildadas, esa corrida graba su trayectoria **en sincro con el play**:

- `ai_run` — la **caja negra** de Boris (telemetría de la corrida; ver §15.1). Se arma **desde este check** (ya no con tecla) para esa corrida puntual.
- `boris_native` — la **trayectoria server-side** de Boris, en la misma forma que una grabación humana: sirve para compararla con tu toma y ver dónde se desvió (detalle en §15.2).

> Vienen **destildadas** (no graban nada). Es opt-in por corrida: tildás, das play, y solo esa corrida escribe el log. Sin tildar, uso normal, cero archivos.

> *Nota:* las teclas se rebindean desde el menú de Controles del juego (§12). Los checks arman los loggers desde la propia UI, sin depender de las teclas.

13.1 — ¿Quién es admin? — configurar `AdminSteamIDs`

Recién dijimos que los controles y las UIs **"solo tienen efecto para vos (admin)"**. ¿Pero cómo decide el framework quién es admin? Por tu **Steam ID**.

Cómo te identifica. El server compara tu Steam ID (el Steam64, también llamado *PlainId*) contra una lista llamada `AdminSteamIDs` en su archivo de settings. Si tu ID está en esa lista, sos admin.

El archivo. `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json` . El server lo **crea vacío** la primera vez que arranca con el mod: editalo. El campo que te importa:

- `AdminSteamIDs` — la lista de Steam IDs autorizados. **Cada ID va entre comillas**, como en el ejemplo de abajo: sin comillas el server no lo toma y seguís sin ser admin.

*(La tecla que abre el panel no se configura acá: se cambia en **Opciones** → **Controles** → **BZ AutoDrive**, §12.)*

⚠️ **LO MÁS IMPORTANTE (esto es el punto de seguridad):** si `AdminSteamIDs` está **VACÍA, NADIE es admin** — ni siquiera vos. El panel no abre y todas las acciones se rechazan del lado del server (fail-closed). **Poné tu(s) Steam ID(s) en la lista: es el primer paso, obligatorio, para usar el framework.** Si apretás la tecla del panel y "no pasa nada", mirá el log del server: vas a ver una línea `[Panel] acceso denegado a <tu SteamID>` que te dice exactamente qué editar.

Ejemplo del JSON:

```
{
  "AdminSteamIDs": ["76561198000000000"]
}
```

Podés poner varios, cada uno entre comillas y separados por coma: `["76561198000000000", "76561198000000001"]`. El server lee el archivo al arrancar: **reinicialo después de editarlo**. Para comprobar que lo tomó, mirá el RPT del arranque: tiene que decir `[Settings] Cargado. Admins: 1` (o la cantidad que pusiste). Si dice `Admins: 0` y aparece `AdminSteamIDs is empty`, el ID no está bien escrito (revisá las comillas).

Cómo encontrar tu Steam ID (el Steam64 / *PlainId*): en tu perfil de Steam, en **steamid.io**, o en el **log del server** cuando te conectás.

Por qué es seguro. El chequeo se hace **server-side en cada acción** (abrir panel, spawnear, cargar ruta, parar…), comparando tu Steam ID autenticado por Steam. No se puede falsear desde un cliente modificado: el gate del lado del cliente es solo comodidad visual.

> *Nota honesta:* un jugador común igual puede **VER** la categoría "BZ AutoDrive" en su menú de Controles (DayZ le muestra a todos los inputs registrados), pero las teclas **no hacen nada** para él si no es admin.

13.2 — Por qué la ruta tarda unos segundos en arrancar

Al lanzar una ruta, el vehículo aparece y queda unos segundos en **cargando** antes de empezar a manejar. **No es lentitud: es a propósito.**

La ruta y su cinta se cargan **repartidas en muchos cuadros**, para que el servidor no se frene. Cargarlas de golpe congela el servidor **casi dos segundos** (medido: 1,84 s con una ruta de 1.193 waypoints y 19.075 cuadros de cinta), y eso **no lo sufre sólo el vehículo que está arrancando**: se frena todo — el vehículo que ya venía manejando, los bots, y cualquiera que esté jugando.

En un **convoy** el problema se multiplica: el segundo vehículo se lanza **mientras el primero ya circula**, así que cada carga le pega en marcha al que salió antes. Con tres vehículos serían tres congelamientos de casi dos segundos, uno arriba del otro. Y en un servidor de verdad el convoy no está solo: está el **bus de línea**

haciendo su servicio permanente, las otras rutas que relanzan solas cada tantos minutos, y lo que cada uno tenga corriendo.

Hay UNA sola cola de carga para todo el servidor, con un presupuesto fijo de milisegundos por cuadro. Eso quiere decir que **lanzar cinco rutas de golpe no empeora el servidor**: sólo tarda más en tenerlas todas listas. Las rutas se cargan **por orden de llegada**. La tarjeta aparece en **Active Runners** apenas se aprieta Load, y su línea de estado dice qué está haciendo:

Estado	Qué significa
`WAITING wp0 occupied 12s`	el punto de salida está ocupado por otro vehículo: espera a que se libere (re-intenta cada 5 s) y recién ahí crea el suyo, en su lugar exacto. No lo corre al costado: un vehículo que no nace sobre su cinta no la puede seguir
`QUEUED 2`	esperando turno: hay 2 cargas delante
`LOADING 43%`	cargando la ruta y su cinta
`BOARDING CREW 2/3`	ya cargó; espera a que la tripulación se siente
`PREPARING`	subiendo el chofer, o el último segundo antes de arrancar
`DRIVING` / `STOPPED` / `PAUSED`	como siempre

Si se aprieta Load otra vez sobre una ruta que ya está en marcha, **no se lanza de nuevo** y aparece un aviso que lo dice (`PATRULLA ya esta en marcha (LOADING 43%): no se lanza de nuevo`).

El porcentaje sale del header de la ruta: el conversor escribe en el `_hdr.json` cuántas filas tienen el `_wp.csv` y la cinta (`"WpRows"` , `"CintaRows"`). Una ruta convertida antes de esto carga igual, pero muestra `LOADING (8400 rows)` en vez del %. Para completarle el número, reconvertí la ruta con el wizard.

El número se queda quieto un momento al terminar la ruta y sigue con la cinta: es un % de lo leído, no una barra inventada, y no marca 100 hasta que la carga terminó de verdad.

Desde la 1.3, además, todo lo pesado hace fila. No sólo la carga: crear el vehículo, el chofer y cada bot de la tripulación, el embarque, el retiro y los pasos de armado del cerebro pasan por **una sola puerta, una operación por cuadro**, en orden de llegada. Dos cosas pesadas ya no caen en el mismo cuadro, ni al lanzar quince rutas juntas: cada una tarda unos cuadros más, y el server no lo siente. En el RPT se ve como `[EMBUDO] <ruta> <operación>: X ms · espero N cuadro(s)` .

Y una toma ya leída no se vuelve a leer. Al relanzar una ruta (un servicio que da vueltas, un mod que lanza el mismo recorrido cada tanto), sus waypoints salen de la memoria del server, con una copia por vehículo: `[CACHE] <ruta>: N waypoints copiados de la cache, sin abrir el CSV` . Se vuelve a leer sólo si el archivo cambió (lo compara por tamaño y huella; el header, entero). **La primera lectura** va en tajadas de pocos milisegundos: `[LOADTIME] ... X ms de trabajo en N tajada(s)` .

La perilla (opcional) va en `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json` :

```
{
  "AdminSteamIDs": ["76561198000000000"],
  "LoadBudgetMsPerFrame": 5.0
}
```

- `LoadBudgetMsPerFrame` — milisegundos por cuadro que el servidor le dedica a cargar rutas, **entre todas las que haya en cola**. Default **5.0**, que es lo que cuesta un tick normal del piloto: no se nota.
- **Más bajo** (2-3): el servidor ni se entera, las rutas tardan más en estar listas.
- **Más alto** (10-15): arrancan antes, a costa de cuadros más caros. No conviene pasarse: es exactamente la palanca que causaba el problema.
- `0` o negativo se ignora y vale el default (una perilla mal puesta no puede volver a congelar el servidor).

Y por qué espera también a la tripulación. Si la ruta lleva bots de tripulación, el vehículo **no arranca hasta que estén todos sentados**, sean 2, 3 o 10. No es un reloj: es la plaza ocupada. Antes salía a los pocos segundos y los bots lo perseguían a pie hasta la primera parada — y en esa primera etapa no había nadie a bordo para bajarse a defender.

Si alguna plaza no se llena (un bot que murió al aparecer, uno que no encuentra la puerta), **sale igual** cumplido `CrewBoardTimeoutS` y el log dice con cuántas plazas salió:

```
[MUSTER] plazas completas 3/3 en 4.2 s -> arranca
[MUSTER] TOPE de 120 s: arranca con 2/3 plazas ocupadas -- 1 bot(s) no subieron
```

Antes del tope hay una red *(desde la 1.3)*: si a los **15 s** un bot vivo sigue a pie **con todo lo que necesita para subir** (la formación, el chofer sentado, una plaza libre con la puerta despejada, sin amenaza), el framework lo **sienta por teleport** en la primera plaza libre y sigue: `[MUSTER] RED a los 15 s: el bot 0 no subio con las condiciones de eAI cumplidas -> teleport a la plaza 1`. Un vehículo con una puerta que el bot no resuelve (medido en un blindado) deja de costar dos minutos. El tope de 120 s queda para lo que la red no resuelve: amenaza, sin plazas, sin chofer. Cada línea `[MUSTER]` lleva el nombre de la ruta.

- `CrewBoardTimeoutS` — segundos de espera antes de salir igual; va en el mismo `BZAutoDrive_settings.json`. Default **120**. Es una **red de seguridad, no una perilla de tuning**: tiene que estar lejos de lo que tarda un embarque sano. Si ves `[MUSTER] TOPE` en corridas normales, el problema no es el número.

Si preferís que arranque antes, grabá rutas más cortas: el costo es proporcional al tamaño de la ruta, no una espera fija. En el log del servidor cada carga deja su número:

```
[CARGA] PATRULLA LISTA: 1193 wps + 19075 frames de cinta en 264 tajadas / 8200 ms de reloj
[CARGA] PATRULLA reparto: leer ruta 358 ms · armar ruta 330 ms · leer cinta 1105 ms · armar cinta 12 ms · PEOR TAJADA 6.1 ms
```

Cuánto le pesa a tu server *(medido con 20 vehículos a la vez)*. No te damos fps ni milisegundos: dependen de tu procesador y de tus mods, y un número medido en otra máquina no te sirve. Lo que vale en cualquier server es la **proporción** contra algo que ya corre en él:

- **Un vehículo en ruta pesa más o menos como tres bots de eAI sentados, más su propio chofer** (que es un bot más). Si tu server es más lento, los dos se vuelven más lentos en la misma proporción. La comparación es contra un bot **sentado**: uno caminando o peleando pesa más que eso.
- **Crece parejo:** cada vehículo suma lo mismo, tengas 5 o 20. No hay un punto donde el framework se dispare.
- **Lo pesado va de a uno:** cargar una ruta, crear el vehículo, el chofer y la tripulación, embarcar y retirar pasan por una sola puerta, **una operación por cuadro**, haya los vehículos que haya. Lo único que se puede llegar a notar es el motor creando el vehículo cuando sale una vuelta nueva, y nunca caen dos juntos en el mismo cuadro.
- **Probado con hasta 20 vehículos a la vez.** Si tu server ya va justo con sus bots, contá cada vehículo como cuatro bots sentados más.

13.3 — Notificaciones: quién ve qué

El framework avisa cosas por tres canales, y cada uno tiene su público:

Canal	Qué	Quién lo ve
Grabación	"Recording started", "Stop #N marked", el contador `REC`	sólo el que graba (grabar es de admin: NUMPAD 5 y NUMPAD 4 sólo andan para los admins)
Avisos automáticos	las llegadas y los administrativos (abajo)	**sólo los admins** (`AdminSteamIDs` , §13.1), por pantalla o por chat, a tu elección
Mensajes tuyos	el verbo `ui_broadcast` en un evento (§10.3)	a quien elijas: los admins, todos, los que están cerca o los pasajeros

Todo aviso automático queda además en el log del server (`[GLOBAL] …` en el RPT). `log_event` escribe **sólo** ahí: eso no sale nunca a pantalla.

Lo que ves vos: los avisos automáticos. Son de dos tipos, y cada tipo se elige por separado en `BZAutoDrive_settings.json` (el mismo archivo de §13.1):

- `AdminArrivalNotifications` — **las llegadas**: llega a una parada y cuánto espera, la distancia a la parada, fin de línea, ruta completa, "salió de la terminal".
- `AdminSystemNotifications` — **los administrativos**: vehículo detenido, pausa y reanudación, spawn fallido, la seña a Boris.

Cada uno acepta `"toast"` (en pantalla: el default), `"chat"` (en el chat del juego, con el prefijo `BZ AutoDrive:`), `"both"` (las dos cosas) u `"off"` (no sale; queda en el RPT). Si falta la clave, o el valor está mal escrito, vale `"toast"` . El server lee el archivo al arrancar: reinicialo para que tome el cambio.

```
{
  "AdminSteamIDs": ["76561198000000000"],
  "AdminArrivalNotifications": "chat",
  "AdminSystemNotifications": "toast"
}
```

Los jugadores comunes **nunca** ven estos avisos. Las respuestas del panel (por ejemplo, "is already running: not launched again") te llegan siempre en pantalla: son la respuesta a tu clic.

Lo que ven los jugadores: lo armás vos, con eventos. Un jugador común sólo ve lo que le mandes con `ui_broadcast`:

- `value`, **a quién**: `"all"` (todos los conectados), `"near"` (los que están a menos de `radius` metros del vehículo; sin `radius`, 150 m), `"passengers"` (los que van sentados en el vehículo) o `"admins"`. Sin `value`, o con un valor mal escrito, va **sólo a los admins**: un error de tipeo nunca le muestra a los jugadores algo que era para vos.
- `format`, **por dónde**: `"toast"` (en pantalla, el default), `"chat"` o `"both"`.

Por ejemplo, avisarles a todos, en pantalla, que el bus salió de la terminal (un evento en un waypoint justo después de la salida):

```
{ "wp": 3, "actions": [ { "verb": "ui_broadcast", "value": "all", "format": "toast", "msg": "The bus left the terminal" } ] }
```

Y por chat, a los que van arriba: `{ "verb": "ui_broadcast", "value": "passengers", "format": "chat", "msg": "Next stop: Chernogorsk" }`. Cada jugador lo recibe una vez, y el RPT dice cuántos lo recibieron: `[EVENT n @ wp X] ui_broadcast (all → 3 jugadores, toast): …`.

13.4 — Los vehículos se ven suaves: el filtro de cliente *(desde la 1.3)*

Un vehículo que maneja el framework lo mueve el server; cada jugador lo recibe por red, y la orientación llega **en escalones** aunque el vehículo gire parejo. Se veía como un temblor constante, sobre todo parado arriba de un bus o mirándolo de cerca.

Desde la 1.3 el cliente suaviza lo que dibuja, y nada más: ni la posición, ni la velocidad, ni la física, ni el manejo. El server marca qué vehículos maneja; el filtro corre sólo sobre esos, sólo en el cliente, y un vehículo con un **jugador al volante** no se filtra. Viene **encendido** y no hay que configurar nada.

Para apagarlo (para todos los jugadores, también los que entran después), en `<server>\profiles\BZ_AutoDrive\BZAutoDrive_settings.json`:

```
{
  "DisableClientVehicleSmoothing": true
}
```

Si se apaga, los vehículos se ven como antes de la 1.3.

14. Audio (.ogg) — ponerle voz a tus eventos

Con el verbo `play_sound` (en un evento NUMPAD 4) reproducís un audio **3D pegado al vehículo**. El audio lo ponés vos en tu addon:

1. Metés tu `.ogg` en tu addon.
2. Lo declarás en tu `config.cpp` (un `CfgSoundShaders` + un `CfgSoundSets` — copiás el patrón de ejemplo).

3. En el evento: `play_sound` con el nombre del SoundSet.

> *Ejemplo:* tu taxi, al levantar pasajero, dice "buen día" — grabás `buen_dia.ogg`, lo declarás, y lo disparás en el evento de la parada. *Tip:* para probar sin crear un OGG, usá un sonido que ya exista (ej. del mod de radio). *Ojo:* a veces un sonido lanzado desde el server no llega al cliente — si no suena, hay que reenviarlo por red (la infra ya está).

---

15. Si algo no anda

Algunos clásicos (el detalle fino + más casos están en el **AI knowledge pack**, ver abajo):

- **Instalación nueva: el panel no abre y NUMPAD 5 no graba** → todavía no sos admin. El RPT del arranque lo dice: `AdminSteamIDs is empty: nobody is admin…` (o `Admins: 0`). Agregá tu SteamID64 **entre comillas** en `BZAutoDrive_settings.json` y reiniciá (§13.1).
- **El RPT dice `No routes yet` al arrancar** → todavía no hay ninguna ruta: grabá una (NUMPAD 5) y convertila con el Wizard (§5). Si dice `No default route (BZBusRoute.json): nothing starts at boot; N route(s) available in the panel`, está todo bien: tus rutas están en el panel y ninguna arranca sola al iniciar el server.
- **En la 1.30 la lista de rutas sale vacía y el RPT dice `DayZ 1.30 cannot list files under an absolute -profiles path`** → tu server arranca con un `-profiles=` con ruta absoluta (p. ej. `-profiles=C:\DayZServer\profiles`), y en la 1.30 el juego no deja listar archivos ahí. Usá una ruta relativa a la carpeta del server: `-profiles=profiles`.
- **El vehículo aparece pero no se mueve** → casi siempre las coordenadas de la ruta no coinciden con el mapa (grabaste en otro mapa). Grabá tu propia ruta en el mapa donde corrés.
- **No se grabó mi corrida del NPC** → el check `ai_run` estaba destildado. Tildalo en el Reproductor *antes* de dar play (§13).
- **El NPC no entra a una curva cerrada** → es el tramo más exigente (§17); el control **capea la velocidad** solo, pero si el vehículo es **pesado**, lo mejor es **grabar la ruta con ÉL** (nace a su medida). Si es una maniobra cerrada de verdad (galpón, cambio de sentido), va como **reverse** — ver §9.4–9.5.
- **No suena el audio** → ver §14 (replicación server→cliente).
- **El final dice `AR x2` (la ruta completa) o `AR fisico con accion` (`ROUTE_INCOMPLETE`)** → el AutoRecovery intervino (§9.6): los re-arranques suaves no invalidan la ruta; una maniobra o una re-ubicación, sí (§17, el veredicto). Buscá en el RPT las líneas `[AR]` o abrí `profiles/BZ_AutoDrive/AR_log.csv`: te dicen el waypoint y qué pasó. Si siempre es el mismo punto, andá a verlo en el mapa.
- **La ruta se corta en el mismo lugar cada vez y no hay nada ahí** → mirá la pendiente y la velocidad grabada en ese tramo. Una bajada que se empina de golpe, tomada a alta velocidad, puede hacer que el vehículo toque el terreno con la trompa.
- **El vehículo aparece hundido y no arranca** → al spawnear, el RPT escribe un bloque `[SpawnProbe]` (a los 1,5 s y a los 3,5 s) con la altura del vehículo respecto del piso y cuántas ruedas tocan el suelo. Si dice `RESUMEN: 0/4 ruedas en contacto`, nació apoyado sobre la carrocería y no va a poder moverse. Es un caso raro, visto en un solo vehículo de terceros; si te pasa, reportalo con ese bloque.
- **La ruta vuelve a largarse sola mientras estoy probando otra cosa** → es la **ruta por defecto** del server (o una lanzada por API), que se re-lanza a los `RespawnDelay` segundos (300 por default; `0` vale

60 s, no la apaga — §8). Las que largás desde el Reproductor no se re-lanzan. Para que no moleste mientras probás, pará ese runner con **[]** en el panel.

- **El vehículo no aparece y el RPT dice** `La clase '<X>' EXISTE pero NO ES UN VEHICULO` → hay packs de "autos" que en realidad son ítems de inventario (autitos de juguete, maquetas). Revisá el `VehicleClass` del header.

> **Este framework viene con un AI knowledge pack:** un documento pensado para pasarle a una IA (Claude/GPT/Gemini) que te asista a configurar tu caso, resolver errores y armar tus rutas. Si te trabás, esa es la vía rápida — la IA tiene todo el contexto del framework cargado.

> **Tip de performance — correr el server en otra PC.** Si iterás mucho (grabás, convertís, probás), conviene **correr el server en una 2da PC** mientras tu PC principal corre el **cliente del juego + el wizard y las herramientas**. Así ninguna de las dos se ahoga y las corridas salen más fluidas. Para usarlo: *copiás* el `@BZ_AutoDrive` al 2do server (el mismo que usa tu cliente) y configurás su carpeta de rutas en **[6] Configurar paths**: el wizard le copia las rutas al convertir.

Líneas nuevas del RPT desde la 1.3, y qué significan:

Línea	Qué pasó
`[CACHE] <ruta>: N waypoints copiados de la cache, sin abrir el CSV`	la toma ya estaba leída: el relanzamiento no leyó el archivo
`[CACHE] <ruta>: la toma o su header cambiaron ... se relee`	cambiaste el `_wp.csv` o el `_hdr.json`: se lee de nuevo y se reemplaza
`[CARGA] <ruta>: binario de la cinta descartado (<motivo>): se carga el CSV`	falta el `_cinta.bin`, o no corresponde a su CSV: la ruta anda igual; `[B]` en el wizard lo rehace
`[EMBUDO] <ruta> <operación>: X ms · espero N cuadro(s) · quedan P`	una operación pesada esperó su turno; normal al lanzar varias rutas juntas
`[C12] RED: nadie leyo el FIN en 60 s ... -> retiro`	una ruta lanzada por un mod terminó mal y nadie la reclamó: el framework la retiró
`[C12] RED: FIN visto hace N s y el vehiculo sigue`	un mod leyó el fin de esa ruta y no hizo nada con el vehículo (cada 5 min)
`[CLEANUP] ... paso 5: N ocupante(s) sentado(s) -> el vehiculo NO se borra`	hay un jugador sentado: el retiro espera a que baje

15.1 — AI logger — la caja negra de Boris (ai_run)

Cuando una ruta **no sale como esperabas** y querés saber *por qué*, el framework te deja grabar una **caja negra** de la corrida del NPC. Funciona como el *flight recorder* de un avión: registra, segundo a segundo, lo que Boris hizo mientras manejaba, para que después puedas diagnosticar el problema.

Qué es. Un **log de telemetría** de las corridas de Boris. No cambia cómo maneja: solo **observa y anota** (posición, velocidad, marcha, qué pedales apretó, qué modo usó). Es la materia prima para entender un comportamiento raro.

Es OPT-IN, viene apagado. Se arma **tildando el check `ai_run` en el Reproductor ANTES de dar play** (§13, ya **no con tecla**) → se graba en sincro con esa corrida puntual. **Si no lo tildás, no se escribe nada.** No es parte del uso diario: un admin que solo quiere sus buses o convoyes andando **no lo necesita**.

Cuándo usarlo. SOLO cuando una ruta se porta mal —Boris **se abre en una curva**, **se traba**, **va lento**— y querés entender **por qué**, o cuando estás **afinando el config a mano**. El resto del tiempo, dejalo apagado.

Dónde vive. En el **server que corrió a Boris** (no en tu cliente): `<server>\profiles\BZ_AutoDrive_PathLogger\ai_run_*.csv`. *Ojo:* si probaste la ruta en otro server, el `ai_run` queda en **ESE** server — buscalo en la máquina donde corrió.

Qué registra. Unas **~27 columnas de telemetría**: la **posición** y el rumbo, la **velocidad real y la objetivo**, el **desvío lateral** (ya calculado), los **inputs** que aplicó (`throttle`, `brake`, `steering`), las **rpm**, la marcha, el **modo activo**, los **marcadores de evento**, y el **waypoint que persigue** (`wp_idx`) en ese instante.

> ⚠ **Principio importante — que quede clarísimo:** el `ai_run` es una **MEDICIÓN para comparar CONTRA tu toma** (un *feedback* funcional de cómo le fue a Boris), **NO una grabación nueva**. **No se lo des al wizard como si fuera una toma** — de hecho el wizard los **filtra a propósito**. ¿Por qué? Porque realimentar la salida de Boris **clona sus propios errores**: la ruta se degrada generación tras generación (es el *model collapse* de Machine Learning — la fotocopia de la fotocopia). La fuente de verdad es siempre **tu** grabación humana; el `ai_run` solo sirve para **medir el desvío** contra esa verdad.

Cómo sacarle provecho. Dos caminos:

- **Con IA (recomendado):** le pasás el `ai_run` **junto con la ruta** a una IA con el **AI Knowledge Pack** cargado, y te lo diagnostica — dónde Boris sufrió y qué ajustar (qué campo del config tocar).
- **A mano (si sos técnico):** lo abrís (es un CSV) y mirás vos mismo dónde **se desvió**, **se trabó** o **perdió velocidad** — cruzándolo contra tu toma original.

> *Ejemplo:* el bus se abre siempre en la misma curva. Tildás el check **`ai_run`**, corrés la ruta, agarrás el `ai_run_*.csv` del `profiles` del server, se lo das a la IA con la ruta → te dice "*en el wp 340 el volante llega al tope y no alcanza el giro: tu toma pasa esa curva más rápido de lo que este vehículo puede*". La respuesta suele ser **regrabar ese tramo** (más despacio, o con ese vehículo); **no** realimentás el `ai_run` como toma.

15.2 — boris_native — la trayectoria que manejó Boris

Aparte del `ai_run` (la telemetría de diagnóstico), el framework puede grabar la **trayectoria** de la corrida de Boris: por dónde pasó, a qué velocidad, con qué rumbo — en el **mismo formato que una grabación humana**. Se llama **`boris_native`**.

Qué es. Un CSV con la línea que Boris **realmente manejó** (posición, rumbo, velocidad, `throttle` / `brake` / `steering`, marcha, ángulo de rueda), server-side, a ~40 Hz. Como está en el mismo formato que tu `path_` / `frame_`, se puede **comparar contra tu toma** para ver dónde Boris siguió tu línea y dónde se abrió.

En qué se diferencia del `ai_run` — la distinción clave:

- `ai_run` = **telemetría de diagnóstico** (desvío lateral, saturaciones, corredor, pedales) → responde **"por qué"** Boris hizo algo raro. Se lee con la IA / a mano (§15.1).
- `boris_native` = la **trayectoria cruda** (la línea + velocidad, en formato de grabación) → responde **"por dónde"** fue → se compara contra tu toma.

> Uno **mide**, el otro **muestra el recorrido**. Para **diagnosticar** mirás el `ai_run`; para ver por dónde fue, el `boris_native`.

Es OPT-IN. Se arma **tildando el check `boris_native` en el Reproductor ANTES de dar play** (§13). Vive en el **server que corrió a Boris**: `<server>\profiles\BZ_AutoDrive_PathLogger\boris_native_*.csv` (uno por corrida, con timestamp — nunca se pisa).

> ⚠ **Igual que el `ai_run`: es MEDICIÓN, no una toma nueva.** No se lo des al wizard como grabación — realimentar la salida de Boris **clona sus errores** (§15.1, *model collapse*). Sirve para **comparar** contra tu grabación humana (la fuente de verdad), no para regenerar la ruta.

15.3 — Diagnóstico y cajas negras: `Diagnostics` y `BlackBox` *(desde la 1.2.1)*

Dos perillas en `profiles\BZ_AutoDrive\BZAutoDrive_settings.json`, las dos **apagadas por defecto**. Se leen **al arrancar el server** (cambiarlas pide reiniciar).

```
"Diagnostics": 1,
"BlackBox": 1,
"BlackBoxKeepPerRoute": 10
```

- **`Diagnostics`** — con `0` (el default) el RPT queda en lo esencial: la carga, el fin de cada ruta (`REPOSO` y `FIN`), los errores y cada intervención del AutoRecovery. Las líneas que se repiten mientras el vehículo anda (la dirección a cada paso, el latido de cada ruta, el `SLAM` de la llegada, los rescates del cursor…) solo salen con `1`. Con rutas en marcha, apagado el RPT pesa **más de un 90 % menos**. Prendelo cuando algo no anda y tengas que mirar.
- **`BlackBox`** — con `1` cada corrida escribe su caja negra, `profiles\BZ_AutoDrive\BZ12_LOG_<ruta>_<hora>.csv`: el estado del vehículo y del cerebro 40 veces por segundo (posición, velocidad, pedales, volante, el desvío de costado, el modo, quién manda). Es lo que se usa para entender una falla fina. Con `0` no se escribe nada.
- **`BlackBoxKeepPerRoute`** — cuántas cajas se guardan por ruta; las más viejas se borran solas (el registro va en `BZ12_LOG_indice.txt`). Default **10**.

---

16. Ideas — qué podés construir (y qué se podría sumar)

Framework + Quest abren un espacio grande. Estas ideas son para inspirarte: algunas las armás **hoy** con lo que hay; otras son **extensiones** para sumar (verbos/triggers nuevos). La regla: *si lo podés describir como "un vehículo que va de acá para allá y en el camino pasa X", probablemente lo podés armar — o sumar el verbo que falta.*

Arquetipos que armás HOY

- **Transporte público** — bus/taxi con paradas y eventos (§10).
- **Convoy** — refuerzos, escolta, huida, emboscada (§11).
- **Patrulla motorizada** — un vehículo en loop por una zona.
- **Logística / supply run** — un camión que lleva loot (`add_cargo`) a un punto y lo deja.
- **Rescate / extracción** — el vehículo llega, sube bots o jugadores, los saca.
- **Evento ambiental** — un auto que cruza el pueblo y se va; vida en el mundo, sin misión.

> *Ejemplo combinando:* misión **"protegé el convoy"** — un camión del framework hace la ruta, el quest pone enemigos que lo emboscan, y el jugador tiene que defenderlo hasta el destino.

Acciones que se podrían sumar *(extensión del DSL de eventos)*

- **Bloquear / expulsar asientos** (`lock_seat` / `unlock_seat` / `eject_passenger`) — que **los jugadores no puedan subirse** (o bajarlos). El campo `slot` ya está **reservado** en el DSL (`BZAction`); falta escribir el handler en `ExecuteAction` .
- **Cambiar la facción** de los ocupantes en runtime (`set_faction`) — que Boris o su crew cambien de bando a mitad de misión (de neutral a hostil, p. ej.).
- **Cambio de chofer** (`swap_driver`) — reemplazar al conductor en un punto (baja Boris, sube otro; o pasa a manejar un bot del quest).
- **Gestos** — saludo, venia, señalar (en el roadmap; todavia sin implementar).
- **Balizas / luces de emergencia** — balizas en una avería. *(La **bocina y las luces** comunes ya andan: como verbos de evento — `horn` , `lights_on` , `lights_off` — y como modo de la ruta — `HornMode` / `LightsMode` , Apéndice A.2b.)*
- **Soltar loot / abrir baúl** — dejar un ítem en un punto; abrir el baúl al llegar.
- **Llamar refuerzos** — spawnear *otro* vehículo del framework (cadenas de convoy).
- **Marcador en el mapa** — que el vehículo aparezca en el mapa del jugador (seguir el bus en vivo).
- **Dar / avanzar objetivo de quest** — que llegar a un punto **complete un objetivo** (el framework como motor de objetivos).

Triggers que se podrían sumar

- **Por ocupación** (`occupancy >= N`) — que el vehículo espere a **llenarse** (N pasajeros/bots sentados) antes de arrancar. Hoy `player_enter_vehicle` ya dispara al subir **el primero**; falta el umbral de "cuando estén los N". Combinado con `start_engine` + `resume_route` = el bus sale **cuando se completa el cupo**.
- **Al disparar** (`on_player_shoot`), **de noche**, **por cantidad de jugadores** en la zona, **al entrar a un área**.

Combos framework + Quest *(el espacio grande)*

- **Taxi que es quest-giver** — te subís, te lleva, y en el viaje te ofrece una misión.
- **Jefe que llega en vehículo** — el boss aparece manejando, baja y pelea.
- **Trader móvil** — un vendedor recorre una ruta; los jugadores lo paran para comprar. *(Necesita integración con Market — idea parqueada.)*
- **Eventos dinámicos** — spawns aleatorios de vehículos + rutas, para que el mundo se sienta vivo.

- **El framework como motor de objetivos** — que los eventos del vehículo (llegó / fue destruido / entregó) sean las **condiciones de completion** de un quest. *Este es el norte del proyecto.*

Framework + cada tipo de objetivo de Quest

La integración se validó in-game con **AICamp/AIPatrol** (los convoyes, §11), **Travel** (§11.10) y **AIEscort/AIVIP** (§11.11) — los tipos donde el vehículo cumple un rol. Los objetivos sin vehículo (Collection, Crafting, etc.) coexisten sin conflicto. La tabla completa:

Objetivo de Quest	Rol del vehículo (BZ_AutoDrive)	Ejemplo
AICamp / AIPatrol ✅	trae/saca los bots, los embarca y maneja	el convoy que huye, la emboscada (§11)
AIEscort ✅	el vehículo **es lo que se escolta**	"acompañá el convoy de Boris hasta la base"; si llega entero, cumplido
AIVIP ✅	el VIP **viaja en** el vehículo	"protegé al VIP" mientras el NPC lo lleva a zona segura
Travel ✅	el vehículo **te lleva**	"viajá a Cherno" se completa subiéndote al bus que va para allá
Delivery	el vehículo **hace la entrega**	supply-run: el camión NPC lleva la carga al punto; el objetivo es que llegue
Target	el target **va en** el vehículo	"eliminá al jefe" que se escapa en el convoy → lo perseguís
Action / Collection	el vehículo **deja algo** o **es el lugar de la acción**	inspeccionar el accidente, reparar el camión, recoger el loot que soltó

> *Lo validado:* el framework "escucha" el quest y aporta el vehículo, sea cual sea el objetivo. Ya andan in-game los **tres ejes donde el vehículo cumple un rol** — **AICamp/AIPatrol** (convoyes, §11.3/§11.4), **Travel** (§11.10) y **AIEscort/AIVIP** (§11.11). Los demás (Delivery, Target, Action/Collection) **reusan el mismo hook** con su propio campo declarativo: el enganche ya existe, falta el caso de uso.

> *Para modders:* si se te ocurre una acción o trigger que no está, casi siempre **encaja en el DSL existente** (un verbo nuevo en `ExecuteAction`, un trigger nuevo en la lista). El AI knowledge pack te muestra cómo agregarlo.

---

17. Alcance y límites (qué esperar)

Para que sepas qué pedirle al framework y qué no:

- **Régimen nominal:** maneja al límite *normal* del vehículo. Maniobras extremas (derrapes, sobreviraje fuerte, contravolante) **no** las reproduce con fidelidad — el receptor del motor no las entrega. Eso queda fuera de alcance, y está bien: el framework apunta al manejo realista, no al stunt.
- **Alta velocidad:** anda, con **degradación graceful**. Probado hasta ~190 km/h: la línea se afloja un poco con la velocidad (de ~0.2m parado a ~0.8m a +120 km/h) pero **no se rompe** (sin oscilación, 99% dentro de 2m).

- **Curvas de 90° cerradas:** es el punto más exigente. Salen, pero ahí se nota el vehículo — uno pesado se abre más que uno chico.
- **El CONTROLADOR generaliza** sobre un **rango amplio de R_min** (de un hatchback chico a un camión) con **una sola config**: la misma traza sirve para todos y el matiz lo pone la física de cada uno.
- **Maniobras (parking / reversa / K-turn):** el **intercambio reversa→forward** —el punto más exigente del control, un K-turn adentro de un galpón— está **resuelto**: se **auto-detecta del cambio de sentido** que hiciste al grabar (no hace falta marcarlo — §9.4), funciona con reglas físicas y **sin constantes por-vehículo**, y se probó en un banco amplio (Golf FWD, BMW E60, OffroadHatchback, Hatchback FWD, Porsche GT2RS motor-central, Sedan, Toyota 86 RWD y el camión Truck_01) con **0 rescates de AutoRecovery**. **La condición es tuya y es una sola: hacé el cambio de sentido en recta, no en curva** (§9.5). Los ángulos de volante de la reversa son del vehículo, no del camino. **Para la misma ruta en otro vehículo, grabala con ese vehículo** — con una maniobra de por medio es lo único que generaliza.
- **Precisión de llegada (endpoint).** El freno final se arma con lo que declara el config del vehículo y lo que se midió de él (cuanto más rápido llega, antes se arma: con la distancia de frenado que pide su velocidad real, más un margen), y **queda a unos 10 cm del punto**: apunta al punto, y ahí es donde lo deja el freno real. El freno se planea con **la frenada que mediste vos en la toma** cuando la toma termina frenando a fondo (`[C12] FRENADA de la toma: 5.4 m/s2 …` en el RPT): el freno que declara el config suele ser optimista. De una moto a un bus, la misma regla: no se relaja por tamaño, porque el framework tiene que servir para vehículos que todavía no conocemos. **Los números, vehículo por vehículo, en §17.1.**
- **El veredicto: cuándo una ruta está completa** *(desde la 1.2.1)*. El fin de ruta se decide **con el vehículo quieto**: medio segundo después de detenerse (a lo sumo unos 3 s después de llegar) sale `[C12] REPOSO tras T s: a X m de la pose final (longitudinal a m, lateral l m)`, y enseguida el `FIN`. `ROUTE_COMPLETE` exige **las tres**: (1) en reposo, a **0,25 m o menos** del punto final de la toma; (2) **ningún choque contra un objeto** en toda la corrida (un bache o un golpe contra el terreno no cuentan); (3) **ninguna intervención física del AutoRecovery** (maniobra o re-ubicación; los re-arranques suaves no cuentan, pero se listan). Si falla alguna, `ROUTE_INCOMPLETE`, y la misma línea dice por qué: `quedo a X m de la pose final en reposo`, `choco contra un objeto: wp N contra <clase>`, `AR fisico con accion: wp N <accion>`. Vale igual para autos y motos. *(En la 1.2 una llegada entre 0,25 m y 1 m contaba como completa con un aviso: ya no.)*
- **Qué vehículos probamos.** Más de 40 clases de varios autores: autos, deportivos, 4x4, furgones, buses, camiones (hasta 7,5 m de batalla), militares, tractores y motos (§17.1).
- **Lo que NO hace:** no inventa rutas (necesita tu grabación) ni descubre comportamiento más allá de lo que demostraste. Es determinista e inspeccionable — esa es su gracia.

> En una frase: **fiel y predecible en el régimen normal; el límite honesto está en lo extremo.**

17.1 — La llegada, vehículo por vehículo (medido)

Estos números salen de los logs del servidor de pruebas, **con el cerebro 1.2**. Cada fila es un vehículo y cada "llegada" es una corrida que terminó su ruta; la distancia es la que quedó entre el vehículo y el punto final grabado.

Cómo leerla:

- **La mediana ronda los 0,10-0,12 m.** El freno apunta al punto y queda a unos **10 cm**, que es donde lo deja el freno real. El objetivo es **0,25 m**: desde la 1.2.1, más que eso en reposo es `ROUTE_INCOMPLETE` (el veredicto, arriba). *(La primera tabla es de la 1.3; las dos de abajo, de la 1.2.1 y la 1.2, quedan como historia.)*
- **p90** = 9 de cada 10 llegadas quedaron a esa distancia o menos.
- **Las corridas que no llegaron no están en la tabla**: se cuentan aparte, abajo.
- Una corrida repetida desde el mismo punto de salida puede dar casi el mismo número: más llegadas no siempre son más variedad.

Con la 1.3 (noviembre de 2026): un circuito de montaña con subidas, 13 vehículos a la vez y, en la mitad de las corridas, restos de vehículos sobre la ruta. 227 llegadas: mediana **0,11 m**, **97 %** dentro de 0,25 m. Autos, 4x4, camión y blindado cierran todos a **0,10-0,15 m** de mediana, también los pesados y en cuesta; de sus 159 llegadas, una sola quedó a más de 0,25 m (el blindado, después de chocar con un cisterna del evento). Las motos cierran con su propio conductor (§12): dos de ellas a 0,02 y 0,19 m fijos; la `Motorbike_01` llega a 0,02-0,05 m cuatro veces de cada diez y a 0,4-0,7 m las otras seis, entrando torcida (5-12°).

Tipo	Vehículo (clase)	Llegadas	Mediana	p90	Dentro de 0,25 m
auto	`Sedan_02`	20	0,10 m	0,10 m	100 %
auto	`minicooper`	18	0,11 m	0,11 m	100 %
auto	`V140`	18	0,11 m	0,11 m	100 %
deportivo	`rx8hachi`	14	0,11 m	0,11 m	100 %
deportivo	`Star_Agera`	12	0,15 m	0,16 m	100 %
4x4 / pickup	`Expansion_Landrover`	17	0,10 m	0,10 m	100 %
4x4 / pickup	`Star_Jeep_Rubi_Extra`	18	0,11 m	0,17 m	100 %
4x4 / pickup	`Star_Chevrolet_Z71`	14	0,11 m	0,13 m	100 %
camión	`Star_Oshkosh_M1083`	16	0,12 m	0,14 m	100 %
militar	`Star_APC_Cobra_camoorange`	12	0,10 m	0,11 m	92 %
moto	`Motorbike_02_Yellow`	26	0,02 m	0,03 m	100 %
moto	`Motorbike_02_Red`	32	0,19 m	0,19 m	100 %
moto	`Motorbike_01_Blue`	10	0,43 m	0,56 m	40 %

La `Motorbike_01_Blue` sale de una toma que termina doblando a paso de hombre: por eso queda lejos. Con una toma que termina en recta (§7.1) cierra a centímetros.

Con la 1.2.1 (desde el 17/09/2026). 52 llegadas: mediana **0,11 m**, **87 %** dentro de 0,25 m, ninguna a más de 0,36 m. En ese período, 7 corridas no llegaron (vehículo trabado sin salida, o chofer muerto).

Tipo	Vehículo (clase)	Llegadas	Mediana	p90	Dentro de 0,25 m
4x4 / pickup	`MBM_LandCruiser_Grey`	21	0,18 m	0,31 m	67 %
furgón	`MBM_Sprinter_Green`	1	0,10 m	0,10 m	100 %
bus	`BZ_ChernoBus_02`	17	0,11 m	0,18 m	100 %
bus	`MBM_ApocalypticPAZ_White`	8	0,11 m	0,11 m	100 %
bus	`MBM_ApocalypticPAZ_Black`	4	0,11 m	0,11 m	100 %
bus	`BZ_ChernoBus`	1	0,11 m	0,11 m	100 %

Del 2 al 16/09/2026, con una flota mucho más variada. Todavía faltaban los dos últimos ajustes de la llegada (entrar al freno final a 40 veces por segundo y corregir el desvío lateral en los últimos metros), así que estos números son **los peores esperables**: los de hoy son iguales o mejores. 430 llegadas en 41 vehículos: mediana **0,19 m**, **77 %** dentro de 0,25 m. En ese período, 40 corridas no llegaron.

Tipo	Vehículo (clase)	Llegadas	Mediana	p90	Dentro de 0,25 m
auto	`Hatchback_02`	50	0,28 m	0,57 m	48 %
auto	`Sedan_02`	11	0,21 m	0,63 m	64 %
auto	`MBM_ImprezaWRX_Rusty_Orange`	4	0,83 m	0,94 m	0 %
auto	`a_m5`	3	0,13 m	0,25 m	100 %
auto	`CivilianSedan`	3	0,11 m	0,20 m	100 %
deportivo	`jesko`	26	0,16 m	0,23 m	100 %
deportivo	`m4w_Blue`	7	0,25 m	0,51 m	57 %
deportivo	`Star_Agera`	6	0,11 m	0,12 m	100 %
deportivo	`Gemera`	6	0,21 m	0,23 m	100 %
deportivo	`lfa`	5	0,19 m	0,66 m	60 %
deportivo	`r32`	3	0,21 m	0,22 m	100 %
4x4 / pickup	`Expansion_Landrover`	47	0,22 m	0,32 m	87 %
4x4 / pickup	`OffroadHatchback`	5	0,22 m	0,60 m	60 %
4x4 / pickup	`MBM_F350_Monster_Red`	4	0,11 m	0,12 m	100 %
4x4 / pickup	`MBM_LandCruiser_Grey`	3	0,16 m	0,17 m	100 %
furgón	`MBM_Sprinter_Green`	13	0,10 m	0,10 m	100 %
bus	`BZ_ChernoBus`	39	0,22 m	1,02 m	54 %
bus	`MBM_ApocalypticPAZ_Yellow`	11	0,11 m	0,11 m	100 %
bus	`BZ_ChernoBus_02`	6	0,20 m	4,65 m	83 %
bus	`MBM_ApocalypticPAZ_Black`	6	0,12 m	0,34 m	83 %
bus	`MBM_ApocalypticPAZ_Blue`	3	0,10 m	0,10 m	100 %
camión	`AEC_Matador`	48	0,17 m	0,37 m	90 %
camión	`hx58`	9	0,17 m	0,24 m	100 %
camión	`Star_Oshkosh_M1083`	9	0,13 m	0,19 m	100 %
camión	`MBM_Apocalypse_Truck`	3	0,11 m	0,11 m	100 %
militar	`m4_tractor_winter`	50	0,10 m	0,41 m	74 %
militar	`Star_APC_Cobra_grey`	4	0,23 m	0,62 m	75 %
tractor	`ExpansionTractor`	30	0,22 m	0,44 m	67 %

Con una o dos llegadas (sin estadística): `MBM_HarleyDavidsonFatBoy_Black` 0,11 m, `MBM_ImprezaWRX_WRC` 0,11 m, `OffroadHatchback_White` 0,22 m, `PzKpfw_VI_Tiger_camo` 0,52 m, `SVJ` 0,15 m, `Star_Camaro_1969` 0,24 m, `Star_Chevrolet_Z71` 0,10 / 0,17 m, `Star_Porsche_944` 0,11 m, `Toyota_86_RB_Red` 0,22 / 0,71 m, `Truck_01_Covered` 0,11 / 0,19 m, `VW_T6` 0,22 m, `ZiS5_Benzovoz` 0,24 m, `esc` 0,13 m.

> **Los casos que quedan lejos se repiten en el mismo vehículo y la misma ruta.** El `MBM_ImprezaWRX_Rusty_Orange` quedó 0,8 m **de costado** las 4 veces; el `Hatchback_02` tiende a **pasarse** del punto. Si un vehículo tuyo llega siempre corrido, mirá primero cómo termina su toma (§7.1) y probá regrabar ese final.

---

18. Frontera abierta — llevalo más lejos

Esto es open source por una razón: **llegué hasta acá; el siguiente nivel es tuyo.** Acá está, sin vueltas, dónde está el borde y por dónde se empuja. Si vas a contribuir, arrancá por una de estas.

El salto grande: manejo extremo (un "v3")

Dónde está hoy: maneja al régimen **nominal** del vehículo (§17). Drifts, sobreviraje fuerte, contravolante — el receptor del motor no los reproduce con fidelidad. *El salto:* override de física propio (saltear el modelo de eAI en esos tramos) o una capa de ML que capture lo que el control clásico no. Es el más grande y el más difícil — y el que abre el manejo "de película".

Wizard 100% autónomo

Dónde está hoy: convierte sin preguntar y **avisa** lo raro de la toma (trompos, saltos, detenciones sin marca), pero no corrige nada: si la toma tiene un problema, la regrabás vos. *El salto:* que te diga **qué tramo regrabar y por qué** antes de que lo corra el NPC — y que mida solo lo que hoy no se puede medir de una toma (el piso de despegue, §6C).

Coordinación multi-vehículo

Dónde está hoy: varios vehículos pueden correr la misma ruta (un runner por vehículo, Apéndice B.7) y tu mod puede coordinarlos —horarios, cesión de paso—, pero el framework no los coordina entre sí. *El salto:* convoyes reales — spacing dinámico, el de atrás sigue al de adelante, reformación si uno cae. Hay técnica prestada de ARMA/RV (breadcrumb + separación de convoy) para inspirarse.

El NPC vivo

Dónde está hoy: el chofer maneja, pero es mudo. *El salto:* un conductor con LLM que hable, reaccione y hasta dé una misión en el viaje. (No viene nada de esto en el paquete: es una idea abierta.)

Más mundos

- **Trenes** — el caso más fácil: la red *son* los rieles, sin volante ni giros inventados. ~70% cubierto sin tocar nada.
- **Otros motores** — el principio (demostración + lectura de config + control clásico) no es exclusivo de Enfusion; es portable.

> **Si tomás una:** el AI knowledge pack te deja el contexto completo —por qué cada decisión, qué se probó, qué falló— para que arranques **en el borde y no repitas el camino**. Esa es la idea de abrirlo:

> que el próximo empiece donde yo terminé. Tomala y seguí. 🚀

Apéndice A — Referencia completa de la config de ruta

Todos los campos del JSON (`BZBusRoute*.json`). Casi todos tienen un default sensato: **solo tocás lo que necesitás**. Convención: un valor `-1` o `0` suele significar "*usá el default del código / desactivado*".

> **Qué lee cada cerebro.** El cerebro 1.2 (el que corre por defecto) lee **A.1, A.1b, A.2b, A.5, A.6 y A.8**. Los campos de **A.2, A.3, A.4, A.5b, A.5c y A.7** son del cerebro 1.1: sólo actúan si el servidor tiene `CEREBRO_11.txt` con contenido (§4). Se dejan documentados para quien use el 1.1.

A.1 Básicos

Campo	Default	Qué hace
`VehicleClass`	`"ExpansionBus"`	qué vehículo spawnea
`DriverClass`	`eAI_SurvivorM_Boris`	el NPC chofer
`RespawnDelay`	`300`	seg antes de respawnear (servicio continuo)
`AverageSpeedMS`	`11`	velocidad promedio, para el cálculo de ETA
`SpawnHoldSeconds`	`3`	espera antes de arrancar (`0` =al toque, `30+` =esperar trigger de misión)
`VehicleInvincible`	`true`	irrompible; `false` = destructible (habilita daño/emboscada)
`VehiclePartsLocked`	`true`	**inviolable**: mientras el framework lo maneja, los jugadores no le pueden sacar piezas (ruedas, puertas, batería…); `false` = se pueden sacar, como en cualquier auto. La carga no se toca. Un evento lo cambia (`set_vehicle_parts`)
`DriverInvincible`	`true`	Boris irrompible; `false` = mortal (puede morir en combate / al recibir daño)
`DriverClothing`	`[]`	ropa del **chofer** (classnames a equipar). Vacío = outfit default del framework. Ver §A.1b
`MaxGear`	`6`	marcha máxima de la caja automática (FIRST=2 … SIXTH=7)
`Attachments`	`[]`	partes a equipar al spawnear (ruedas, batería, bujías…)
`Wheelbase`	`0`	distancia entre ejes (`0` =del código); lo usa el reverse

A.1b Cómo vestir al chofer

El framework viste a **Boris (el chofer, seat 0)** con un outfit configurable **por ruta**. Lo definís con `DriverClothing` en el header/JSON de la ruta: una lista de **classnames** de ropa, cada uno se equipa en su slot.

```
"DriverClothing": ["BZ_AutoDrive_TShirt", "PolicePants", "PolicePants", "PoliceCap", "CombatBoots_Black"]
```

- **Si NO seteás `DriverClothing`** (o lo dejás `[]`): Boris usa el **outfit default del framework** — `BZ_AutoDrive_TShirt` (la remera branded, va plegada dentro de `@BZ_AutoDrive`, siempre cargada) + `PolicePants` + `PolicePants` + `PoliceCap` + `CombatBoots_Black`. Las rutas viejas siguen viéndose igual que siempre.
- **Si seteás `DriverClothing` con items:** esa lista **reemplaza** el default por completo (ponés el outfit entero que querés, no se mezcla).
- Cada classname se equipa con `CreateAttachment` por string: si un classname **no existe o su mod no está cargado**, ese ítem **falla en silencio** (no rompe nada, simplemente no aparece). Usá classnames válidos (de un mod de ropa cargado, o vanilla).

> **Alcance:** esto viste **solo al chofer**. La ropa/loadout de los **bots de convoy/crew** la maneja **Quest (Expansion Quests)**, no el framework — ver §11.7.

A.2 Velocidad / control de manejo *(cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

*(Con el 1.1: por default la ruta usa **tu velocidad grabada** (capeada por la curva) con **modelo inverso**; `FollowPath=true` calcula la velocidad por curvatura en vez de usar la grabada. Estos flags eran los viejos "modos 1/2/3".)*

Campo	Default	Qué hace
`GearStrategy`	`"auto_box"`	`auto_box` (la AT decide) o `follow_recording` (usa el gear grabado — para sport cars que patinan en 1ra)
`FollowPath`	`false`	`true` = calcula la velocidad por **curvatura** (ignora la grabada); `false` (default) = usa tu velocidad grabada
`FollowPathLatAccel`	`4.0`	grip lateral (m/s²) → define la vmax por curva
`FollowPathMaxKmh`	`50`	tope de velocidad en recta (cuando `FollowPath=true`)
`FollowPathCurveSpan`	`5`	espaciado de wps para medir la curvatura (evita ruido)
`FollowPathSpeedSmooth`	`8`	suaviza el perfil de velocidad (anticipa curvas)
`FollowPathUseReference`	`false`	usa la velocidad grabada capeada por la curva (parte del control por default)
`UseInverseModel`	`false`	throttle/brake por PID + modelo inverso (vehicle-agnostic)
`InverseModelKp/Ki/Kd`	`-1`	gains del PID de velocidad (`-1`=default)
`InverseModelLowRpmMin`	`false`	gear amortiguado (gears altos en cruise, más estable)
`TargetSpeedSmoothWindow`	`0`	suaviza la velocidad objetivo (menos oscilación del PID)
`AccelShiftThreshold`	`999`	anti-catapulta: sube gear si acelera de más (`999`=off)

A.2b Bocina y luces *(§9)*

Campo	Default	Qué hace
`HornMode`	`"off"`	`off` (nunca) · `stops` (bocina al terminar cada parada) · `finish` (al llegar al final, si la ruta termina completa). `replay` no suena: la toma no graba la bocina (sólo el verbo `horn`; el server lo avisa al cargar)
`LightsMode`	`"auto"`	**`auto`** (default — prende de noche, 19:00–06:00, y sigue la hora durante la ruta) · **`off`** (siempre apagadas → **desactiva las luces**, p/ misiones nocturnas sigilosas) · `auto_inverted` (apaga de noche → sigilo) · `on` (siempre). `replay` no sirve en esta versión: la toma no graba las luces. Un verbo `lights_on` / `lights_off` manda hasta el próximo spawn

> **¿Bocina o luces en un punto exacto?** Usá los verbos `horn`, `lights_on` y `lights_off` en un evento (§10).

> **Default `auto`:** todas las tomas prenden los faros solas de noche, sin grabar nada. Las luces se encienden **apenas arranca el motor** (durante el spawn-hold), no a mitad de ruta. Umbral de noche = 19:00–06:00 (hora del mundo del juego). **Desactivar las luces:** poné `"LightsMode": "off"` en el header de la ruta (misión sigilosa nocturna → el vehículo no se delata). Para que las luces se vean, el vehículo necesita **batería + faros (bombillas) instalados** (§7); el framework energiza la batería al spawn, pero los faros tienen que estar en los `Attachments`.

A.3 Dirección (steering) *(cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

Campo	Default	Qué hace
`SteeringScale`	`-1` (auto)	escala del volante; wheelbase corto → bajar (auto lo deriva del wheelbase)
`CurvatureSteerBoost`	`0`	amplifica el volante en curva (contra el sub-giro en 90°)
`PathSmoothWindow`	`5`	suaviza las posiciones del path
`CruiseLateralDeadband`	`0`	"paredón": banda donde no corrige (anti-microvolante); `0.5` recomendado
`CruiseLateralKGain`	`1.0`	ganancia de la corrección lateral
`CruiseLateralDamp`	`0`	damping del lateral (mata el zigzag); `0.3` moderado
`CruiseLateralCenterOffset`	`0`	corrige el sesgo lateral (`+` =derecha, `−` =izquierda)
`CruiseHybridSteerThreshold`	`-1`	usa el volante grabado si supera el umbral (pulsos de teclado)
`CruiseHybridThrottleThreshold`	`-1`	usa el throttle grabado cuando el humano aceleraba
`CruiseFFWeight`	`-1`	peso del feedforward de curvatura en cruise (`-1` =0.25)
`CurveThrottleEnabled` (+ `...LookaheadM/StartDeg/FullDeg/MinScale`)	`true`	corta el throttle ANTES de una curva cerrada

A.4 Pendientes *(cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

Campo	Default	Qué hace
`SlopeCompensationEnabled`	`true`	suma throttle en subida, resta en bajada
`SlopeLookaheadWps`	`5`	cuántos wps adelante mira la pendiente
`SlopeGain`	`1.0`	cuánto compensa (`1` =completo)
`SlopeLateralGain`	`1.0`	corrige el bias lateral que mete la pendiente

A.5 AutoRecovery — que la ruta no se corte *(ver §9.6)*

Bloques **opcionales** del header/JSON de la ruta. **Sin escribir nada, rigen estos defaults**, que es el comportamiento validado. Si escribís el bloque a medias, los campos que falten toman su default (`{"Physical": {"ReverseM": 4}}` deja `Enabled` en `true` y cambia solo los metros de reversa).

```
"AutoRecovery": {
  "Logical":  { "Enabled": true, "WaitS": 0.5, "MaxAttempts": 3, "CooldownS": 1.0 },
  "Physical": { "Enabled": true, "ReverseM": 3.0, "ReverseMaxKmh": 5.0, "LadderMaxM": 6.0, "SweepMaxM": 10.0
}
},
"BogRecovery": { "Enabled": true, "WindowS": 3.0, "MinProgressM": 0.5, "MaxAttempts": 2 },
"ManualAR":    { "AdvanceM": 10.0, "GoodLatM": 1.0 }
```

`AutoRecovery.Logical` **— re-arranque en el lugar** (el cerebro se trabó; el vehículo puede moverse)

Campo	Default	Qué hace	Cuándo cambiarlo
`Enabled`	`true`	re-arranca en el lugar, sin mover el vehículo	`false` para diagnosticar: la traba queda registrada igual y ves dónde pasa
`WaitS`	`0.5`	segundos detenido antes de actuar	subilo si querés que la detención se note en paradas sin evento de espera
`MaxAttempts`	`3`	intentos en el mismo lugar (radio 8 m); `0` = sin límite	bajalo si preferís que una ruta rota se declare trabada antes
`CooldownS`	`1.0`	pausa mínima entre intentos	subilo si ves intentos encadenados muy seguidos

`AutoRecovery.Physical` **— la escalera** (choque, quedar cruzado, montado sobre algo, hundido o volcado)

Campo	Default	Qué hace	Cuándo cambiarlo
`Enabled`	`true`	activa la escalera de §9.6	`false` en réplica pura, o para ver dónde se traba sin que nada lo destrabe
`ReverseM`	`3.0`	metros de reversa recta del primer paso	subilo con vehículos largos; bajalo en calles angostas
`ReverseMaxKmh`	`5.0`	tope de velocidad de esa reversa	bajalo en convoy o con tránsito detrás
`LadderMaxM`	`6.0`	metros de zonas seguras, contados **desde la primera zona**	subilo para que el AutoRecovery nunca corte la ruta
`SweepMaxM`	`10.0`	metros de barrido cuando el destino está ocupado o el vehículo queda montado	subilo si el obstáculo típico de tu servidor es largo (un camión, una barricada)

`BogRecovery` **— ahogo del motor** (lo hace el controlador)

Campo	Default	Qué hace	Cuándo cambiarlo
`Enabled`	`true`	ante un ahogo fuerza **primera** y arranca como en pendiente	`false` deja solo el registro, sin corregir
`WindowS`	`3.0`	segundos con acelerador y sin avanzar antes de decidir	subilo para vehículos pesados de arranque lento
`MinProgressM`	`0.5`	avance que ya cuenta como "avanza"	subilo si el vehículo cabecea mucho al arrancar
`MaxAttempts`	`2`	intentos en el mismo lugar; al agotarse pasa a la escalera	—

`ManualAR` **— la intervención del operador** (bandera `BZ12_AR_MANUAL.txt`)

Campo	Default	Qué hace
`AdvanceM`	`10.0`	metros adelante del último punto bien pisado donde re-ubica
`GoodLatM`	`1.0`	desvío lateral máximo para que un punto cuente como "bien pisado"

Ejemplos.

```
// Servicio de transporte desatendido: que nunca corte la ruta.
"AutoRecovery": { "Physical": { "LadderMaxM": 20.0, "SweepMaxM": 30.0 } }

// Réplica pura: quiero ver exactamente lo que hace el vehículo, sin ayudas.
"AutoRecovery": { "Logical": { "Enabled": false }, "Physical": { "Enabled": false } },
"BogRecovery": { "Enabled": false }

// Convoy: reversa más corta y lenta, por el que viene atrás.
"AutoRecovery": { "Physical": { "ReverseM": 2.0, "ReverseMaxKmh": 3.0 } }
```

> **Dónde mirar el resultado:** la línea `[C12] AutoRecovery <ruta> ...` del RPT imprime, una vez por corrida, la configuración que realmente quedó activa. Si creés que un valor no se está aplicando, esa es la línea que lo dice.

A.5b Obstáculos externos — AR_OnWay *(cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

Distinto del AutoRecovery (que cuida a Boris de trabarse **solo**), esto lo cuida del **mundo**: otro vehículo detenido en el camino, o uno que lo choca/empuja. Dos flags independientes; el **import de rutas v1** (wizard **[2]**) te pregunta el perfil.

Campo	Default	Qué hace
`ObstacleSlow`	`false`	freno **predictivo** ante un vehículo adelante (lookahead que crece con la velocidad y la física del vehículo; frena a su máximo real hasta detenerse antes del obstáculo)
`ObstacleEscape`	`false`	si el obstáculo **persiste** (o lo chocan/empujan fuera de la línea), **teleporta** al primer wp limpio pasado el obstáculo
`ObstacleScanDist`	`50`	distancia mínima (**piso**) de escaneo del path; el lookahead real crece con la velocidad
`ObstacleStopDist`	`15`	a cuántos metros del obstáculo se detiene
`ObstacleCorridorHalf`	`2.3`	medio-ancho del carril: un auto en la **banquina** (offset lateral > esto) NO frena a Boris
`ObstacleEscapeWaitS`	`6`	seg frenado ante el obstáculo antes de escapar
`ObstacleEscapeResumeKmh`	`10`	velocidad suave al retomar tras el teleport

> **Perfiles (el import v1 los pregunta; en una toma nueva se setean por config): Transporte robusto** = ambos ON (el bus sortea lo que le tapa el camino). **Interceptable** = `ObstacleSlow` ON + `ObstacleEscape` **OFF** (Boris frena lindo ante el que lo bloquea pero **no** se escapa → la misión de interceptación funciona). **Ninguno** = ambos OFF (réplica pura).

A.5c AutoRecovery clásico por teleport *(cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

Mecanismo anterior, **independiente** del de A.5 y **apagado** por defecto: cuando el vehículo lleva N segundos trabado, lo teletransporta unos waypoints más adelante. El de A.5 lo cubre sin salto visual; esto queda para quien lo necesite como última red.

Campo	Default	Qué hace
`AutoRecoveryEnabled`	`false`	si Boris se traba, lo teleporta adelante
`AutoRecoveryStuckTimeS`	`10`	seg de trabado antes de teleportar
`AutoRecoveryAdvanceWps`	`5`	cuántos wps adelante teleporta
`AutoRecoveryCooldownS`	`8`	mínimo entre teleports (anti-spam)
`AutoRecoveryMaxPerMission`	`0`	`0` =ilimitado, `X` =falla la misión si supera
`StuckAdvanceTimeoutS`	`0`	safety **separada**: si no avanza ningún wp por N s, **empuja** el índice (no teleporta). `0` =default 60 s. Corre aunque el resto esté off; subir mucho para deshabilitar

A.6 Convoy / Quest *(ver §11)*

Campo	Default	Qué hace
`ConvoyMode`	`""`	`""` / `"flee_on_kill"` / `"ambush_on_damage"`
`Crew`	`[]`	bots que viajan desde el arranque (§11.7)
`Events`	`[]`	nodos de evento (§10, §11.7)

A.7 Reversa e intercambios — *avanzado (cerebro 1.1)*

> ⚠ **Sólo cerebro 1.1.** Estos campos sólo se leen si el servidor tiene `CEREBRO_11.txt` con contenido (§4). El cerebro 1.2, el que corre por defecto, no los usa.

> **El flujo hoy es simple.** La **reversa** se auto-detecta del **gear grabado**, la **aproximación** es automática y el **cambio de sentido** se **auto-detecta** del cambio de gear forward↔reverse —sin tecla— (§9.4). Los viejos **modos de waypoint marcados a mano** (`parking` , `maniobra` , `approach`) **ya no se producen** — sus campos quedan solo por compatibilidad con tomas viejas (marcados *legacy* abajo). | Campo | Default | Qué hace | |---|---|---| | `DirectReplayFromWaypoint` | `-1` | desde este wp, replay literal de los inputs grabados | | `ManiobraTargetSpeedCap` | `18` | *(legacy)* tope del viejo modo `maniobra` (deprecado, no se produce) | | `ParkingTargetSpeedCap` | `0` | *(legacy)* tope del viejo modo `parking` (deprecado, no se produce) | | `ModeEntrySnapEnabled` / `...MaxDist` | `false` / `0.5` | snap (teleport) a la pos+heading grabados al entrar a la **reversa**. **OFF por default**: el control genuino posiciona sin teleport (§9.5); reactivalo solo si una ruta puntual lo necesita | | `AntiRollbackEnabled` / `...PitchThreshold` | `true` / `0.05` | clava frenos en pendiente a velocidad ~0 (no rueda hacia atrás) | | `ParkingStanleyK` / `ParkingFFWeight` | `-1` | *(legacy)* Stanley del viejo modo `parking` (deprecado) | | `ReverseStanleyK` / `ReverseFFWeight` / `ReverseFFSign` / `ReverseFFMaxSteerRad` | `-1` / `-1` / `0` / `0` | control del modelo-bicicleta en reversa | | `ReverseSteerGateOffset` / `ReverseSteerThrottleFloor` / `ReverseSteerMax` | `0` | gates anti-stall y anti-overshoot en reversa | | `ReverseRecordedSteerThreshold` | `0` (→0.2) | seguir el volante grabado en reversa (recording-as-manual) | | `ReverseTargetSpeedCap` / `ReverseStanleyFineMax` / `ReverseHeadingDeadbandDeg` | `0` | techo de velocidad / corrección fina / deadband de ángulo en reversa | | `EndFreezeDisabled` | `0` | `0` =al final, frena y congela donde el humano se detuvo; `1` =sigue sin frenar | | `ReverseStanleyMinSpeed` | `0` (→2 m/s) | piso de velocidad para la corrección lateral en reversa (rompe la espiral 1/v a baja velocidad) | | `ApproachAuto` | `false` | freno predictivo **automático** a la maniobra — es el comportamiento por default (ya no se marca `approach` a mano) | | `ApproachExitKmh` | `0` (→~20) | velocidad objetivo al final de la rampa de aproximación |

> **Velocidad de reversa — automática, sin config (validado en 4 vehículos, R_min 3.44–4.57).** Boris reversa a `min(velocidad grabada, velocidad que su física le permite para el arco)` → generaliza a cualquier vehículo (uno de giro ancho reversa más lento y por eso sigue mejor el arco). Al acercarse al **final** del tramo de reversa frena solo (*endpoint-taper*): en **plano** llega a paso de hombre (mata la sobre-pasada de la salida), en **cuesta arriba** mantiene el momentum para trepar (no frena de más). No hay campos que tocar — sale de la física del vehículo y de la pendiente.

> **Corte de la maniobra en recta (para generalizar — §9.5).** El replay directo (`DirectReplayFromWaypoint`) reproduce **ángulos de volante específicos del vehículo**: si el tramo de

> reversa empieza **en una curva**, un vehículo con otro wheelbase se abre. Hacé el **cambio de sentido** (frená del todo antes de invertir la marcha, ahí el auto-detect corta el tramo) **donde la trayectoria ya es recta** (la curva la resuelve el control cerrado, agnóstico al vehículo). Ojo con el auto-approach (`ApproachAuto`): puede **frenar de más** si el corte cae en una recta rápida — por eso conviene cortar donde ya venís lento.

> **Intercambio y endpoint del cerebro 1.1.** El *cusp* (reversa→forward) y el freno de parada autoadaptativo del 1.1 se validaron en un banco amplio de tracciones y tamaños. **La precisión de llegada del cerebro 1.2 está en §17.** | Campo | Default | Qué hace | |---|---|---| | `EndpointThrottleCapEnabled` | `true` | activa el corte de gas + freno autoadaptativo en la zona del endpoint final | | `EndpointStopDecelMS` | `0` (→auto) | decel objetivo de la parada; `0` = la deriva del config del vehículo × agarre del piso real | | `EndpointStopDecelFactor` | `0.85` | fracción del freno máximo real que aplica (margen para no bloquear) | | `FwdClimbFactor` | `1.6` | ayuda de throttle sostenida a un **tracción-delantera** para trepar en la subida del endpoint (evita los "pasitos") |

A.8 Waypoints

Cada entrada de `Waypoints[]`:

Campo	Qué es
`pos`	`[x, y, z]` la posición
`targetSpeed`	velocidad objetivo (km/h) en ese punto
`targetGear`	marcha (si `follow_recording`)
`mode`	`normal` o `reverse` — la reversa se **auto-detecta del gear** grabado; el cambio de sentido va aparte en `legBreak` . *(Los viejos `parking` / `maniobra` / `approach` ya no se producen.)*
`isStop` / `stopDuration` / `stopRadius`	parada: si es stop, cuánto frena, radio
`targetThrottle/Brake/Steering`	pedales grabados (sólo cuentan si `hasInputData=1` ; el volante grabado va en la cinta, no acá)
`hasInputData`	`1` = la ruta trae tus pedales grabados — **el wizard siempre escribe 1**, y el cerebro 1.2 los usa como punto de partida (§9); `0` = sin pedales grabados (rutas viejas o importadas de v1)
`name`	etiqueta del punto (las paradas)

Apéndice B — Para desarrolladores: extender con código

Si querés ir más allá del JSON —sumar un verbo, un trigger o un comportamiento— acá están los puntos de extensión. Es **Enforce** (el scripting de DayZ).

B.1 El corazón: cómo el framework maneja el vehículo

El framework inyecta los inputs en el `OnInput` de un `modded class CarScript`, **después** del de eAI (override-last):

```
modded class CarScript {
    override void OnInput(float dt) {
        super.OnInput(dt);                    // eAI corre su lógica (incluido ShiftTo(FIRST))
        if (!GetGame().IsServer()) return;
        BZBusService srv = BZBusService.GetRunnerForCar(this);   // el runner DUEÑO de este vehículo
        if (!srv) return;
        // (si maneja un jugador real, el framework se hace a un lado)
        if (srv.SampleDrive12(this, dt)) return;   // cerebro 1.2: Sample a 40 Hz + aplicar los inputs
        // ... si no, el camino del cerebro 1.1 ...
    }
}
```

> **El breakthrough fue ese orden:** eAI fuerza 1ra cada frame; nuestro override corre DESPUÉS y pone el gear/inputs correctos.

Superficie que el framework **escribe** sobre `Car`: `SetThrottle(0..1)`, `SetSteering(-1..1)`, `SetBrake`, `SetHandbrake`, `ShiftTo(gear)`, `EngineStart/Stop`. Y **lee**: `GetSpeedometer()`, `EngineGetRPM()`, `WheelGetContactPosition(i)` (de ahí sale el wheelbase), `WheelGetSurface(i)`, etc.

B.2 Agregar un verbo nuevo

Todos los verbos viven en `BZBusService.ExecuteAction(Car car, BZAction action, int evIdx)`, un `else if` por verbo. Patrón real del código:

```
private void ExecuteAction(Car car, BZAction action, int evIdx) {
    if (!action) return;
    string verb = action.verb;
    if (verb == "start_engine") {
        if (!car.EngineIsOn()) car.EngineStart();
    } else if (verb == "freeze_vehicle") {
        m_Frozen = true;
    }
    // ... sumás tu rama:
    } else if (verb == "mi_verbo") {
        // tenés a mano: car (el vehículo), action (sus campos: value, fvalue, msg, slot...),
        // m_WaypointIndex (dónde está). Hacé lo tuyo:
        car.SetHandbrake(1.0);
        BZBusLog.Info("[EVENT " + evIdx + "] mi_verbo ejecutado.");
    }
}
```

El `delay` (coreografía) ya lo maneja el dispatcher por vos (`CallLater`). Y listo: el modder ya puede poner `{ "verb": "mi_verbo" }` en sus `Events[]` . *(Para campos nuevos, agregalos a la clase `BZAction` — es plana, no rompe el parser.)*

B.3 Agregar un trigger nuevo

Los triggers se evalúan por `BZTrigger.type` en `BZBusService.IsTriggerFired` . Sumás el campo que necesite a la clase `BZTrigger` , su rama en `IsTriggerFired` (ej. `on_player_shoot` con su condición) **y el nombre en `TriggerTipoConocido`** : si no, al cargar la ruta avisa que el trigger es desconocido y nunca dispara.

B.4 El hook de Quest

La integración con Expansion-Quests entra por un `modded class MissionServer` :

```
modded class MissionServer {
    override void Expansion_OnQuestStart(ExpansionQuest quest) {
        super.Expansion_OnQuestStart(quest);
        BZBusService.GetInstance().OnQuestStart(quest);   // el framework "escucha" el quest
    }
}
```

Desde `OnQuestStart` , el framework **pollea** `ExpansionQuestModule…QuestPatrolExists(questID, patrols)` para alcanzar los bots vivos (son lazy por proximidad). *(La subclase del objetivo `ExpansionQuestObjective*Event` NO compila en nuestro scope → por eso se usa el hook de MissionServer + poll.)*

B.5 Gotchas de Enforce (los que nos pegaron)

Enforce no es C# ni C++; estas trampas costaron horas:

- **No hay ternario** `?:` ni `if` multilínea con `&&` al inicio de línea → "Syntax error". Usá `if/else` o bools intermedios.
- `new Clase(args)` **revienta** → creá vacío y seteá los campos.
- **No moddees clases del engine** (`CGame` , etc.) → rompe al cargar.
- **"Formula too complex"** a ~9 operandos con `+` → dividí en varias líneas con `+=` .
- **Scope de ramas hermanas:** variables en distintas ramas de un `if/else` (o `case`) **chocan** (no tienen scope propio como en C++) → renombralas o hoisteá.
- **Aritmética inline en concat de string** (`"x" + (seat-1)`) rompe → hoisteá a una `int` antes.
- **AddonBuilder NO valida Enforce** — compila igual con errores de sintaxis; recién fallan al **cargar el server**. Confirmá siempre en el RPT.

> Estos gotchas (y más) van también al **AI knowledge pack**, para que una IA te avise antes de que los pises.

B.6 — Integrar BZ_AutoDrive como dependencia de tu mod

Como es open source, podés construir **tu propio mod sobre** BZ_AutoDrive: que tu mod de misiones, eventos o transporte use el framework para el **manejo**, sin reimplementar nada. Hay dos formas de acoplarte:

(a) Como dependencia + llamando su API. En el `config.cpp` de tu mod, dependé del addon:

```
class CfgPatches {
    class TuMod {
        requiredAddons[] = { "BZ_AutoDrive" };   // el addon del framework
    };
};
```

Y desde tu código, el servicio es un singleton — le pedís que spawnee/maneje una ruta tuya:

```
BZBusService srv = BZBusService.GetInstance();
srv.RespawnFromPath("BZBusRoute_MiRuta.json");   // spawnea el vehículo y maneja esa ruta
// (RespawnAs(clase) para cambiar el vehículo en runtime)
```

Tu mod decide **cuándo** (un trigger tuyo, un horario, un evento de tu mod); el framework pone el **manejo**. Eso usa el servicio **principal**, que maneja una ruta por vez. Para manejar **varios vehículos a la vez** (una línea de buses, un convoy, varios taxis), usá un runner por vehículo: está todo en **B.7**.

(b) Extendiéndolo con `modded class` (B.1–B.4): sumás verbos, triggers o hooks propios sin tocar el código del framework.

> *Ejemplo:* tu mod de misiones, al iniciar una, llama a `RespawnFromPath` para que un convoy salga manejando — vos ponés la lógica de misión, el framework el vehículo. (Misma división que con Quest, pero desde **tu** mod.)

> *Naming:* el addon se llama **`BZ_AutoDrive`**; el mod-demo del bus (el servicio costero) es contenido aparte que lo usa. Es el nombre que va en `requiredAddons`.

B.7 — La API del runner: manejar una flota desde tu mod

Cada `BZBusService` es un **runner**: un vehículo, su chofer y su ruta. El servicio principal (`GetInstance()`) es uno más. Todo esto corre **en el server**, sin RPC. Tu mod lanza runners, les consulta el estado una vez por segundo y decide qué hacer; el framework maneja.

B.7.1 — Dos formas de lanzar un vehículo

A) Un viaje: **`BZBusService.PlayRouteByName(string fname, bool holdAtStops = true)`.** Devuelve el runner, o `null` si la ruta no está en la lista del perfil. No duplica: si esa ruta ya está corriendo, devuelve ese runner. Usa el principal si está libre; si no, crea uno nuevo. Un runner por archivo de ruta.

B) Un servicio: un runner propio por vehículo, varios sobre la misma ruta:

```
BZBusService r = new BZBusService();
BZBusService.GetRunners().Insert(r);   // el registro guarda la UNICA referencia fuerte
r.SetHoldAtStops(true);                // espera en cada parada hasta ResumeFromHold()
r.SetEtiqueta("101");                  // ANTES de cargar: firma el log como <ruta>#101
r.RespawnFromPath("BZBusRoute_MiLinea.json");   // carga la ruta, crea vehiculo y chofer en el wp 0 y arranca
if (!r.RunnerIsActive() && !r.RunnerEsperandoCabecera()) {
    BZBusService.UnregisterRunner(r);  // no arranco de verdad: limpiar
    return false;
}
// ... consultar el estado (B.7.2) y controlar el viaje (B.7.3) ...
BZBusService.UnregisterRunner(r);      // al terminar: borra vehiculo y chofer y lo saca del registro
```

Lo que tenés que saber:

- **El registro guarda la referencia.** Un runner que no está en `GetRunners()` se pierde, y su vehículo queda manejando sin que el panel lo vea. `UnregisterRunner` borra primero el vehículo y el chofer, y después lo saca.
- `SetEtiqueta` **va antes de** `RespawnFromPath`: la carga ya escribe en el log con esa firma.
- **Si el punto de salida está ocupado** por otro vehículo, el runner **espera** y re-intenta cada 5 s. Mientras tanto no hay vehículo: `RunnerIsActive()` da `false` y `RunnerEsperandoCabecera()` da `true`. **No es una falla**: no lo trates como tal. No hay tope de espera; si el lugar queda ocupado para siempre, qué hacer lo decidís vos.
- **Qué pasa cuando la ruta se cierra** (`RunnerRouteEnded()`). Depende de cómo terminó y de cómo la lanzaste:
  - `ROUTE_COMPLETE` **de un runner lanzado con** `RespawnFromPath` (el camino B, y también el Reproductor): el vehículo **se retira** a los `EndHoldSeconds` segundos (0 = en el acto) y el runner **sale del registro**.
  - `ROUTE_COMPLETE` **de una ruta lanzada con** `PlayRouteByName` (o la ruta por defecto del server): se **re-lanza** a los `RespawnDelay` segundos (`0` vale 60 s).
  - `ROUTE_INCOMPLETE` **o** `STUCK`**:** el vehículo **queda quieto**. Retirarlo, reponerlo o dejarlo es de tu mod. **Desde la 1.3 hay una red:** si **nadie lee el fin** (`RunnerRouteEnded()`, `RunnerRouteCompleted()` o `RunnerEndReason()`) en **60 s**, el framework lo retira solo, como una ruta del panel (y nunca con un jugador a bordo: espera). Si tu mod **leyó el fin** queda a cargo: la red no lo toca, y el RPT avisa cada 5 minutos (`[C12] RED: FIN visto hace N s y el vehiculo sigue`) por si lo olvidaste. Si lo lanzaste desde el panel o es la ruta por defecto, concluye solo (se retira o respawnea como una completa).
  - Con `holdAtStops`, al llegar a la última parada el vehículo queda quieto en el hold hasta que lo sueltes.
- **Cuál usar:** A para un viaje que dispara otro sistema (un taxi, una misión); B para un servicio con varios vehículos y reglas propias.

B.7.2 — Consultar el estado

Método	Devuelve
`RunnerIsActive()`	`bool` : el vehículo existe y no está destruido
`RunnerWpIdx()` / `RunnerWpTotal()`	`int` : por qué punto de la ruta va / cuántos tiene
`RunnerPos()` / `RunnerSpeedKmh()`	posición y velocidad del vehículo
`RunnerAtStop()`	`bool` : está cumpliendo una parada
`IsHeld()`	`bool` : detenido en una parada o por `HoldRunner()` , esperando que lo sueltes. El paso de `false` a `true` es "llegó a una parada"
`IsHoldPendiente()`	`bool` *(desde la 1.2.5)*: el vehículo está frenando hacia una parada pedida (`HoldRunnerSuave`) o hacia la parada grabada de un `HoldRunner()` . Cuando se detiene pasa a `IsHeld()` ; si vuelve a `false` sin `IsHeld()` , la parada se canceló (el log dice por qué)
`RunnerFrenadaMs2()` / `RunnerCintaS()`	*(desde la 1.2.5)* la deceleración que usa `HoldRunnerSuave` si no le pasás una (la frenada medida de la grabación o, si no la trae, la del vehículo) / por qué metro de la grabación va. `-1` sin el cerebro 1.2
`RunnerIsPaused()`	`bool` : la pausa blanda (`SetPaused`). **No es lo mismo que el hold**
`RunnerRouteEnded()`	`bool` : el cerebro cerró la ruta; desde ahí nadie maneja el vehículo
`RunnerRouteCompleted()`	`bool` : y la cerró completa, con el veredicto estricto (en reposo a ≤ 0,25 m, sin choque contra un objeto, sin intervención física del AutoRecovery: §17). *(En la 1.2 era `true` también entre 0,25 m y 1 m: ya no)*
`RunnerReposoFinM()`	`float` : la distancia en reposo al punto final con la que se decidió el veredicto; `-1` mientras corre, y si la ruta terminó por otra vía (`STUCK` , `StopRunner`). Autos y motos *(desde la 1.2.1)*
`RunnerEndReason()`	`string` : vacío mientras corre; al cerrar, el motivo (`ROUTE_COMPLETE en wp N ...` , `ROUTE_INCOMPLETE ...` , `STUCK en wp N (escalera del AR agotada: declarado)`). Desde la 1.2.1 termina con `· REPOSO tras T s: a X m de la pose final (longitudinal …, lateral …)` y, en un `ROUTE_INCOMPLETE` , trae los motivos encadenados
`RunnerFellBackTo11()`	`bool` : la ruta no se pudo cargar para el cerebro 1.2 y la maneja el 1.1. El vehículo anda igual
`RunnerBloqueado()`	`bool` : el AutoRecovery está tratando de destrabarlo (§9.6). **Es un obstáculo para los demás**
`RunnerBloqueadoMotivo()` / `RunnerBloqueadoDesdeS()`	por qué (con el último choque, si hubo) / desde hace cuántos segundos
`RunnerRecuperando()` / `RunnerRecuperandoQue()`	`bool` : cualquier recuperación en curso (arranque en pendiente, alejarse, re-ubicarse, escalera) / cuál. Responde "¿va a poder moverse ahora?". Siempre que `RunnerBloqueado()` es `true` , éste también
`RunnerChoqueHaceS()`	`float` : segundos desde el último choque de ese vehículo (`-1` si no chocó). Sirve para que **los dos** vehículos de un choque aparezcan, aunque sólo uno quede bloqueado

Método	Devuelve
`RunnerEsperandoCabecera()` / `RunnerEsperandoCabeceraS()`	`bool`: espera que se libere su punto de salida / desde hace cuántos segundos

Con el cerebro 1.1 (`RunnerFellBackTo11()`), los de bloqueo, recuperación y choque devuelven "nada".

B.7.3 — Controlar el viaje

Método	Qué hace
`SetHoldAtStops(bool)`	modo "espera en cada parada": ponelo antes de cargar (`PlayRouteByName` lo hace por vos)
`HoldRunner()`	detiene y mantiene quieto el vehículo. Si hay una parada grabada a menos de 8 m, lo deja llegar hasta ella y lo detiene ahí
`HoldRunnerSuave(float distM, float aMs2)`	*(desde la 1.2.5)* lo detiene **en cualquier punto**, a `distM` metros de recorrido: sigue su línea y frena con una curva suave (`aMs2` en m/s²; `0` = `RunnerFrenadaMs2()`) hasta quedar detenido (`IsHeld()`). Devuelve `false` si no puede (sin el cerebro 1.2, arrancando o recuperándose, en moto, o el punto ya quedó atrás): ahí usá `HoldRunner()`
`ResumeFromHold()`	lo suelta y sigue la ruta (también cancela una parada suave pendiente)
`SetPaused(bool)`	la pausa blanda: congela el manejo (pensada para teletransportar o preparar). No pasa por las paradas
`SetVehiclePartsLocked(bool)` / `VehiclePartsLocked()`	vehículo inviolable o no (lo mismo que `set_vehicle_parts`). Aplica en el acto. Si tu mod entrega el vehículo a un jugador, desbloquealo
`HornBeep()`	un bocinazo corto
`StopRunner()`	termina: borra vehículo y chofer. Para un runner propio usá `UnregisterRunner(r)` , que además lo saca del registro

⚠ **Un `HoldRunner()` hasta 8 m antes de una parada grabada la da por cumplida**: al soltarlo, el vehículo no vuelve a frenar ahí. Si querés que después de tu detención se cumpla esa parada, detenelo **en** la parada.

La parada suave (`HoldRunnerSuave` , desde la 1.2.5) es para detener el vehículo donde la grabación no tiene una parada, sin que se vaya de su línea: `HoldRunner()` lejos de una parada grabada frena en el acto con la dirección derecha, y en una curva rápida el vehículo se sale. Reglas:

- **Una parada grabada antes del punto manda:** para ahí.
- **Una reversa o el fin de la ruta antes del punto** lo recortan a 1 m antes.
- **Un `HoldRunner()` durante la frenada** la convierte en frenada inmediata (emergencia). Otro `HoldRunnerSuave()` reemplaza el punto.
- **Una recuperación durante la frenada** (AutoRecovery, re-ubicación, atasco) la cancela: `IsHoldPendiente()` vuelve a `false` sin `IsHeld()` y el log dice por qué. Volvé a pedirla o usá `HoldRunner()` .
- **La moto no la soporta** (devuelve `false`).
- Para saber cuándo empezar a frenar podés usar `RunnerFrenadaMs2()` : con esa deceleración, desde `v` km/h hacen falta unos `(v/3,6)² / (2·a)` metros.

B.7.4 — Identidad y registro

Método	Devuelve
`static BZBusService.GetInstance()`	el servicio principal (también es un runner)
`static BZBusService.GetRunners()`	el registro de runners
`static BZBusService.GetRunnerForCar(EntityAI car)`	el runner dueño de ese vehículo, o `null`
`static BZBusService.UnregisterRunner(BZBusService r)`	borra vehículo y chofer y saca el runner del registro (el principal no se saca nunca)
`GetBus()` / `GetDriver()`	el vehículo y el chofer
`GetRouteName()`	el archivo de la ruta
`SetEtiqueta(string)` / `Etiqueta()` / `NombreLog()`	la etiqueta del vehículo / su firma en el log (`<ruta>#<etiqueta>`)
`GetOrigin()`	de dónde salió (config, taxi, lista, misión)
`GetInstance().IsControlPanelAdmin(string steamId)`	`true` si ese SteamID64 está en `AdminSteamIDs` (§13.1). Si tu mod tiene su propia interfaz de operador, usá esta lista: el dueño del server mantiene una sola

B.7.5 — Eventos

El cerebro emite estos eventos y **el runner los consume él mismo** (la cola es interna: tu mod no la lee). Te sirven para entender el log y los estados de B.7.2:

Valor	Nombre	Significado	¿Cierra la ruta?
1	`ARRIVED`	llegó a una parada intermedia	no
2	`ROUTE_COMPLETE`	llegó al final y cumple el veredicto (§17). Sale **en el reposo**: hasta ~3,5 s después de llegar	sí
3	`STUCK`	trabado, con la escalera del AutoRecovery agotada	sí
4	`ROUTE_INCOMPLETE`	terminó sin cumplir el veredicto: quedó a más de 0,25 m en reposo, chocó contra un objeto, el AutoRecovery tuvo que moverlo, una maniobra no cerró, o un tramo se cerró sin progreso. También sale en el reposo	sí
5	`BLOQUEADO`	el AutoRecovery empezó a destrabarlo (un aviso por vez)	**no**

Tu mod ve lo mismo consultando los métodos de B.7.2 una vez por segundo, que es lo que hacen los mods que ya lo usan.

B.7.6 — El log: qué mirar y cómo contarlo

Línea	Qué es
`[C12] BLOQUEADO en wp N · <motivo>` / `[C12] BLOQUEO terminado tras X s`	se abre / se cierra un intento de destrabarlo. Por vehículo, tiene que haber tantas de una como de la otra
`[BZDrive] *** CHOQUE #N *** ...` / `... episodio cerrado: K impactos ...`	un choque (uno por episodio: los golpes a menos de 5 s del anterior se cuentan juntos) / el resumen, si hubo más de un golpe
`[BZDrive] *** BACHE #N ***`	una frenada brusca por el terreno, sin contacto: no es un choque
`[C12] REPOSO tras T s: a X m de la pose final (longitudinal a m, lateral l m)`	el vehículo quedó quieto al final; con esto se decide el veredicto *(1.2.1)*
`[C12] FIN: ROUTE_COMPLETE en wp N · cierre: … · REPOSO tras …` / `*** ROUTE_INCOMPLETE en wp N · <motivos> · …`	el cierre de la ruta, con el veredicto y sus motivos
`[C12] FIN: STUCK en wp N (<motivo>: declarado)`	la ruta se cerró trabada, con el motivo real
`[C12] LEG N cerrado · <motivo> · remFr X m · lat Y m · … · la ultima parada grabada de la fase (cuadro K) queda a Z m`	se cerró un tramo (útil en los cambios de sentido). `lat` es el desvío de costado; la distancia a **la última parada grabada** es la referencia fiel de dónde paraste vos
`[SpawnGuard] wp0 ocupado ...: el runner ESPERA ...` / `cabecera libre tras T s ...`	espera por el punto de salida ocupado / se liberó
`[SpawnBus] quedo: ...` / `[Continuar] quedo: ...`	dónde quedó el vehículo 3 s después de aparecer, y cuánto se desvió de lo pedido
`[AR] <ruta> wp N · FISICO (<evidencia>) · escalon K: ...`	cada paso de la escalera; también en `AR_log.csv`

Para contar: los choques **por episodio**, los problemas **por lugar** (agrupá por posición, no por número de waypoint: rutas distintas numeran distinto), y los bloqueos **por vehículo**. ⚠ Los milisegundos del RPT no llevan ceros a la izquierda: `19:44:41.27` es 41,027 s.

B.7.7 — Lo que garantiza el framework y lo que es de tu mod

- **El framework garantiza que el vehículo no se quede detenido para siempre**: detecta la traba y la destraba (§9.6), y sólo se rinde con la escalera agotada.
- **Y que ningún jugador lo deje detenido desarmándolo**: mientras lo maneja, el vehículo viene **inviolable** (`VehiclePartsLocked`): nadie le saca ruedas ni piezas. Si tu mod lo entrega, desbloquealo (`SetVehiclePartsLocked(false)`).
- **Qué hacer mientras tanto es de tu mod**: dejar de cederle el paso a un vehículo bloqueado, retirarlo, esperarlo, avisar a los jugadores.
- **Lo que tu mod decide, el framework no lo adivina**: cuándo lanzar, dónde esperar, a quién ceder el paso, cuándo retirar. Por ejemplo, largar un vehículo con el punto de salida ocupado lo deja esperando (B.7.1); decidir no largarlo es de tu mod.

B.8 — Build & deploy your fork

Para solo usar el mod: suscribite en el Workshop — ya viene firmado, no tenés que compilar nada.

Para compilar tu fork: empaquetá el PBO como prefieras —**DayZ Tools** (AddonBuilder) o tu propio pipeline— y firmalo con tu **propia** llave (`.biprivatekey`). No se distribuye ningún script de build: cada modder empaqueta a su manera. Copiá tu `.bikey` a la carpeta `keys\` del server; sino el cliente patea con "faltan PBO".

El único gotcha que conviene recordar: cuando renombrás, movés o borrás un `.c` , **limpiá la carpeta `temp/` de AddonBuilder antes de reempaquetar** — sino empaqueta versiones viejas ("zombie") del código. Después de eso, validá RPT sin errores + smoke test ingame antes de subir al Workshop.